

**PENGEMBANGAN ARSITEKTUR AKUISISI DAN
EKSTRAKSI DOKUMEN BERBASIS *PLUGIN* PADA SISTEM
OMNI-RAG MULTIDOMAIN**

Laporan Tugas Akhir

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

DEWANTORO TRIATMOJO

NIM: 13522011



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Agustus 2026

**PENGEMBANGAN ARSITEKTUR AKUISISI DAN
EKSTRAKSI DOKUMEN BERBASIS *PLUGIN* PADA SISTEM
OMNI-RAG MULTIDOMAIN**

Laporan Tugas Akhir

Oleh

DEWANTORO TRIATMOJO

NIM: 13522011

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir
di Bandung, pada tanggal 24 Agustus 2026

Mengetahui,

Pembimbing I,

Pembimbing II,

Dr. Agung Dewandaru, S.T., M.Sc.

NIP. 122110013

I Wayan Gunada, S.H., M.H.

NOPEG. 107000001

LEMBAR IDENTITAS
TUGAS AKHIR CAPSTONE

Judul Proyek TA : OMNI-RAG

Project Owner : I Wayan Gunada, S.H., M.H.

Anggota Tim dan Pembagian Peran:

No.	NIM	Nama	Peran
1	13522011	Dewantoro Triatmojo	Pengembangan Komponen Akuisisi dan Ekstraksi Data
2	13522057	Moh Fairuz Alauddin Yahya	Pengembangan Komponen <i>Chunker</i> , <i>Indexer</i> , dan <i>Knowle- dge Graph</i>
3	13522058	Imanuel Sebastian Girsang	Pengembangan Komponen <i>Explai- nable AI</i> (XAI)
4	13522081	Albert	Pengembangan Komponen <i>Retrieval</i>

Bandung, 24 Agustus 2026

Mengetahui,

Pembimbing I,

Pembimbing II,

Dr. Agung Dewandaru, S.T., M.Sc.

NIP. 122110013

I Wayan Gunada, S.H., M.H.

NOPEG. 107000001

LEMBAR PERNYATAAN PENGGUNAAN AI

Dengan ini, saya menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran Tugas Akhir. Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Sebutkan <i>tools</i> yang digunakan	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan <i>tools</i> seperti Grammarly, Paperpal, Quillbot, Grammarbot, LanguageTool, ChatGPT, Google Gemini, atau sejenisnya.	Ya	ChatGPT	Rendah
Pembuatan Teks: menggunakan <i>tools</i> seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, Jasper, Jenni AI, atau sejenisnya.	Ya	ChatGPT	Rendah
Bantuan Diskusi dalam Penyusunan Konten: menggunakan <i>tools</i> seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, atau sejenisnya.	Ya	ChatGPT	Rendah
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan <i>tools</i> seperti Craiyon, DALL-E, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-	-

Saya juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada Tugas Akhir telah dinyatakan di dalam surat ini.

Bandung, 24 Agustus 2026

Dewantoro Triatmojo
NIM 13522011

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan Laporan Tugas Akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung bagian Penegakan Norma Akademik dan Kemahasiswaan khususnya Pasal 2.1 dan Pasal 2.2.

Bandung, 24 Agustus 2026

Dewantoro Triatmojo

NIM 13522011

ABSTRAK

PENGEMBANGAN ARSITEKTUR AKUISISI DAN EKSTRAKSI DOKUMEN BERBASIS *PLUGIN* PADA SISTEM OMNI-RAG MULTIDOMAIN

Oleh:

Dewantoro Triatmojo

NIM: 13522011

[TODO: abstrak ditulis paling akhir, setelah Bab I–V selesai.]

Kata kunci: [TODO]

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmat-Nya sehingga laporan tugas akhir yang berjudul “Pengembangan Arsitektur Akuisisi dan Ekstraksi Dokumen Berbasis *Plugin* pada Sistem OMNI-RAG Multi-domain” dapat diselesaikan dalam rangka memenuhi syarat kelulusan tingkat sarjana. Dalam pengerjaan tugas akhir ini, penulis memperoleh banyak dukungan dari berbagai pihak. Untuk itu, penulis mengucapkan terima kasih kepada:

1. Dr. Agung Dewandaru, S.T., M.Sc. dan I Wayan Gunada, S.H., M.H. selaku dosen pembimbing atas bimbingan, arahan, dan kesabarannya selama pengerjaan tugas akhir.
2. Seluruh dosen Program Studi Teknik Informatika ITB atas ilmu pengetahuan yang telah diberikan.
3. Rekan-rekan satu tim *capstone* “OMNI-RAG” atas kerja sama dan kolaborasinya selama pengembangan sistem.
4. Kedua orang tua dan keluarga penulis atas dukungan moral, motivasi, dan doa yang tiada henti.
5. Max Kusnadi selaku manajer dan Nitin Pillai selaku pemimpin tim di Xendit atas izin cuti selama dua minggu sehingga penulis dapat berfokus menyelesaikan tugas akhir ini.
6. Malik Irawan dan rekan-rekan asisten Laboratorium Teknik Produksi FTMD yang telah menemani penulis mengerjakan tugas akhir hingga larut malam.
7. Seluruh pihak lain yang tidak dapat penulis sebutkan satu per satu yang telah membantu dalam proses pengerjaan tugas akhir ini.

Penulis menyadari bahwa laporan tugas akhir ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan. Semoga laporan ini dapat bermanfaat bagi pembaca dan pengembangan penelitian selanjutnya.

Bandung, 24 Agustus 2026

Dewantoro Triatmojo

NIM 13522011

DAFTAR ISI

ABSTRAK	vi
KATA PENGANTAR	vii
DAFTAR ISI	ix
DAFTAR GAMBAR	xiv
DAFTAR TABEL	xvii
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	4
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian	4
I.4 Batasan Masalah	6
I.5 Metodologi	7
I.6 Sistematika Pembahasan	8
BAB II KAJIAN PUSTAKA	10
II.1 Sistem <i>Retrieval-Augmented Generation</i> dan Tahap Ingesti	10
II.1.1 Arsitektur dan Paradigma RAG	10
II.1.2 Mutu Ingesti sebagai Batas Atas Mutu Jawaban	10
II.1.3 RAG Lintas Domain	11
II.2 Arsitektur Perangkat Lunak untuk <i>Pipeline</i> yang Dapat Diperluas ...	11
II.2.1 Gaya Arsitektur <i>Pipeline</i>	11
II.2.2 Arsitektur <i>Plugin</i>	12
II.2.3 Representasi Terpadu pada Sistem Pemrosesan Dokumen....	12
II.3 Akuisisi Dokumen dari Sumber Web	13
II.3.1 Arsitektur <i>Web Crawler</i>	13
II.3.2 Protokol Pengecualian Robot	13
II.3.3 <i>Tool</i> Pengumpulan Data Web	14
II.3.4 Pendeteksian Berkas Duplikat dengan <i>Hash</i>	14
II.4 Ekstraksi Konten dari Dokumen PDF	15
II.4.1 Halaman <i>Born-Digital</i> dan Halaman Hasil Pindai	15
II.4.2 <i>Document Outline</i> pada PDF	15

II.4.3	Pustaka Ekstraksi Teks <i>Native</i>	15
II.4.4	Keputusan <i>Routing</i> OCR	16
II.4.5	Model Mutakhir untuk Parsing Dokumen	17
II.4.6	<i>Model Serving</i> OCR pada Apple Silicon.....	18
II.4.7	<i>Layout Analysis</i> dan <i>Reading Order</i>	19
II.5	Ekstraksi Struktur Hierarkis Dokumen	19
II.5.1	Peran Struktur bagi Sistem <i>Retrieval</i>	19
II.5.2	Pemulihan Struktur Hierarkis Dokumen Multihalaman.....	20
II.5.3	Analisis Leksikal dan Pengenalan Penanda.....	20
II.5.4	<i>Finite-State Machine</i> untuk Urutan Bagian Dokumen	21
II.5.5	<i>Parsing</i> Hierarki Berbasis <i>Stack</i>	21
II.5.6	Standar Representasi Dokumen Hukum	21
II.5.7	Struktur Peraturan Perundang-undangan Indonesia	22
II.5.8	Dokumentasi Produk untuk <i>Customer Service</i>	24
II.5.9	Struktur Manual Pengguna NETGEAR	24
BAB III	ANALISIS MASALAH DAN RANCANGAN SOLUSI.....	26
III.1	Analisis Masalah	26
III.1.1	Posisi Komponen Akuisisi dan Ekstraksi	26
III.1.2	Analisis Sumber Data	27
III.1.2.1	Portal Peraturan BPK.....	27
III.1.2.2	Google Drive	34
III.1.2.3	Perbandingan dan Konsekuensi Rancangan	36
III.1.3	Analisis Dokumen Peraturan Perundang-undangan	37
III.1.4	Analisis Dokumen Manual Produk.....	40
III.1.5	Analisis Kebutuhan Ekstensibilitas	41
III.1.6	Kebutuhan Fungsional dan Non-Fungsional	41
III.2	Analisis Solusi	43
III.2.1	Arsitektur	43
III.2.1.1	Arsitektur <i>Plugin</i>	43
III.2.1.2	Arsitektur <i>Pipeline</i>	44
III.2.2	Solusi <i>Datasource</i> dan <i>Scraper Plugin</i>	44
III.2.2.1	<i>Datasource</i> dan Kontrak <i>Scraper</i>	44

III.2.2.2	Pemisahan Enumerasi, Pengunduhan, dan <i>Ingestion</i>	45
III.2.2.3	Validasi, Deduplikasi, dan Penyimpanan	46
III.2.2.4	Penjadwalan, <i>Concurrency</i> , Pembatalan, dan Sta- tus <i>Run</i>	46
III.2.2.5	Fan-out ke Profil dan Sinkronisasi Dokumen	47
III.2.3	Solusi <i>Parser Plugin</i>	48
III.2.3.1	Kontrak <i>Parser Plugin</i> dan Dokumen Kanonik	48
III.2.3.2	Ekstraksi <i>Legal Parser</i> per Halaman.....	48
III.2.3.3	OCR dan Deteksi <i>Layout</i> pada <i>Legal Parser</i>	49
III.2.3.4	Pemrosesan <i>Legal Parser</i> Berbasis Penanda	50
III.2.3.5	<i>Generic Outline Parser</i> Berbasis <i>Bookmark</i>	50
III.2.3.6	Verifikasi, Peringatan, dan Kegagalan Parser.....	51
III.2.3.7	Materialisasi Dokumen Kanonik dan <i>Handoff</i>	53
III.3	Rancangan Solusi	54
III.3.1	Rancangan Arsitektur Implementasi	54
III.3.1.1	Arsitektur <i>Deployment</i> dan <i>Service</i>	54
III.3.1.2	Arsitektur Modul dan Komponen	56
III.3.2	Rancangan Data, <i>Schema</i> , dan Kontrak	57
III.3.2.1	ERD Basis Data.....	57
III.3.2.2	Skema <i>Message</i> dan Artefak <i>Pipeline</i>	65
III.3.2.3	Skema Konfigurasi <i>Plugin</i>	68
III.3.2.4	Kontrak API.....	71
III.3.3	Rancangan <i>Lifecycle</i> dan <i>State</i>	74
III.3.3.1	<i>Lifecycle Datasource</i>	74
III.3.3.2	<i>Lifecycle Scrape Run</i>	75
III.3.3.3	<i>Lifecycle Scrape Entry</i>	75
III.3.3.4	<i>Lifecycle Pipeline Run</i>	76
III.3.4	Rancangan Alur Interaksi	76
III.3.4.1	Pembuatan <i>Datasource</i>	77
III.3.4.2	Perubahan Konfigurasi <i>Datasource</i>	78
III.3.4.3	Pengarsipan <i>Datasource</i>	79
III.3.4.4	Pengaktifan Kembali <i>Datasource</i>	81

III.3.4.5	Pembuatan <i>Profile</i> dan Pemilihan <i>Parser</i>	82
III.3.4.6	Unggah Dokumen Manual	85
III.3.4.7	Pemicu <i>Scraper</i> Manual	86
III.3.4.8	Pemicu <i>Scraper</i> Terjadwal	88
III.3.4.9	Eksekusi <i>BPK Peraturan Scraper</i>	90
III.3.4.10	Eksekusi <i>Google Drive Scraper</i>	92
III.3.4.11	<i>Parsing</i> dan <i>Handoff</i> ke <i>Chunker</i>	95
III.3.4.12	Sinkronisasi Dokumen Lama ke <i>Profile</i>	96
III.3.4.13	Pengulangan <i>Parsing</i>	97
III.3.4.14	Pembatalan <i>Scrape Run</i>	98
BAB IV	EKSPERIMEN DAN EVALUASI	101
IV.1	Evaluasi <i>Datasource</i> dan <i>Scraper</i> Peraturan BPK	101
IV.1.1	Eksperimen <i>Scraper</i> Portal Peraturan BPK	101
IV.1.1.1	Tujuan dan Pertanyaan Evaluasi	101
IV.1.1.2	Rancangan Eksperimen	102
IV.1.1.3	Perbandingan <i>Throughput</i> dan Durasi	103
IV.1.1.4	Bentuk Penyelesaian <i>Run</i>	104
IV.1.1.5	Hasil <i>Run</i> Representatif dan Kategori <i>Failure</i>	104
IV.1.1.6	Deduplikasi dan Identitas Sumber	106
IV.1.1.7	Kesimpulan Eksperimen	107
IV.1.2	Analisis Data Korpus BPK	108
IV.1.2.1	Ruang Lingkup dan Korpus Analisis Data	108
IV.1.2.2	Komposisi Korpus Peraturan	109
IV.1.2.3	Kelengkapan <i>Metadata</i>	110
IV.1.2.4	Distribusi Ukuran Berkas dan <i>Storage</i>	111
IV.1.2.5	Relasi Antarperaturan	111
IV.2	Evaluasi <i>Datasource</i> dan <i>Scraper</i> Google Drive	113
IV.2.1	Tujuan dan Pertanyaan Evaluasi	113
IV.2.2	Rancangan Evaluasi	113
IV.2.3	Hasil Pemrosesan	114
IV.2.4	Analisis Fleksibilitas Sumber	114
IV.3	Evaluasi <i>Legal Parser</i>	115

IV.3.1	Eksperimen Klasifikasi Halaman <i>Vector</i> dan <i>Scan</i>	116
IV.3.1.1	Tujuan dan Pertanyaan Evaluasi	116
IV.3.1.2	Rancangan Eksperimen.....	116
IV.3.1.3	Hasil Evaluasi pada Sampel Berlabel.....	117
IV.3.1.4	Penerapan Ambang pada Seluruh Korpus	118
IV.3.1.5	Kesimpulan Eksperimen.....	119
IV.3.2	Eksperimen <i>Legal Parser</i> pada Korpus ALL_VECTOR	119
IV.3.2.1	Tujuan dan Pertanyaan Evaluasi	119
IV.3.2.2	Rancangan Eksperimen.....	120
IV.3.2.3	Hasil dan Analisis Pengaruh Toleransi Pengga- bungan.....	121
IV.3.2.4	Analisis dan Kesimpulan Eksperimen	124
IV.3.3	Eksperimen <i>Legal Parser</i> pada Korpus ALL_SCAN	125
IV.3.3.1	Tujuan dan Pertanyaan Evaluasi	125
IV.3.3.2	Rancangan Eksperimen.....	125
IV.3.3.3	Hasil Keutuhan Struktur	126
IV.3.3.4	Biaya Pemrosesan dan Kegagalan OCR	127
IV.3.3.5	Analisis dan Kesimpulan Eksperimen	129
IV.4	Evaluasi <i>Generic Outline Parser</i> pada Domain <i>Customer Service</i>	129
IV.4.1	Tujuan dan Pertanyaan Evaluasi.....	129
IV.4.2	Rancangan Evaluasi.....	130
IV.4.3	Hasil Pemrosesan	130
IV.4.4	Analisis Anchoring, Kinerja, dan Kesimpulan	131
BAB V	KESIMPULAN DAN SARAN.....	133
V.1	Kesimpulan.....	133
V.2	Saran.....	135
DAFTAR PUSTAKA		142

DAFTAR GAMBAR

Gambar III.1. Kedudukan tahap akuisisi dan ekstraksi dokumen di dalam alur sistem OMNI-RAG	26
Gambar III.2. <i>Response</i> HTML pertama halaman hasil pencarian sudah memuat seluruh isi yang ditampilkan	28
Gambar III.3. Satu entri pada halaman hasil pencarian portal peraturan BPK beserta kendali <i>pagination</i> di bawahnya	29
Gambar III.4. Kartu metadata dan kartu status pada halaman rincian Perbup Kabupaten Magetan Nomor 27 Tahun 2020 (peraturan.bpk.go.id/Details/249238)	31
Gambar III.5. Empat keadaan ketersediaan berkas pada halaman rincian peraturan	32
Gambar III.6. Tiga tipe halaman; kotak merah menandai <i>text span</i> yang dapat diekstraksi langsung	37
Gambar III.7. Halaman pertama Peraturan Bupati Langkat Nomor 37 Tahun 2017 (peraturan.bpk.go.id/Details/100120)	39
Gambar III.8. Kedudukan destinasi sebuah <i>outline item</i> terhadap baris judul pada awal bab manual NETGEAR R7000	41
Gambar III.9. Arsitektur <i>deployment</i> dan <i>service</i> pada alur <i>datasource</i> sampai <i>parser</i>	54
Gambar III.10. Komponen modul dan kontrak pada <i>datasource</i> , <i>profile</i> , dan <i>parser</i>	56
Gambar III.11. ERD entitas <i>datasource</i> , <i>profile</i> , <i>plugin</i> , dan <i>monitoring pipeline</i>	58
Gambar III.12. <i>Lifecycle datasource</i>	75
Gambar III.13. <i>Lifecycle scrape run</i>	75
Gambar III.14. <i>Lifecycle scrape entry</i>	76
Gambar III.15. <i>Lifecycle pipeline run</i>	76
Gambar III.16. Formulir pembuatan <i>datasource upload-only</i> pada <i>Admin UI</i>	77

Gambar III.17.	Alur interaksi pembuatan <i>datasource</i>	78
Gambar III.18.	Detail <i>datasource</i> aktif pada <i>Admin UI</i>	79
Gambar III.19.	Alur interaksi perubahan konfigurasi <i>datasource</i>	79
Gambar III.20.	Daftar <i>datasource</i> dengan status <i>active</i> dan <i>archived</i> pada <i>Admin UI</i>	80
Gambar III.21.	Alur interaksi pengarsipan <i>datasource</i>	81
Gambar III.22.	Alur interaksi pengaktifan kembali <i>datasource</i>	81
Gambar III.23.	Pemilihan <i>datasource</i> dan <i>parser</i> pada <i>profile wizard</i>	82
Gambar III.24.	Konfigurasi <i>generic outline parser</i> pada <i>profile wizard</i>	83
Gambar III.25.	Konfigurasi <i>legal parser</i> pada <i>profile wizard</i>	84
Gambar III.26.	Alur interaksi pembuatan <i>profile</i> dan pemilihan <i>parser</i>	84
Gambar III.27.	Tab <i>Documents</i> pada detail <i>datasource</i>	85
Gambar III.28.	Dialog unggah dokumen pada <i>Admin UI</i>	86
Gambar III.29.	Alur interaksi unggah dokumen manual sampai <i>fan-out</i>	86
Gambar III.30.	Tab <i>Scraper</i> dengan pemicu manual dan riwayat <i>scrape run</i> ..	87
Gambar III.31.	Alur interaksi pemicu <i>scraper</i> manual sampai <i>fan-out</i>	88
Gambar III.32.	Konfigurasi jadwal dan <i>stop condition scraper</i> pada <i>Admin UI</i>	89
Gambar III.33.	Alur interaksi pemicu <i>scraper</i> terjadwal sampai <i>fan-out</i>	90
Gambar III.34.	Konfigurasi <i>BPK Peraturan Scraper</i> pada <i>Admin UI</i>	91
Gambar III.35.	Alur internal <i>BPK Peraturan Scraper</i> pada <i>Scraper Worker</i> ..	92
Gambar III.36.	Konfigurasi <i>Google Drive Scraper</i> pada <i>Admin UI</i>	93
Gambar III.37.	Konfigurasi plugin <i>Google Drive Scraper</i> pada <i>Admin UI</i>	94
Gambar III.38.	Alur internal <i>Google Drive Scraper</i> pada <i>Scraper Worker</i>	95
Gambar III.39.	Alur interaksi <i>Parser Worker</i> , <i>plugin parser</i> , <i>OCR/layout</i> , dan <i>handoff</i> ke <i>chunker</i>	96
Gambar III.40.	Alur interaksi sinkronisasi dokumen lama ke <i>profile</i>	97
Gambar III.41.	Detail dokumen pada <i>profile</i> dengan kontrol <i>retry</i> parser.....	98
Gambar III.42.	Alur interaksi pengulangan tahap <i>parser</i>	98
Gambar III.43.	Detail <i>scrape run</i> dan riwayat perubahan <i>state</i> pada <i>Admin UI</i>	99
Gambar III.44.	Alur interaksi pembatalan <i>scrape run</i>	100

Gambar IV.1. Perbandingan durasi dan <i>throughput</i> pada konfigurasi <i>scraper</i> BPK.....	103
Gambar IV.2. <i>Progress</i> penyelesaian entri dan akumulasi <i>byte</i> tersimpan pada keenam <i>run</i> BPK.....	104
Gambar IV.3. Hasil <i>skip</i> pada <i>run</i> representatif <i>scraper</i> BPK.....	105
Gambar IV.4. Kategori <i>failure</i> , kestabilan sumber, dan tipe berkas yang tidak didukung pada <i>scraper</i> BPK	106
Gambar IV.5. Deduplikasi berdasarkan kesamaan isi pada eksperimen <i>scraper</i> BPK.....	107
Gambar IV.6. Komposisi korpus peraturan BPK berdasarkan <i>metadata</i>	109
Gambar IV.7. Kelengkapan dan variasi <i>metadata</i> korpus BPK	110
Gambar IV.8. Distribusi ukuran berkas pada korpus BPK.....	111
Gambar IV.9. Relasi antarperaturan pada <i>metadata</i> korpus BPK	112
Gambar IV.10. Hasil evaluasi integrasi <i>scraper</i> Google Drive	115
Gambar IV.11. Evaluasi grid ambang <i>classifier</i> ; bintang menandai kandidat terpilih.....	118
Gambar IV.12. Komposisi hasil <i>routing</i> pada seluruh korpus peraturan	119
Gambar IV.13. <i>Clean rate</i> pada grid toleransi penggabungan jalur <i>native</i>	121
Gambar IV.14. Komposisi hasil pemrosesan pada grid profil <i>legal parser</i> jalur <i>native</i>	123
Gambar IV.15. Peringatan <i>parser</i> pada jalur <i>native</i>	124
Gambar IV.16. <i>Throughput</i> profil $y_6-x_{0.5}$	124
Gambar IV.17. Komposisi hasil pemrosesan pada eksperimen <i>legal parser</i> jalur OCR	127
Gambar IV.18. Peringatan <i>parser</i> pada jalur OCR	128
Gambar IV.19. Biaya dan kegagalan pada eksperimen <i>legal parser</i> jalur OCR.....	128

DAFTAR TABEL

Tabel II.1.	Perbandingan model pada OmniDocBench v1.6.....	17
Tabel II.2.	Struktur hierarkis batang tubuh peraturan perundang-undangan ...	23
Tabel III.1.	Parameter <i>query</i> penelusuran pada /Search.....	29
Tabel III.2.	Label metadata pada halaman rincian peraturan	31
Tabel III.3.	Perbandingan portal peraturan BPK dan Google Drive pada aspek yang menyentuh rancangan akuisisi	36
Tabel III.4.	Kebutuhan fungsional akuisisi dokumen	42
Tabel III.5.	Kebutuhan fungsional ekstraksi dokumen.....	42
Tabel III.6.	Kebutuhan non-fungsional.....	43
Tabel III.7.	Kode peringatan dan kondisi yang ditandai oleh <i>legal parser</i>	52
Tabel III.8.	Kode peringatan dan kondisi yang ditandai oleh <i>generic outline parser</i>	53
Tabel III.9.	<i>Schema</i> tabel datasources	59
Tabel III.10.	<i>Schema</i> tabel datasource_documents.....	60
Tabel III.11.	<i>Schema</i> tabel datasource_scrape_runs.....	60
Tabel III.12.	<i>Schema</i> tabel datasource_scrape_run_entries	61
Tabel III.13.	<i>Schema</i> tabel pipeline_profiles	62
Tabel III.14.	<i>Schema</i> tabel pipeline_profile_nodes.....	62
Tabel III.15.	<i>Schema</i> tabel plugin_registry	63
Tabel III.16.	<i>Schema</i> tabel rag_pipeline_runs	63
Tabel III.17.	<i>Schema</i> tabel rag_node_runs	64
Tabel III.18.	Skema ScrapeEventEnvelope	65
Tabel III.19.	Skema NodeEventEnvelope	66
Tabel III.20.	Skema RagScrapeRequestedMessage.....	66
Tabel III.21.	Skema RagParseRequestedMessage	66
Tabel III.22.	Skema ArtifactReference	67
Tabel III.23.	Skema dokumen kanonik.....	67
Tabel III.24.	Metadata <i>provenance</i> pada <i>canonical document artifact</i>	68

Tabel III.25.	Field <i>manifest plugin</i>	68
Tabel III.26.	Skema konfigurasi BPK <i>scraper</i>	69
Tabel III.27.	Skema konfigurasi Google Drive <i>scraper</i>	69
Tabel III.28.	Skema konfigurasi <i>legal parser</i>	69
Tabel III.29.	Skema konfigurasi <i>generic outline parser</i>	71
Tabel III.30.	Kontrak API utama untuk akuisisi dan parsing	71
Tabel III.31.	Kontrak API oMLX yang digunakan oleh <i>parser</i>	74
Tabel III.32.	Kontrak <i>Layout Model HTTP Server</i>	74
Tabel IV.1.	Hasil enam konfigurasi eksperimen <i>scraper</i> BPK	103
Tabel IV.2.	Ringkasan korpus pada analisis <i>metadata</i> BPK	109
Tabel IV.3.	Hasil evaluasi integrasi <i>scraper</i> Google Drive	114
Tabel IV.4.	Hasil evaluasi <i>classifier</i> halaman	117
Tabel IV.5.	Matriks kebingungan <i>classifier</i> pada sampel berlabel	118
Tabel IV.6.	Komposisi korpus berdasarkan hasil <i>routing</i> halaman	118
Tabel IV.7.	Klasifikasi kode peringatan pada eksperimen <i>legal parser</i>	120
Tabel IV.8.	Rancangan eksperimen <i>legal parser</i> pada korpus ALL_VECTOR ...	120
Tabel IV.9.	Hasil profil operasional <i>legal parser</i> pada korpus ALL_VECTOR ...	121
Tabel IV.10.	<i>Clean rate</i> marginal berdasarkan toleransi penggabungan	122
Tabel IV.11.	Parameter eksperimen <i>legal parser</i> jalur OCR	126
Tabel IV.12.	Profil eksperimen <i>legal parser</i> jalur OCR	126
Tabel IV.13.	Hasil keutuhan struktur pada eksperimen <i>legal parser</i> jalur OCR	127
Tabel IV.14.	Biaya pemrosesan eksperimen <i>legal parser</i> jalur OCR	128
Tabel IV.15.	Konfigurasi evaluasi <i>generic outline parser</i>	130
Tabel IV.16.	Ringkasan hasil evaluasi <i>generic outline parser</i>	131
Tabel IV.17.	Hasil per dokumen pada evaluasi <i>generic outline parser</i>	131

BAB I

PENDAHULUAN

I.1 Latar Belakang

Pertanyaan yang paling sulit dijawab sering kali bukan pertanyaan yang jawabannya belum ada, melainkan pertanyaan yang jawabannya terkubur di dalam dokumen. Seorang pengacara yang menyiapkan pembelaan harus menelusuri peraturan perundang-undangan untuk menemukan pasal yang tepat, sementara seorang agen *customer service* yang menerima keluhan perangkat harus menemukan satu prosedur di antara ratusan halaman manual produk. *Large Language Model* (LLM) dapat menjawab keduanya dengan lancar, tetapi kelancaran itu tidak menjamin jawabannya bersandar pada dokumen yang benar. Sistem *Retrieval-Augmented Generation* (RAG) menutup celah tersebut dengan mengambil informasi dari sumber pengetahuan eksternal ketika *query* dijawab, lalu menyusun jawabannya dari dokumen yang berhasil diambil (Gao, Xiong, Gao, dkk., 2024). Dengan cara kerja seperti itu, jawaban sistem hanya akan sebaik dokumen yang tersedia untuk diambil.

Kebutuhan tersebut tidak berhenti pada satu bidang. Domain hukum, layanan pelanggan, dan dokumen operasional internal sama-sama membutuhkan jawaban yang bersandar pada dokumennya masing-masing. Membangun sistem tersendiri untuk setiap domain berarti mengulang seluruh rantai pengolahan dokumen dari awal, sehingga satu sistem yang dapat dikonfigurasi untuk banyak domain menjadi pilihan yang lebih efisien. Persoalannya, satu sistem tidak berarti satu perlakuan. Misrahi dkk. (2025) menemukan bahwa besar peningkatan performa RAG berbeda-beda antardomain karena perbedaan kompleksitas dan jenis pertanyaannya. Packowski dkk. (2024) bahkan perlu menyesuaikan cara dokumentasi produk dipotong menjadi *chunk* agar setiap *topic* tetap utuh ketika diambil. Perbedaan perlakuan tersebut berakar pada dokumen sumbernya, jauh sebelum pertanyaan pertama diajukan.

Dokumen sumber tidak datang dalam keadaan siap pakai. Sebelum satu kalimat pun dapat dicari, dokumennya harus dikumpulkan dari sumbernya, lalu diubah menjadi

teks yang tetap membawa struktur aslinya. Kedua langkah itu berada di hulu *pipeline* dan menentukan batas atas seluruh sistem, sebab apa yang hilang di sana tidak dapat dikembalikan oleh tahap mana pun sesudahnya. Sun dkk. (2026) menunjukkan bahwa *error* struktural pada hasil ekstraksi, misalnya *reading order* yang keliru pada halaman multikolom, dapat menggagalkan *retrieval* meskipun hampir seluruh karakternya dikenali dengan benar. Struktur dokumen juga dapat menentukan batas *chunk*, karena *chunking* yang mengikuti batas alami dokumen memberikan hasil pencarian yang lebih baik daripada *fixed-length chunking* (Shaukat dkk., 2026).

Langkah pertama, mengumpulkan dokumen, berhadapan dengan sumber yang berada di luar kendali sistem. Dokumen tersebar di portal publik dan penyimpanan berbasis API, jumlahnya besar dan terus bertambah, sehingga pengumpulannya harus berjalan otomatis dan berulang. *Crawler* yang menjalankan tugas itu perlu membatasi tingkat *concurrency* pengunduhan dan jeda antarpermintaan agar tidak membebani *server* tujuan (Olston & Najork, 2010). Laju yang terlalu berhati-hati memperlambat pengumpulan korpus, sedangkan laju yang terlalu agresif berisiko membebani sumbernya dan menggagalkan pengumpulan itu sendiri. Pengumpulan yang diulang secara berkala juga akan menemukan kembali berkas yang isinya sudah pernah tersimpan, sehingga setiap dokumen perlu diperiksa terlebih dahulu apakah sudah ada di dalam korpus sebelum disimpan.

Berkas yang berhasil dikumpulkan pun belum tentu terbaca mesin. PDF menyimpan isi halaman sebagai *glyph* beserta posisinya, bukan sebagai paragraf, judul, dan daftar, sehingga urutan objek teks di dalam berkas tidak selalu mengikuti *reading order* halamannya (International Organization for Standardization, 2020). Sebagian halaman bahkan tidak memuat teks sama sekali karena hanya berisi citra hasil pindai, dan halaman semacam itu dapat berada dalam satu berkas bersama halaman *born-digital* (Turski dkk., 2023). Menyerahkan seluruh halaman kepada *Optical Character Recognition* (OCR) memboroskan komputasi dan menambahkan *error* pengenalan pada halaman yang teksnya sebenarnya dapat dibaca langsung dari berkas, sedangkan ekstraksi *native* pada halaman hasil pindai hanya menghasilkan teks kosong. Pilihan di antara keduanya perlu diambil pada tingkat halaman, sementara *classifier* heuristik yang dilaporkan Turski dkk. (2023) menilai satu berkas PDF

secara utuh dan hanya mencapai *recall* 43,31%.

Teks yang berhasil dibaca masih kehilangan satu hal, yaitu tempatnya di dalam dokumen. Pengguna hampir tidak pernah membutuhkan satu dokumen secara utuh karena yang dicari biasanya satu bagian tertentu di dalamnya, sehingga batas antarbagian dan kedudukannya dalam hierarki perlu dipulihkan. Ma dkk. (2023) mencatat bahwa sebagian besar penelitian struktur dokumen masih berhenti pada batas elemen di dalam satu halaman, padahal hierarki dokumen membentang lintas halaman. Sumber hierarkinya pun tidak seragam antardomain. Peraturan perundang-undangan Indonesia mengikuti tata penomoran baku berupa bab, bagian, paragraf, pasal, ayat, dan rincian (Republik Indonesia, 2011, Lampiran II, angka 61–76), sehingga strukturnya dapat dibaca dari penomoran teksnya sendiri. Manual produk tidak memiliki aturan sejenis karena susunannya ditentukan oleh penulisnya dan berbeda antarproduk, sehingga struktur eksplisitnya hanya tersedia melalui *outline* yang tertanam di dalam berkas (NETGEAR, 2019). Satu cara pemulihan hierarki dengan demikian tidak akan melayani seluruh domain.

Seluruh persoalan di atas akan berulang setiap kali sistem menerima sumber atau domain baru. Setiap domain baru membawa portal, format, serta aturan strukturnya sendiri. Bila setiap penambahan menuntut perubahan pada kode inti, biaya perluasan menumpuk dan jalur pengolahan yang sudah berjalan ikut berisiko rusak. Gao, Xiong, Wang, dan Wang (2024) memodelkan RAG sebagai susunan modul yang dapat dikonfigurasi ulang alih-alih satu alur tetap, sejalan dengan gaya arsitektur *pipes and filters* yang menjadikan setiap komponen pengolah dapat ditukar selama bentuk data antarkomponennya dipertahankan (Garlan & Shaw, 1994). Akuisisi dan ekstraksi berhadapan langsung dengan keragaman tersebut, sehingga keduanya perlu dapat diperluas tanpa membongkar tahap sesudahnya.

Akuisisi dokumen dari sumber yang beragam, penentuan cara ekstraksi yang tepat untuk setiap halaman, pemulihan hierarki yang berbeda antardomain, serta kemampuan menerima domain baru merupakan empat syarat yang harus dipenuhi pada satu jalur ingesti yang sama sebelum dokumen siap digunakan. Tahap hulu itulah yang menjadi pokok bahasan tugas akhir ini.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan yang dibahas dalam tugas akhir ini dirumuskan sebagai berikut:

1. Bagaimana dokumen dari sumber data eksternal, yaitu portal web peraturan perundang-undangan Indonesia `peraturan.bpk.go.id` dan layanan penyimpanan Google Drive yang diakses melalui Google Drive API, dapat diakuisisi dan disimpan untuk pemrosesan RAG dengan tingkat kegagalan yang rendah dan *throughput* ingesti yang optimal, serta tetap fleksibel bagi penambahan sumber baru?
2. Bagaimana sistem ingesti dapat memperoleh representasi hierarkis yang utuh dari dokumen peraturan perundang-undangan yang halamannya dapat berupa teks *native* maupun hasil pindai, dan dari manual produk yang hierarkinya hanya tersedia melalui *outline* berkas, serta tetap fleksibel bagi penambahan domain baru?

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tujuan utama tugas akhir ini adalah mengembangkan komponen akuisisi dan ekstraksi dokumen pada sistem RAG multidomain, yaitu komponen yang mengumpulkan dokumen dari sumber eksternal dan mengubahnya menjadi representasi hierarkis yang siap digunakan oleh tahap berikutnya. Secara lebih rinci, tujuan yang ingin dicapai adalah sebagai berikut:

1. Mengembangkan mekanisme akuisisi dokumen dari portal web peraturan perundang-undangan Indonesia dan layanan penyimpanan Google Drive yang diakses melalui Google Drive API, yang dapat dijalankan berulang dengan *throughput* yang tinggi, tingkat kegagalan yang rendah, dan tanpa menyimpan berkas berisi sama lebih dari satu kali, serta dapat diperluas ke sumber baru tanpa mengubah kode inti.
2. Mengembangkan mekanisme ekstraksi dokumen PDF menjadi representasi hierarkis yang utuh, mencakup dokumen peraturan perundang-undangan

yang halamannya dapat berupa teks *native* maupun hasil pindai serta manual produk yang hierarkinya bersandar pada *outline* berkas, dan dapat diperluas ke domain baru tanpa mengubah kode inti.

Ketercapaian setiap tujuan tersebut diukur melalui kriteria berikut:

1. Ketercapaian tujuan pertama diukur melalui:

- (a) *Throughput* ingesti, yaitu jumlah dokumen baru yang berhasil disimpan per satuan waktu pada pengumpulan yang dijalankan berulang dari portal peraturan perundang-undangan, dengan target semakin tinggi semakin baik selama tidak mengorbankan keandalan.
- (b) Tingkat kegagalan, yaitu proporsi entri yang gagal diunduh akibat kegagalan jaringan atau *server* terhadap total entri yang dicoba, dengan target serendah mungkin dan tanpa proses berhenti total akibat satu *error* enumerasi.
- (c) Ketiadaan duplikasi penyimpanan, yaitu kemampuan mengenali berkas yang isinya sama meskipun sumber atau namanya berbeda, dengan target setiap berkas identik hanya tersimpan satu kali.
- (d) Fleksibilitas terhadap sumber baru, yaitu kemampuan menambahkan sumber data baru tanpa mengubah kode orkestrasi inti.

2. Ketercapaian tujuan kedua diukur melalui:

- (a) Ketepatan penentuan jalur ekstraksi per halaman, yaitu akurasi klasifikasi halaman *native* versus halaman hasil pindai, dengan penekanan pada *recall* kelas hasil pindai agar halaman yang sebenarnya membutuhkan OCR tidak lolos ke jalur *native*.
- (b) Keutuhan hierarki dokumen peraturan perundang-undangan, yaitu proporsi dokumen yang hasil ekstraksinya lolos pemeriksaan keutuhan struktur, ditinjau baik untuk halaman *native* maupun halaman hasil OCR.
- (c) Keutuhan hierarki manual produk, yaitu tingkat keberhasilan struktur *outline* berkas dipetakan menjadi hierarki bagian yang bersesuaian dengan judulnya.

- (d) Fleksibilitas terhadap domain baru, yaitu kemampuan menambahkan cara ekstraksi baru tanpa mengubah kode inti *pipeline* ekstraksi.

I.4 Batasan Masalah

Agar pelaksanaan tugas akhir ini dapat berjalan secara efektif dan terarah, diperlukan pembatasan ruang lingkup masalah yang jelas. Pembatasan ini bertujuan untuk menjaga fokus pengembangan pada aspek akuisisi dokumen dari sumber eksternal dan ekstraksinya menjadi representasi hierarkis. Pembatasan tersebut sekaligus menetapkan cakupan yang menjadi dasar penilaian ketercapaian tujuan pada Bab IV. Adapun batasan masalah yang ditetapkan dalam tugas akhir ini adalah sebagai berikut:

1. Pembahasan dibatasi pada tahap akuisisi dan ekstraksi dokumen, sedangkan tahap *chunking*, *indexing*, *retrieval*, dan *generation* berada di luar lingkup dan menjadi bagian anggota tim capstone lain.
2. Sumber data dibatasi pada dua sumber, yaitu portal peraturan perundang-undangan peraturan.bpk.go.id yang diakses melalui halaman web dan layanan penyimpanan Google Drive yang diakses melalui Google Drive API.
3. Akuisisi dilakukan dengan menelusuri daftar dokumen yang disediakan oleh sumber, baik melalui halaman web maupun API, secara berurutan hingga kebijakan penghentian yang dikonfigurasi terpenuhi, lalu mengunduh berkas dari setiap dokumen yang terdaftar tanpa penelusuran bertingkat mengikuti tautan lain di dalam sumber tersebut.
4. Korpus peraturan yang digunakan dibatasi pada dokumen dengan tema “Pendidikan” dan “pendidikan dan pelatihan” pada portal tersebut, bukan seluruh peraturan yang tersedia di dalamnya.
5. Pemeriksaan duplikasi dilakukan atas isi berkas secara utuh, sehingga hanya berkas yang isinya sama persis yang dikenali sebagai duplikat; dokumen yang sama tetapi tersimpan dalam berkas yang berbeda, misalnya karena dipindai ulang, tidak dikenali sebagai duplikat.
6. Sistem hanya mengumpulkan dokumen yang belum pernah tersimpan dan

tidak mengambil ulang dokumen yang isinya diperbarui di sumber, serta tidak menandai dokumen yang telah dihapus dari sumber, sehingga penelusuran riwayat versi dokumen berada di luar lingkup.

7. Berkas yang diproses dibatasi pada format PDF, sedangkan format lainnya tidak dicakup.
8. Representasi yang dipulihkan dibatasi pada hierarki bagian beserta teksnya, sedangkan gambar, formula, dan catatan kaki tidak dimodelkan.
9. Pemulihan tabel tidak dijamin: jalur ekstraksi *native* hanya mengambil teks tanpa memodelkan susunan barisnya, sedangkan pada jalur OCR susunan tabel bergantung pada keluaran model yang digunakan, terutama untuk tabel yang membentang lintas halaman atau memiliki sel gabungan.
10. Domain ekstraksi dibatasi pada dua domain, yaitu peraturan perundang-undangan Indonesia yang penyusunan dan urutan bagiannya mengikuti Undang-Undang Nomor 12 Tahun 2011 (Republik Indonesia, 2011) serta manual produk jaringan NETGEAR yang berkasnya menyediakan *document outline* (NETGEAR, 2026).
11. Pada dokumen peraturan, lampiran hanya dikenali sebagai bagian beserta rentang halamannya dan isinya tidak diuraikan lebih lanjut.
12. Mutu hasil OCR dinilai melalui keutuhan struktur hasil ekstraksi, bukan melalui perbandingan terhadap transkripsi acuan, karena korpus yang digunakan tidak menyediakan transkripsi acuan.
13. Pengembangan dan pengujian dilakukan pada satu mesin lokal dengan layanan OCR yang dijalankan pada perangkat Apple Silicon, sehingga hasil pengukuran tidak dimaksudkan sebagai kapasitas pada lingkungan produksi.

I.5 Metodologi

Pelaksanaan tugas akhir ini menempuh tahapan berikut:

1. Menganalisis kebutuhan akuisisi dan ekstraksi berdasarkan kondisi nyata sumber data, yaitu portal peraturan BPK dan Google Drive, serta karakteristik do-

kumen pada tiap domain, yaitu peraturan yang halamannya dapat berupa teks *native* maupun hasil pindai dan manual produk yang hierarkinya bersandar pada *outline* berkas, sehingga diperoleh Rumusan Masalah dan Tujuan pada bab ini.

2. Merancang arsitektur komponen akuisisi dan ekstraksi sebagai *plugin* yang mengikuti kontrak bersama, sehingga penambahan sumber data atau domain baru tidak mengubah kode inti.
3. Mengimplementasikan *plugin* akuisisi untuk portal peraturan BPK dan Google Drive, lalu menjalankannya untuk mengumpulkan korpus peraturan yang dipakai pada tahapan berikutnya.
4. Merancang dan menjalankan eksperimen atas berbagai konfigurasi akuisisi untuk mengukur *throughput* dan tingkat kegagalan, lalu memilih konfigurasi yang memenuhi kriteria ketercapaian tujuan pertama.
5. Mengimplementasikan *plugin* ekstraksi untuk dokumen peraturan dan manual produk.
6. Merancang dan menjalankan eksperimen klasifikasi jalur ekstraksi per halaman, pemulihan hierarki dokumen peraturan, dan pemeriksaan cakupan ekstraksi manual produk, lalu menilai hasilnya terhadap kriteria ketercapaian tujuan kedua.
7. Mengevaluasi seluruh hasil eksperimen terhadap kriteria ketercapaian dan Rumusan Masalah pada Bab IV, lalu menarik kesimpulan pada bab penutup.

I.6 Sistematika Pembahasan

Laporan tugas akhir ini disusun ke dalam lima bab. Bab I memuat latar belakang, rumusan masalah, tujuan beserta kriteria ketercapaiannya, batasan masalah, dan metodologi yang menjadi dasar dan arah pelaksanaan tugas akhir. Bab II menguraikan kajian pustaka yang menjadi landasan teori, meliputi arsitektur sistem RAG dan *pipeline* ingesti, arsitektur *pipeline* dan *plugin* yang dapat diperluas, akuisisi dokumen dari sumber web, ekstraksi konten dari dokumen PDF, dan ekstraksi struktur hierarkis dokumen pada domain peraturan perundang-undangan Indonesia maupun

manual produk. Bab III menganalisis persoalan akuisisi dan ekstraksi dokumen pada tiap domain, lalu menguraikan rancangan arsitektur komponen berbasis *plugin* yang menjadi solusinya. Bab IV menjelaskan rancangan eksperimen yang dijalankan atas komponen akuisisi dan ekstraksi, menyajikan hasilnya, dan mengevaluasi ketercapaiannya terhadap kriteria yang telah dirumuskan. Bab V menyimpulkan apakah rumusan masalah tugas akhir ini terjawab dan menyampaikan saran bagi pengembangan selanjutnya.

BAB II

KAJIAN PUSTAKA

II.1 Sistem *Retrieval-Augmented Generation* dan Tahap Ingesti

II.1.1 Arsitektur dan Paradigma RAG

RAG menggabungkan *Large Language Model* (LLM) dengan informasi dari sumber pengetahuan eksternal yang diambil ketika *query* dijawab. Gao, Xiong, Gao, dkk. (2024) membagi perkembangan RAG menjadi tiga paradigma, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*, serta menguraikan tiga fondasi penyusunnya: *retrieval*, *generation*, dan *augmentation*. Paradigma modular kemudian diformalkan lebih lanjut oleh Gao, Xiong, Wang, dan Wang (2024), yang memodelkan sistem RAG sebagai susunan modul dan operator yang dapat dikonfigurasi ulang seperti balok LEGO, alih-alih sebagai satu alur tetap.

Kerangka modular tersebut menempatkan setiap tahap *pipeline* sebagai komponen yang dapat ditukar selama antarmuka antartahap dipertahankan. Konsekuensi rancangan inilah yang relevan bagi tugas akhir ini, karena keragaman sumber dan format dokumen menuntut tahap akuisisi dan ekstraksi yang dapat diganti tanpa mengubah tahap sesudahnya.

II.1.2 Mutu Ingesti sebagai Batas Atas Mutu Jawaban

Kualitas dokumen yang masuk ke sistem membatasi kualitas jawaban yang dapat dihasilkan. Sun dkk. (2026) menyusun *benchmark* yang mencakup sebelas jenis dokumen sulit untuk mengukur pengaruh *error Optical Character Recognition* (OCR) terhadap performa RAG. Temuan utamanya adalah akurasi OCR yang tinggi tidak dengan sendirinya menghasilkan performa RAG yang baik: *error* struktural dan semantik dapat menyebabkan kegagalan *retrieval* meskipun *character error rate* tetap rendah.

Jenis gangguan struktural yang diuji mencakup urutan pembacaan yang tidak tepat

pada dokumen multikolom dan fragmentasi tabel lintas halaman. Gangguan tersebut merusak hubungan antareleman dan alur logis dokumen meskipun sebagian besar karakternya dikenali dengan benar. Pada dokumen berstruktur bersarang, temuan ini menunjukkan bahwa evaluasi ekstraksi perlu mencakup keutuhan susunan dan urutan bagian dalam hierarki dokumen, bukan hanya ketepatan teks.

II.1.3 RAG Lintas Domain

Misrahi dkk. (2025) mencatat bahwa *benchmark* RAG multidomain yang telah ada umumnya hanya mencakup sedikit domain dan sumber data serta berfokus pada satu bentuk jawaban. Untuk memperluas cakupan evaluasi, mereka menyusun *benchmark* dari delapan sumber yang meliputi tiga belas domain serta beragam jenis pertanyaan dan jawaban. Hasil eksperimennya menunjukkan bahwa peningkatan performa RAG bervariasi antardomain karena perbedaan kompleksitas dan jenis pertanyaan serta performa *retrieval*. Oleh karena itu, evaluasi sistem RAG multidomain perlu menyajikan hasil setiap domain secara terpisah agar perbedaan tersebut tidak tertutup oleh nilai gabungan.

II.2 Arsitektur Perangkat Lunak untuk *Pipeline* yang Dapat Diperluas

II.2.1 Gaya Arsitektur *Pipeline*

Gaya arsitektur *pipes and filters* menyusun sistem sebagai rangkaian komponen pengolah (*filter*) yang dihubungkan oleh saluran data (*pipe*). Menurut Garlan dan Shaw (1994), setiap *filter* berdiri sendiri, tidak berbagi keadaan dengan *filter* lain, dan tidak perlu mengetahui identitas komponen di hulu maupun hilirnya. Spesifikasi sebuah *filter* dapat membatasi data masukannya dan menjamin bentuk keluarannya tanpa menyebut komponen yang terhubung dengannya.

Salah satu variasinya adalah *typed pipes*, yaitu saluran yang mengharuskan data antarkomponen memiliki tipe yang terdefinisi dengan baik (Garlan & Shaw, 1994). Kontrak data tersebut memungkinkan komponen diganti selama komponen pengganti menerima masukan dan menghasilkan keluaran dalam bentuk yang sama.

II.2.2 Arsitektur *Plugin*

Plugin adalah komponen yang dapat ditambahkan ke sistem tanpa mengubah kode intinya. Dalam sebuah *pipeline*, mekanisme *plugin* memungkinkan sistem menemukan dan menjalankan komponen yang tersedia melalui aturan yang telah ditentukan.

Birsan (2005) menguraikan penerapan arsitektur tersebut pada platform Eclipse. Sebuah *plugin* dapat mendeklarasikan *extension point*, sedangkan *plugin* lain mendeklarasikan *extension* yang mengisinya. Mesin *runtime* membaca deklarasi tersebut melalui registri *plugin* dan memuat komponen yang diperlukan. Mekanisme ini memungkinkan kemampuan baru ditambahkan dengan mendaftarkan *plugin* tanpa menuliskan daftar seluruh komponen di dalam kode inti.

II.2.3 Representasi Terpadu pada Sistem Pemrosesan Dokumen

Dalam *pipeline* pemrosesan dokumen, kontrak antartahap dapat diwujudkan melalui representasi keluaran yang seragam. Livathinos dkk. (2025) memperkenalkan Docling, sistem *open source* yang mengubah PDF, citra, dokumen Microsoft Office, dan HTML menjadi model data terpadu DoclingDocument. Model tersebut merepresentasikan teks, tabel, gambar, keterangan gambar, daftar, informasi *layout* dan asal halaman, serta hierarki bagian dan kelompok. Arsitektur Docling dapat diperluas melalui *parser backend* dan *pipeline* yang dapat disesuaikan, sementara keluaran setiap jalur pemrosesan tetap menggunakan model data yang sama.

Sallinen dkk. (2025) mengambil pendekatan serupa pada MMORE dengan memproses lebih dari lima belas jenis berkas, termasuk dokumen, surel, gambar, audio, dan video. Hasil ekstraksinya dibakukan dalam format JSON terpadu. Format tersebut disebut MultimodalSample dan memuat teks dengan penanda media serta daftar modalitas yang mempertahankan jenis dan lokasinya. MMORE menyediakan antarmuka pemroses bersama; dukungan untuk jenis berkas baru ditambahkan melalui implementasi pemroses yang tetap menghasilkan format yang sama.

Docling menekankan kekayaan representasi struktur dokumen, sedangkan MMORE menekankan keluasan modalitas dan pemrosesan berskala besar. Pada kedu-

anya, kontrak keluaran yang seragam memungkinkan komponen hilir memproses dokumen tanpa bergantung pada format sumber atau pemroses yang digunakan.

II.3 Akuisisi Dokumen dari Sumber Web

II.3.1 Arsitektur *Web Crawler*

Web crawler adalah program yang menelusuri halaman web secara otomatis dan mengunduh isinya. Secara umum, *crawler* memilih URL yang akan dikunjungi, mengunduh halaman pada URL tersebut, lalu menemukan URL baru dari tautan di dalam halaman (Olston & Najork, 2010). URL yang telah ditemukan tetapi belum dikunjungi disimpan untuk menentukan halaman yang akan diunduh berikutnya. Beberapa komponen pengunduh dapat dijalankan secara paralel untuk meningkatkan kecepatan pengumpulan data. Jumlah pengunduh paralel dan jeda antarpermintaan perlu dikendalikan agar *crawler* tidak mengirimkan beban berlebihan kepada *server* tujuan.

II.3.2 Protokol Pengecualian Robot

Aturan akses otomatis pada sebuah situs dinyatakan melalui berkas `robots.txt`. Protokol tersebut dibakukan melalui RFC 9309 (Koster dkk., 2022), yang mendefinisikan tiga arahan utama, yaitu `user-agent`, `allow`, dan `disallow`. Standar tersebut juga mengatur perilaku ketika berkas aturan tidak dapat diambil: pada saat terjadi *error* pada *server*, *crawler* harus menganggap seluruh akses dilarang.

Sebagian situs mencantumkan arahan tambahan untuk menyatakan laju permintaan yang diharapkan, misalnya `crawl-delay` dan `request-rate`. Kedua arahan tersebut tidak termasuk dalam RFC 9309, sehingga dukungannya bergantung pada *crawler* yang digunakan. Standar tersebut hanya menentukan URL yang boleh diakses, sedangkan penentuan laju permintaan tetap menjadi tanggung jawab perancang *crawler*.

II.3.3 *Tool* Pengumpulan Data Web

Pengumpulan data web memerlukan klien HTTP untuk mengirim permintaan dan mengunduh halaman atau berkas. HTTPX (Encode OSS, 2025) adalah klien HTTP untuk Python yang menyediakan API sinkron sekaligus asinkron, mendukung HTTP/1.1 dan HTTP/2, serta dilengkapi *connection pooling* dan pengaturan *timeout*. Dukungan asinkron dan *connection pooling* tersebut menjadi dasar bagi pengunduhan serentak dengan jumlah koneksi yang terbatas.

Jika sumber berbentuk halaman HTML, isinya dapat dibaca menggunakan BeautifulSoup (Richardson, 2025). Pustaka tersebut menyediakan antarmuka untuk menelusuri dan mencari elemen dalam *parse tree* yang dihasilkan oleh *parser* HTML, seperti lxml (Behnel dkk., 2025).

Sebagian sumber dokumen tidak diakses melalui HTML, melainkan melalui API resmi. Google Drive API (Google, 2025) menyediakan operasi untuk menelusuri daftar berkas dan mengunduh isinya.

II.3.4 Pendeteksian Berkas Duplikat dengan *Hash*

Proses akuisisi yang dijalankan berulang dapat menemukan berkas dengan isi yang sama, meskipun nama atau sumbernya berbeda. Berkas duplikat dapat dikenali berdasarkan isinya menggunakan nilai *hash*. Meyer dan Bolosky (2011) menerapkan prinsip tersebut dengan menghitung *hash* atas bagian-bagian isi berkas dan menganggap dua berkas identik ketika seluruh nilai *hash*-nya cocok. Untuk pemeriksaan pada tingkat berkas utuh, nilai *hash* dihitung dari seluruh byte berkas. Salah satu algoritme yang dapat digunakan adalah SHA-256, yaitu fungsi *hash* dengan keluaran 256 bit yang dibakukan dalam FIPS PUB 180-4 (National Institute of Standards and Technology, 2015). Nilai tersebut kemudian dapat dibandingkan dengan nilai *hash* berkas yang telah tersimpan untuk mencegah penyimpanan berkas identik secara berulang.

II.4 Ekstraksi Konten dari Dokumen PDF

II.4.1 Halaman *Born-Digital* dan Halaman Hasil Pindai

Sebuah berkas PDF dapat memuat teks yang terbaca mesin, tetapi tidak selalu. Halaman yang dihasilkan langsung oleh perangkat lunak (*born-digital*) menyimpan teks sebagai *glyph* beserta posisinya (International Organization for Standardization, 2020), sehingga isinya dapat diekstraksi tanpa OCR. Sebaliknya, halaman hasil pemindaian hanya menyimpan citra halaman, sehingga isinya baru terbaca setelah melalui OCR (Turski dkk., 2023). Kedua jenis halaman itu dapat berada dalam satu berkas yang sama.

II.4.2 *Document Outline* pada PDF

Selain isi halaman, PDF dapat menyimpan *document outline* sebagai struktur navigasi pada tingkat dokumen. ISO 32000-2 mendefinisikan *outline* sebagai hierarki berbentuk pohon yang tersusun dari sejumlah *outline item*, yang umum ditampilkan oleh pembaca PDF sebagai *bookmark* (International Organization for Standardization, 2020, Klausul 12.3.3). Setiap *outline item* dapat menunjuk tujuan atau tindakan tertentu, misalnya lokasi bagian yang bersangkutan di dalam dokumen.

Document outline bersifat opsional dan terpisah dari objek teks maupun citra pada halaman. Jika tersedia, tingkat kedalaman, hubungan induk–anak, judul, dan tujuan setiap *bookmark* menyediakan representasi eksplisit atas susunan dokumen. Karena itu, *outline* dapat digunakan untuk memulihkan hierarki tanpa menyimpulkan tingkat judul hanya dari tampilan visual atau pola penomoran teks.

II.4.3 Pustaka Ekstraksi Teks *Native*

Ekstraksi *native* membaca objek teks yang sudah tersimpan di dalam berkas PDF tanpa melalui pengenalan citra. Menurut ISO 32000-2, isi halaman PDF dinyatakan sebagai rangkaian objek grafis. Teks ditampilkan dengan menempatkan *glyph* melalui operator teks dan transformasi koordinat, bukan dengan menyimpan struktur semantik berupa baris dan paragraf (International Organization for Standardization, 2020). Akibatnya, urutan objek teks di dalam berkas tidak selalu sama dengan

urutan pembacaan pada halaman. Pustaka ekstraksi perlu menggunakan informasi posisi untuk menyusun kembali fragmen teks menjadi baris yang berurutan.

PyMuPDF menyediakan keluaran berupa kata beserta *bounding box* dan nomor blok, baris, serta urutannya. Dokumentasinya juga menyediakan pengurutan berdasarkan koordinat vertikal dan horizontal untuk mendekati urutan pembacaan alami (Artifex Software, 2026). Dalam rekonstruksi baris, fragmen dengan posisi vertikal yang berdekatan dapat dikelompokkan ke dalam baris yang sama, sedangkan jarak horizontal menentukan apakah dua fragmen dipisahkan oleh spasi. Nilai toleransi yang terlalu kecil dapat memisahkan fragmen yang seharusnya berada pada satu baris, sedangkan nilai yang terlalu besar dapat menggabungkan fragmen yang seharusnya terpisah.

Adhikari dan Agarwal (2024) membandingkan sepuluh *parser* PDF pada enam kategori dokumen menggunakan *dataset* DocLayNet (Pfitzmann dkk., 2022). Hasil penelitian tersebut menunjukkan bahwa performa *parser* dipengaruhi oleh jenis dan struktur dokumen yang diproses.

Benchmark komunitas py-pdf (py-pdf, 2025) mengukur kualitas ekstraksi teks dan waktu pemrosesan pada empat belas makalah ilmiah. PyMuPDF memperoleh skor kualitas ekstraksi 96%, satu poin di bawah pypdfium2. Pada waktu pemrosesan, PyMuPDF tercepat bersama pypdfium2 dengan rerata 0,1 detik per dokumen.

II.4.4 Keputusan *Routing* OCR

Menjalankan OCR pada dokumen *born-digital* menambah biaya komputasi meskipun teksnya telah tersedia. Dalam pembangunan korpus 14,5 juta halaman PDF dari Common Crawl, Turski dkk. (2023) menggunakan *classifier* berbasis heuristik untuk mengenali dokumen *born-digital* yang tidak memerlukan OCR. Keputusan tersebut menggunakan tiga kriteria, yaitu panjang teks yang tampak lebih dari 100 karakter, panjang teks tersembunyi nol, dan jumlah citra nol. Metode tersebut berhasil menandai 219 dari 967 dokumen sebagai *born-digital* dengan presisi 93,15%, tetapi hanya mencapai *recall* 43,31% dan skor F1 sebesar 59,13.

Pendekatan tersebut menunjukkan bahwa keputusan *routing* OCR dapat dibuat dari

karakteristik berkas tanpa menggunakan model pembelajaran mesin. Namun, unit klasifikasinya adalah satu berkas PDF secara utuh, sehingga pendekatan ini belum menyediakan keputusan *routing* OCR pada tingkat halaman.

II.4.5 Model Mutakhir untuk Parsing Dokumen

Mesin OCR yang menyusun pengenalan teks sebagai *pipeline* bertahap memiliki tiga kelemahan yang dirangkum Kim dkk. (2022), yaitu biaya komputasi yang tinggi, keterbatasan pada bahasa dan jenis dokumen tertentu, serta perambatan *error* dari satu tahap ke tahap berikutnya. Model mutakhir karena itu menempuh arah yang berbeda, yakni membangkitkan teks terstruktur langsung dari citra halaman dalam satu model *end-to-end* (Wei dkk., 2024). Beberapa model mutakhir dirancang dengan jumlah parameter yang relatif kecil untuk menekan kebutuhan komputasi.

Niu dkk. (2025) menyusun MinerU2.5 berukuran 1,2 miliar parameter, sedangkan Duan dkk. (2026) melaporkan GLM-OCR berukuran 0,9 miliar parameter yang terdiri atas *encoder* visual CogViT 0,4 miliar parameter dan *decoder* bahasa GLM 0,5 miliar parameter. Z. Zhang dkk. (2026) melaporkan PaddleOCR-VL-1.6 yang juga berukuran 0,9 miliar parameter.

Ringkasan ketiga model teratas pada OmniDocBench v1.6 disajikan pada Tabel II.1 (Ouyang dkk., 2025).

Tabel II.1: Perbandingan model pada OmniDocBench v1.6

Model	Parameter	Strategi utama	Skor
PaddleOCR-VL-1.6	0,9 miliar	Optimasi data berbasis wilayah untuk memperbaiki wilayah yang masih lemah	96,33
MinerU2.5-Pro	1,2 miliar	Pemrosesan global-lokal berdasarkan arsitektur MinerU2.5	95,75
GLM-OCR	0,9 miliar	<i>Encoder</i> visual CogViT 0,4 miliar dan <i>decoder</i> GLM 0,5 miliar	95,22

Sumber: Niu dkk. (2025), Duan dkk. (2026), dan Z. Zhang dkk. (2026). Skor v1.6 dilaporkan oleh Z. Zhang dkk. (2026).

Karena ketiga model dievaluasi pada versi dan protokol yang sama, skor keseluruhannya dapat dibandingkan secara langsung. PaddleOCR-VL-1.6 memperoleh skor tertinggi, yaitu 0,58 poin di atas MinerU2.5-Pro dan 1,11 poin di atas GLM-

OCR, sedangkan MinerU2.5-Pro unggul 0,53 poin atas GLM-OCR (Z. Zhang dkk., 2026). Ketiganya berukuran relatif dekat, yaitu antara 0,9 dan 1,2 miliar parameter, sehingga perbedaan hasilnya tidak semata-mata mengikuti jumlah parameter. Meskipun demikian, skor keseluruhan OmniDocBench merangkum beberapa tugas pada korpus uji umum dan belum menunjukkan performa model pada dokumen hukum maupun manual produk yang digunakan dalam tugas akhir ini.

II.4.6 Model Serving OCR pada Apple Silicon

Model OCR berbasis *vision-language model* memerlukan *model server* yang memuat model, menerima permintaan, menjadwalkan inferensi, dan mengembalikan hasil melalui antarmuka jaringan. MLX adalah kerangka komputasi yang dirancang untuk Apple Silicon dengan model memori terpadu, sehingga operasi pada *array* yang sama dapat dijalankan pada CPU maupun GPU tanpa penyalinan data antarkeduanya (Apple Machine Learning Research, 2026). Karakteristik tersebut memungkinkan model bahasa dan model multimodal dijalankan secara lokal pada perangkat macOS berbasis Apple Silicon.

Ollama menyediakan layanan inferensi lokal dan mendukung akselerasi GPU Apple melalui Metal serta antarmuka yang kompatibel dengan API OpenAI (Ollama, 2026a, 2026b). oMLX menyediakan antarmuka serupa khusus untuk model berformat MLX dan mendukung model teks, *vision-language model*, serta model OCR (oMLX Contributors, 2026). Runtime tersebut juga menyediakan *continuous batching*, pembatasan jumlah permintaan serentak, pengelolaan beberapa model, dan pengaturan inferensi per model.

Dalam *continuous batching*, permintaan baru dapat dimasukkan ke kelompok inferensi yang sedang berjalan ketika permintaan lain selesai, sehingga sumber daya komputasi tidak menunggu seluruh kelompok berakhir. Pada Apple Silicon, Barrios (2026) melaporkan peningkatan *throughput* agregat hingga 4,3 kali pada enam belas permintaan serentak untuk model teks. Hasil tersebut menunjukkan bahwa pilihan *runtime* dan kebijakan *concurrency* dapat memengaruhi *throughput* layanan secara terpisah dari akurasi model OCR yang dilayani.

II.4.7 *Layout Analysis dan Reading Order*

Layout analysis memetakan wilayah pada halaman beserta perannya, misalnya judul, paragraf, tabel, atau gambar. Keragaman *dataset* pelatihan memengaruhi kemampuan model mengenali susunan halaman dari berbagai jenis dokumen. PubLayNet (Zhong dkk., 2019) dibangun secara otomatis dan hanya bersumber dari artikel ilmiah, sedangkan DocLayNet (Pfitzmann dkk., 2022) berisi 80.863 halaman beranotasi manusia dalam sebelas kelas *layout* dari sumber yang beragam. Pfitzmann dkk. (2022) menunjukkan bahwa model yang dilatih pada *dataset* artikel ilmiah mengalami penurunan akurasi ketika diterapkan pada *layout* yang lebih beragam.

Selain wilayah dan perannya, urutan pembacaan antarwilayah juga perlu ditentukan. Wang dkk. (2021) memperlakukan *reading order* sebagai persoalan tersendiri dan menyusun ReadingBank sebagai *benchmark* untuk tugas tersebut. Cui dkk. (2026) menyatukan kedua persoalan itu dalam RT-DocLayout, kerangka *end-to-end* berukuran 33 juta parameter yang menghasilkan deteksi *layout* sekaligus *reading order* secara *real-time*. Model yang sama dirilis pada proyek *open source* PaddleOCR dengan nama PP-DocLayoutV3 (PaddlePaddle, 2026).

II.5 Ekstraksi Struktur Hierarkis Dokumen

II.5.1 Peran Struktur bagi Sistem *Retrieval*

Dokumen yang telah diekstraksi umumnya dibagi menjadi sejumlah *chunk* sebelum memasuki tahap *indexing*. Shaukat dkk. (2026) membandingkan 36 metode segmentasi yang mencakup pendekatan *fixed-size*, semantik, *structure-aware*, hierarkis, adaptif, dan berbasis *Large Language Model*. Hasilnya menunjukkan bahwa *content-aware chunking* lebih efektif daripada *fixed-length chunking*. Strategi *Paragraph Group Chunking*, yang membentuk *chunk* berdasarkan batas paragraf, mencapai rata-rata nDCG@5 tertinggi sebesar 0,459, sedangkan *fixed-size character chunking* memperoleh nilai di bawah 0,244.

Tripathi dkk. (2025) juga menunjukkan keterbatasan *chunking* berbasis teks dalam menangani struktur dokumen yang kompleks, tabel lintas halaman, dan gambar tersemat. Metode yang diusulkan menggunakan *Large Multimodal Model* untuk mem-

baca halaman secara langsung dan menentukan batas *chunk* berdasarkan struktur dokumen. Temuan Shaukat dkk. mengenai batas paragraf dan pendekatan Tripathi dkk. yang membaca struktur halaman sama-sama menunjukkan bahwa informasi struktur berperan dalam menentukan batas *chunk*. Informasi tersebut perlu dipertahankan pada tahap ekstraksi agar dapat digunakan oleh tahap *chunking*.

II.5.2 Pemulihan Struktur Hierarkis Dokumen Multihalaman

Rausch dkk. (2021) memperkenalkan DocParser untuk memulihkan struktur hierarkis lengkap dari tampilan dokumen, baik PDF maupun hasil pindai. Sistem tersebut mendeteksi elemen dokumen dan mengklasifikasikan hubungan hierarkis di antaranya, termasuk hubungan pada gambar bersarang, tabel, dan sel tabel. Untuk mengatasi keterbatasan data beranotasi pada domain tertentu, DocParser menggunakan *weak supervision*; pendekatan tersebut meningkatkan F1 klasifikasi hubungan hierarkis sebesar 35,8% dibandingkan model tanpa *weak supervision*.

Ma dkk. (2023) kemudian menyoroti bahwa sebagian besar penelitian struktur dokumen masih berfokus pada batas elemen dalam satu halaman. Mereka memperkenalkan HRDoc, *dataset* berisi 2.500 dokumen multihalaman dengan hampir dua juta unit semantik yang dianotasi pada tingkat baris beserta kategori dan relasinya. Penelitian tersebut juga mengusulkan DSPS, sistem *encoder-decoder* yang membentuk struktur hierarkis dokumen dari informasi visual dan tekstual. Kedua penelitian ini menempatkan pemulihan hubungan antarelemen pada tingkat dokumen sebagai persoalan tersendiri, bukan sekadar kelanjutan dari deteksi *layout* per halaman.

II.5.3 Analisis Leksikal dan Pengenalan Penanda

Pemulihan struktur dokumen dapat diawali dengan memisahkan pengenalan penanda dari pembentukan hierarki. Aho dkk. (2006) mendefinisikan *lexical analysis* sebagai tahap yang membaca aliran karakter dan menghasilkan deretan *token*, sedangkan *syntactic analysis* menyusun *token* tersebut menjadi struktur. Pola *token* dapat dinyatakan menggunakan *regular expression* dan dikenali menggunakan *finite automaton* (Aho dkk., 2006; Hopcroft dkk., 2006).

Penanda pada dokumen menunjukkan batas struktur dan membedakannya dari isi. *Regular expression* dapat mengenali bentuk lokal setiap penanda, tetapi tidak menentukan hubungan induk–anak antarbagiannya. Hubungan tersebut perlu ditentukan pada tahap *parsing* setelah penanda berhasil dikenali.

II.5.4 *Finite-State Machine* untuk Urutan Bagian Dokumen

Finite-state machine memiliki sejumlah keadaan yang terbatas dan berpindah keadaan berdasarkan keadaan saat ini serta simbol masukan yang dibaca (Hopcroft dkk., 2006). Model ini sesuai untuk mengenali bagian dokumen yang tersusun dalam urutan tetap karena setiap penanda dapat memicu perpindahan menuju bagian berikutnya.

Keterbatasan jumlah keadaan membuat *finite-state machine* tidak dapat menyimpan jumlah simpul terbuka pada struktur dengan kedalaman yang tidak terbatas. Oleh karena itu, model tersebut dapat menentukan perpindahan antarbagiannya, tetapi tidak cukup untuk membentuk seluruh hubungan bersarang dalam hierarki dokumen.

II.5.5 *Parsing Hierarki Berbasis Stack*

Pushdown automaton memperluas *finite automaton* dengan *stack* dan digunakan untuk mengenali *context-free language* (Hopcroft dkk., 2006). Sifat *last-in, first-out* pada *stack* memungkinkan struktur yang belum selesai dipertahankan ketika struktur lain dibuka di dalamnya. Dalam pembentukan pohon, simpul yang baru dibuka dapat dimasukkan ke *stack*, sedangkan simpul yang telah selesai dikeluarkan sampai ditemukan induk yang sesuai bagi penanda berikutnya.

Dalam praktiknya, pembentukan hierarki dapat menggabungkan *stack* dengan aturan hubungan induk–anak, validasi urutan, dan pemeriksaan konteks. Pendekatan tersebut dapat disebut *parsing* deterministik berbasis *stack*; istilah ini tidak menyatakan bahwa parser yang dibangun merupakan *pushdown automaton* formal.

II.5.6 Standar Representasi Dokumen Hukum

Representasi dokumen hukum perlu mempertahankan batas antarbagian beserta peran strukturalnya, bukan hanya memuat teks dokumen. Akoma Ntoso, yang disahkan

sebagai standar OASIS pada 29 Agustus 2018, menyediakan kosakata XML bersama untuk dokumen parlemen, legislatif, dan yudisial agar dapat dipertukarkan antarinstansi di berbagai negara (Palmirani dkk., 2018). Standar tersebut memberikan nama elemen bagi bagian dokumen menurut peran struktural atau semantiknya, seperti pembukaan, bagian, paragraf, klausul, dan rujukan. Dengan representasi tersebut, batas dan hubungan antarbagian dapat dinyatakan secara eksplisit dan diproses oleh mesin.

II.5.7 Struktur Peraturan Perundang-undangan Indonesia

Teknik penyusunan peraturan perundang-undangan di Indonesia diatur dalam Pasal 64 dan Lampiran II Undang-Undang Nomor 12 Tahun 2011 (Republik Indonesia, 2011, Pasal 64 dan Lampiran II). Sejumlah ketentuan dalam Lampiran II tersebut kemudian diubah dan ditambah melalui Undang-Undang Nomor 13 Tahun 2022 (Republik Indonesia, 2022, Pasal I angka 19 dan Lampiran II). Perubahan terbaru melalui Undang-Undang Nomor 1 Tahun 2026 hanya mengubah Pasal 15 mengenai materi ketentuan pidana dan tidak mengubah Lampiran II (Republik Indonesia, 2026, Pasal V).

Kerangka peraturan terdiri atas enam bagian yang disusun berurutan, yaitu judul, pembukaan, batang tubuh, penutup, penjelasan jika diperlukan, dan lampiran jika diperlukan (Republik Indonesia, 2011, Lampiran II, angka 1). Pembukaan terdiri atas frasa “Dengan Rahmat Tuhan Yang Maha Esa”, jabatan pembentuk, pertimbangan, dasar hukum, dan diktum (Republik Indonesia, 2011, Lampiran II, angka 14–60). Batang tubuh memuat materi peraturan yang dirumuskan dalam pasal dan pada umumnya dikelompokkan menjadi ketentuan umum, materi pokok yang diatur, ketentuan pidana jika diperlukan, ketentuan peralihan jika diperlukan, dan ketentuan penutup (Republik Indonesia, 2011, Lampiran II, angka 61–66).

Pengelompokan batang tubuh mengikuti tingkatan sebagaimana dirangkum dalam Tabel II.2. Tidak semua tingkatan harus digunakan; bab dapat langsung memuat pasal, bagian dapat memuat pasal tanpa paragraf, dan paragraf digunakan jika masih diperlukan pengelompokan yang lebih rinci (Republik Indonesia, 2011, Lampiran II, angka 67–76).

Tabel II.2: Struktur hierarkis batang tubuh peraturan perundang-undangan

Unsur	Bentuk penanda	Kedudukan struktural
Buku	Bilangan tingkat dan judul dengan huruf kapital	Pengelompokan tertinggi jika peraturan merupakan kodifikasi
Bab	Angka Romawi dan judul dengan huruf kapital	Mengelompokkan materi berdasarkan kesamaan substansi
Bagian	Bilangan tingkat yang ditulis dengan huruf dan disertai judul	Pengelompokan di dalam bab
Paragraf	Angka Arab dan disertai judul	Pengelompokan di dalam bagian
Pasal	Angka Arab	Satuan aturan yang dapat berada langsung di bawah bab, bagian, atau paragraf
Ayat	Angka Arab di antara tanda kurung	Rincian suatu pasal
Rincian	a., 1., a), dan 1)	Tabulasi di dalam pasal atau ayat dengan kedalaman paling banyak empat tingkat

Setelah batang tubuh, bagian penutup memuat perintah pengundangan, penandatanganan pengesahan atau penetapan, pengundangan, serta keterangan lembaran atau berita tempat peraturan dimuat (Republik Indonesia, 2011, Lampiran II, angka 160–173). Bagian ini berbeda dari ketentuan penutup, yang merupakan pasal atau bab terakhir di dalam batang tubuh. Penjelasan, jika ada, terdiri atas penjelasan umum dan penjelasan pasal demi pasal serta tidak boleh memuat norma baru, sedangkan lampiran dapat memuat uraian, daftar, tabel, gambar, peta, atau sketsa dan harus dinyatakan sebagai bagian yang tidak terpisahkan dari peraturan (Republik Indonesia, 2011, Lampiran II, angka 174–197).

Peraturan perubahan memiliki struktur khusus. Batang tubuhnya pada dasarnya terdiri atas Pasal I dan Pasal II yang ditulis dengan angka Romawi. Pasal I memuat identitas peraturan yang diubah beserta materi perubahan; jika terdapat lebih dari satu perubahan, setiap perubahan diberi nomor dengan angka Arab. Pasal II memuat ketentuan mulai berlaku dan dalam keadaan tertentu dapat memuat ketentuan peralihan (Republik Indonesia, 2011, Lampiran II, angka 230–233; Republik Indonesia, 2022, Lampiran II, angka 233–233b). Bab atau pasal yang disisipkan menggunakan nomor sebelumnya dengan tambahan huruf kapital, sedangkan ayat sisipan menggunakan nomor ayat sebelumnya dengan tambahan huruf kecil. Bagian yang dihapus tetap dicantumkan urutannya dan diberi keterangan “Dihapus” (Republik Indo-

nesia, 2011, Lampiran II, angka 234–236; Republik Indonesia, 2022, Lampiran II, angka 234–236c). Dengan demikian, kata “Pasal” dapat menandai Pasal I sebagai wadah perubahan maupun pasal bernomor Arab yang menjadi materi perubahan, sehingga peran strukturalnya perlu ditentukan bersama konteks kemunculannya.

II.5.8 Dokumentasi Produk untuk *Customer Service*

Dokumentasi produk dapat menjadi basis pengetahuan bagi sistem tanya jawab untuk *customer service*. Packowski dkk. (2024) melaporkan pengalaman membangun RAG skala perusahaan untuk menjawab pertanyaan pengguna mengenai perangkat lunak berdasarkan dokumentasi produk. Mereka menekankan bahwa dokumentasi produk memiliki struktur yang berbeda dari artikel Wikipedia, sehingga struktur tersebut perlu diperhatikan ketika konten dikonversi menjadi teks dan dibagi menjadi *chunk*. Sistem yang dilaporkan mengambil satu *topic* secara utuh karena setiap *topic* dirancang singkat, lengkap, dan dapat berdiri sendiri.

Kebutuhan serupa ditemukan pada tanya jawab berbasis manual produk. L. Zhang dkk. (2023) menyusun MPMQA dari 209 manual produk elektronik yang berasal dari 27 merek, dengan anotasi enam jenis wilayah semantik dan 22.021 pasangan pertanyaan–jawaban. Karena jawaban dapat bergantung pada teks sekaligus elemen visual dan biasanya hanya berkaitan dengan beberapa halaman, mereka memisahkan proses menjadi pencarian halaman yang relevan dan pembentukan jawaban multimodal.

II.5.9 Struktur Manual Pengguna NETGEAR

Sebagai contoh dari korpus manual NETGEAR, manual R7000 dan RAX10 menyusun informasi ke dalam bab, bagian, dan subbagian bertingkat (NETGEAR, 2019, 2023). Keduanya memuat uraian konsep dan komponen perangkat, prosedur dengan langkah bernomor, tabel referensi, catatan atau peringatan, bagian *troubleshooting*, serta informasi teknis tambahan. Susunan tersebut memisahkan informasi berdasarkan tujuan pengguna, misalnya memahami fungsi perangkat, melakukan konfigurasi, atau menyelesaikan gangguan.

Kedua berkas PDF tersebut juga menyediakan *bookmark* bertingkat yang mengikuti

susunan daftar isinya (NETGEAR, 2019, 2023). Manual R7000 memiliki *bookmark* hingga empat tingkat, sedangkan manual RAX10 memiliki *bookmark* hingga tiga tingkat. Setiap *bookmark* menghubungkan judul dengan halaman tujuan dan hubungan induk–anak antarjudul, sehingga *outline* PDF dapat merepresentasikan hierarki manual tanpa bergantung pada pola penomoran tertentu. Berbeda dari struktur peraturan yang ditetapkan melalui aturan domain, susunan manual ditentukan oleh penulis dan dapat berbeda antarproduk; karena itu, *outline* yang tertanam dalam PDF menjadi sumber struktur yang lebih langsung bagi dokumen manual.

BAB III

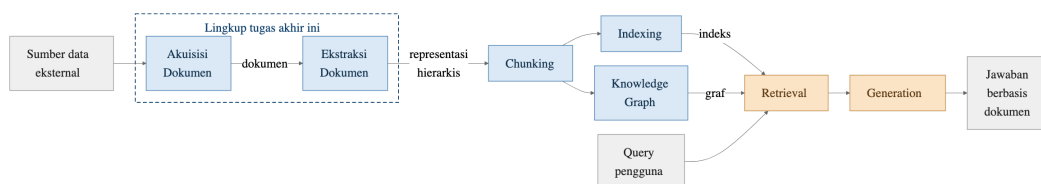
ANALISIS MASALAH DAN RANCANGAN SOLUSI

III.1 Analisis Masalah

Subbab I.1 menyatakan bahwa akuisisi dokumen, penentuan cara ekstraksi, pemulihan hierarki, dan kemampuan menerima domain baru merupakan empat syarat yang harus dipenuhi pada satu jalur ingesti yang sama. Subbab ini menelusuri kondisi yang membuat keempat syarat tersebut sulit dipenuhi, dengan bertumpu pada keadaan sumber data dan dokumen yang benar-benar dihadapi. Setiap analisis ditutup dengan konsekuensi rancangan yang ditimbulkannya, sehingga kebutuhan yang dirumuskan pada akhir subbab ini dapat ditelusuri kembali ke kondisi yang melatarbelakanginya.

III.1.1 Posisi Komponen Akuisisi dan Ekstraksi

Sebagaimana ditetapkan pada Subbab I.4, tugas akhir ini mencakup tahap akuisisi dan ekstraksi dokumen, sedangkan *chunking*, *indexing*, *retrieval*, dan *generation* dikerjakan oleh anggota tim lain. Kedudukan kedua tahap tersebut di dalam *pipeline* sistem OMNI-RAG secara keseluruhan digambarkan pada Gambar III.1.



Gambar III.1: Kedudukan tahap akuisisi dan ekstraksi dokumen di dalam alur sistem OMNI-RAG

Kotak biru pada gambar tersebut menandai tahap yang berjalan secara *background* atas setiap dokumen yang masuk, sedangkan kotak jingga menandai tahap yang baru berjalan ketika pengguna mengajukan *query*. Kotak putus-putus menandai kedua tahap yang menjadi lingkup tugas akhir ini. Tahap *chunking* memicu *indexing* dan

pembangunan *knowledge graph* secara paralel, dan keduanya harus selesai sebelum sebuah dokumen dinyatakan selesai diproses.

Komponen yang dibangun karena itu berada sepenuhnya di hulu, dan seluruh hubungannya dengan tahap-tahap sesudahnya menyempit menjadi satu titik penyerahan: representasi hierarkis yang dihasilkan tahap ekstraksi dan diterima tahap *chunking*. Segala perbedaan antarsumber dan antardomain harus sudah terselesaikan sebelum representasi itu diserahkan, karena tanggung jawabnya berhenti di titik itu.

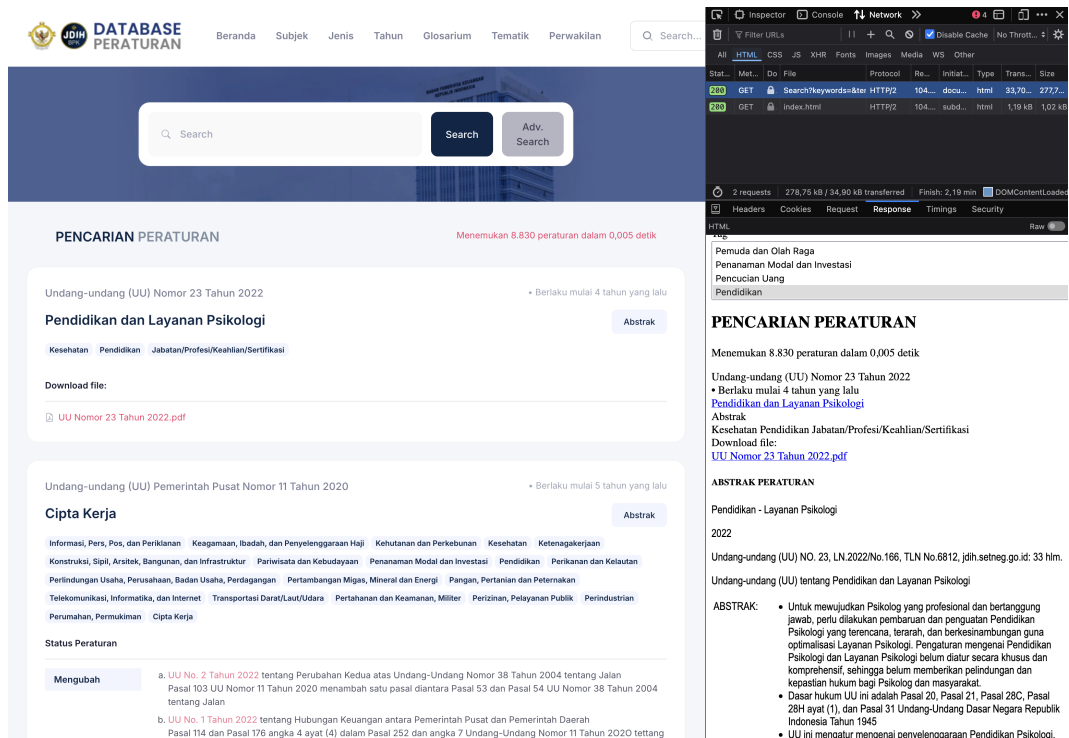
Pembagian ini tidak dapat digeser ke hilir, sebab tahap *chunking* berada di luar kendali penulis. Kalau tahap tersebut dibuat mengenali asal dokumen, ia jadi terikat pada rincian sumber dan domain, dan setiap penambahan sumber atau domain baru akan menuntut perubahan di sana pula. Keluaran ekstraksi karena itu wajib berbentuk satu representasi yang sama untuk sumber dan domain apa pun. Keseragaman ini menjadi syarat pertama yang membatasi seluruh pilihan rancangan pada subbab-subbab berikutnya.

III.1.2 Analisis Sumber Data

Dua sumber data yang menjadi lingkup tugas akhir ini, yaitu portal peraturan perundang-undangan `peraturan.bpk.go.id` dan layanan penyimpanan Google Drive, memiliki karakter yang sangat berbeda. Keduanya dianalisis satu per satu terlebih dahulu, lalu dibandingkan pada akhir subbab ini.

III.1.2.1 Portal Peraturan BPK

Portal peraturan BPK menerapkan *server-side rendering*. Seluruh isi hasil pencarian sudah tersusun di dalam HTML pada *response* GET yang pertama, sebagaimana terlihat pada Gambar III.2. Panel *network* pada gambar tersebut mencatat satu *request* dokumen HTML, dan *response body*-nya sudah memuat judul peraturan, penanda subjek, nama berkas unduhan, hingga abstraknya. Pengumpulan karena itu cukup dilakukan dengan *request* HTTP biasa yang diikuti *parsing* HTML, tanpa perlu menjalankan JavaScript maupun mencari *endpoint* data di baliknya. Yang diambil dengan cara itu adalah tampilan halamannya, sehingga setiap perubahan struktur HTML dapat mengubah cara pengambilan tanpa pemberitahuan apa pun.



Gambar III.2: *Response* HTML pertama halaman hasil pencarian sudah memuat seluruh isi yang ditampilkan

Penelusuran koleksinya berlangsung melalui satu *endpoint*, yaitu */Search*, dengan seluruh *filter* dan nomor halaman dinyatakan sebagai parameter *query* pada *request* GET. *Pagination*-nya berbasis nomor halaman melalui parameter *p*, dan parameter tersebut dihilangkan pada halaman pertama. *Filter* yang menerima banyak nilai dinyatakan dengan mengulang parameter yang sama, sehingga *request* yang memilih dua tahun, dua entitas, dua jenis, dan dua tema berbentuk seperti berikut.

```
curl --location \
  --header "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
    10_15_7) AppleWebKit/537.36 Chrome/124 Safari/537.36" \
  "https://peraturan.bpk.go.id/Search?keywords=&tentang=&nomor=1\
    &tahun=2026&tahun=2025&entitas=474&entitas=284\
    &jenis=39&jenis=6&tema=2&tema=3&p=2"
```

Parameter yang digunakan beserta sifatnya dirangkum pada Tabel III.1. Nilai untuk entitas, jenis, dan tema bukan teks bebas melainkan pengenal numerik yang ditetapkan portal.

Tabel III.1: Parameter *query* penelusuran pada /Search

Parameter	Isi	Sifat nilai
keywords	Pencarian teks bebas	Teks, satu nilai
tentang	Perihal peraturan	Teks, satu nilai
nomor	Nomor peraturan	Teks, satu nilai
tahun	Tahun peraturan	Bilangan, boleh berulang
entitas	Instansi penerbit	Pengenalan numerik dari portal, boleh berulang
jenis	Jenis peraturan	Pengenalan numerik dari portal, boleh berulang
tema	Tema atau subjek peraturan	Pengenalan numerik dari portal, boleh berulang
p	Nomor halaman hasil	Bilangan, dihilangkan pada halaman pertama

Satu halaman hasil memuat sepuluh peraturan beserta kendali *pagination* di bawahnya, sebagaimana terlihat pada Gambar III.3. Tidak ada penanda jumlah halaman yang dapat diandalkan sebagai batas penelusuran, sehingga akhir daftar hanya dapat dikenali dari halaman yang tidak lagi memuat hasil. Yang pasti tersedia pada setiap entri hanyalah tautan halaman rinciannya, berbentuk `/Details/{id}/{slug}`, sedangkan judul, penanda subjek, ringkasan relasi, dan tautan unduh hanya muncul pada sebagian entri. Tautan rincian itulah yang dipakai sebagai *source identifier*, karena hanya bagian itu yang selalu ada dan tidak berubah antar-*run*.



Gambar III.3: Satu entri pada halaman hasil pencarian portal peraturan BPK beserta kendali *pagination* di bawahnya

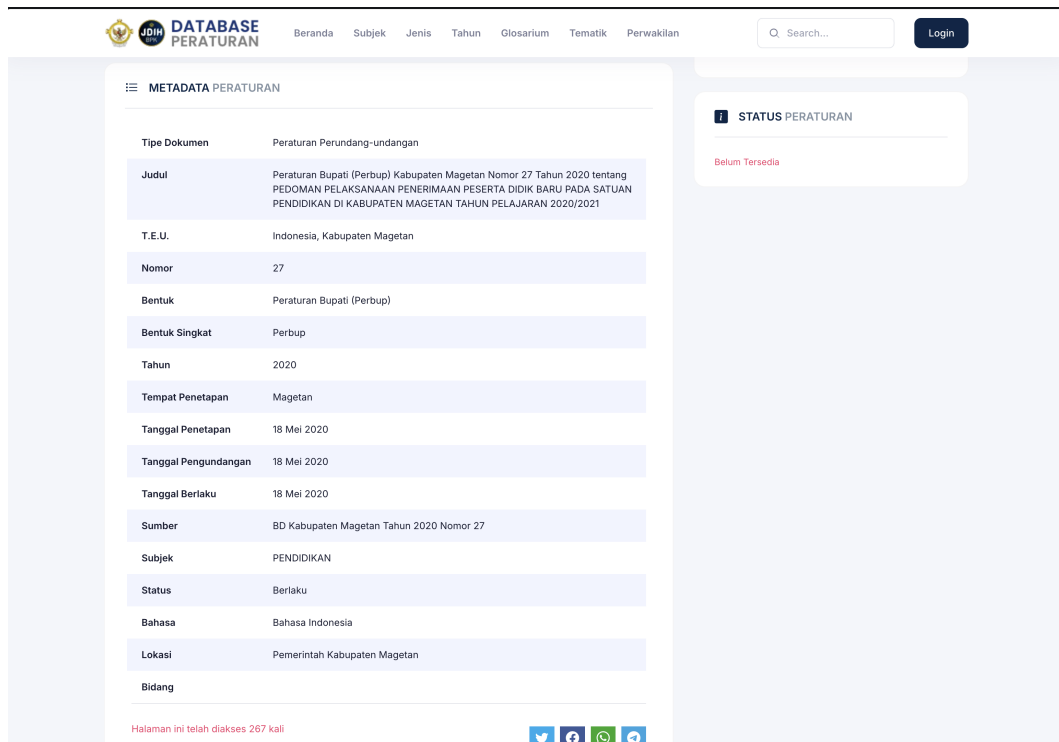
Metadata peraturan sama sekali tidak muncul pada halaman hasil pencarian. Portal mencetak tujuh belas label metadata pada setiap halaman rincian, sebagaimana terlihat pada Gambar III.4 dan dirangkum pada Tabel III.2, dan hanya di sanalah nilai-nilai tersebut dapat dibaca. Ketujuh belas label itu selalu dicetak, tetapi nilai-

nya kerap kosong dan label mana yang kosong berbeda-beda menurut jenis peraturannya, sehingga sel kosong harus dibaca sebagai “tidak dicatat oleh sumber” dan bukan sebagai kegagalan *parsing*. Relasi antardokumen seperti mencabut, dicabut, atau mengubah ditampilkan pada kartu status, dan kartu itu muncul baik pada halaman hasil pencarian maupun pada halaman rincian. Sebagian butir relasinya berupa tautan ke halaman rincian peraturan yang dirujuk, sebagian lagi hanya berupa judul tanpa tautan karena peraturan tersebut tidak tersedia di portal, dan sebagian dokumen tidak memiliki butir relasi sama sekali sehingga kartunya hanya menyatakan “Belum Tersedia”. Karena metadatanya hanya ada di halaman rincian, setiap dokumen menuntut dua *request* lanjutan setelah entrinya ditemukan pada daftar, yaitu untuk halaman rincian dan untuk berkasnya. Bentuk kedua *request* itu adalah sebagai berikut.

```
curl --location \
    "https://peraturan.bpk.go.id/Details/218816/uu-no-23-tahun-2022"

curl --location --remote-name \
    "https://peraturan.bpk.go.id/Download/213209/\
UU%20Nomor%2023%20Tahun%202022.pdf"
```

Berkas itu dikembalikan dengan Content-Type: `application/octet-stream`, sehingga jenis berkas tidak dinyatakan oleh *server* dan hanya dapat dibaca dari nama berkas pada tautan unduhnya.



Gambar III.4: Kartu metadata dan kartu status pada halaman rincian Perbup Kabupaten Magetan Nomor 27 Tahun 2020 (peraturan.bpk.go.id/Details/249238)

Tabel III.2: Label metadata pada halaman rincian peraturan

Label	Isi
Tipe Dokumen	Golongan dokumen, misalnya peraturan perundang-undangan
Judul	Judul lengkap peraturan
T.E.U.	Tajuk entri utama, yaitu pihak yang bertanggung jawab atas peraturan
Nomor	Nomor peraturan
Bentuk	Bentuk peraturan secara lengkap
Bentuk Singkat	Singkatan bentuk peraturan
Tahun	Tahun peraturan
Tempat Penetapan	Kota tempat peraturan ditetapkan
Tanggal Penetapan	Tanggal peraturan ditetapkan
Tanggal Pengundangan	Tanggal peraturan diundangkan
Tanggal Berlaku	Tanggal peraturan mulai berlaku
Sumber	Lembaran atau berita resmi tempat peraturan dimuat
Subjek	Satu atau lebih subjek peraturan
Status	Status keberlakuan, misalnya berlaku atau tidak berlaku
Bahasa	Bahasa naskah peraturan
Lokasi	Instansi yang menyimpan naskah
Bidang	Bidang peraturan, kerap dibiarkan kosong oleh sumber

Ketersediaan berkas pada halaman rincian ternyata menghadirkan empat keadaan yang berbeda, dan keempatnya diperlihatkan pada Gambar III.5. Keadaan perta-

ma adalah keadaan yang diharapkan, yaitu kartu berkas yang menyediakan satu berkas PDF. Keadaan kedua menyajikan berkas dalam format yang berada di luar cakupan tugas akhir ini, misalnya .doc, sehingga dokumennya tetap harus diabaikan. Keadaan ketiga tidak menyediakan berkas sama sekali dan hanya menampilkan keterangan bahwa berkasnya belum tersedia, sehingga entri yang terdaftar pada hasil pencarian tidak selalu menghasilkan dokumen. Keadaan keempat paling menyesatkan. Kartunya menampilkan nama berkas PDF beserta tombol unduh seperti keadaan pertama, tetapi tautannya mengembalikan *error* HTTP 404 ketika benar-benar diakses. Keempat keadaan itu bersama-sama berarti keberhasilan pengambilan satu dokumen baru dapat dipastikan setelah unduhan dicoba, bukan pada saat entrinya ditemukan.



Gambar III.5: Empat keadaan ketersediaan berkas pada halaman rincian peraturan

Source identifier pada portal ini juga tidak menentukan isinya. Dua peraturan dengan tautan rincian yang berbeda dapat merujuk pada berkas yang isinya sama persis. Contohnya adalah /Details/156250 dan /Details/249238: portal menamai yang pertama Perda Kabupaten Magetan Nomor 27 Tahun 2020 dan yang kedua Perbup Kabupaten Magetan Nomor 27 Tahun 2020, tetapi keduanya menyajikan berkas dengan nama yang sama dan isi yang identik. Perbedaan *source identifier* pada kedua entri bahkan menyentuh jenis peraturan, sehingga tidak ada aturan

berdasarkan *source identifier* maupun *metadata* yang dapat mengenali pasangan itu sebagai satu berkas yang sama. Kesamaan isi karena itu hanya dapat dipastikan dengan membandingkan nilai *hash* atas seluruh isi berkasnya sebagaimana diuraikan pada Subsubbab II.3.4. Urutan hasil enumerasi pun tidak dijamin tetap antar-*run*, sehingga halaman ke-*n* pada satu *run* belum tentu memuat entri yang sama pada *run* berikutnya. Kedua keadaan itu menuntut hal yang sama, yaitu penilaian “sudah pernah dilihat” harus bersandar pada *source identifier* dan isi berkas, tidak pernah pada posisi entri di dalam daftar.

Kebijakan penelusuran portal ini dinyatakan melalui berkas `robots.txt`. Isinya dibuka dengan penjelasan panjang tentang arti *content signal*, lalu diikuti sepuluh kelompok arahan yang dikutip berikut ini.

```
User-agent: *  
Content-Signal: search=yes,ai-train=no,use=reference  
Allow: /
```

```
User-agent: Amazonbot  
Disallow: /
```

```
User-agent: Applebot-Extended  
Disallow: /
```

```
User-agent: Bytespider  
Disallow: /
```

```
User-agent: CCBot  
Disallow: /
```

```
User-agent: ClaudeBot  
Disallow: /
```

```
User-agent: CloudflareBrowserRenderingCrawler  
Disallow: /
```

```
User-agent: Google-Extended  
Disallow: /
```

```
User-agent: GPTBot  
Disallow: /
```

```
User-agent: meta-externalagent  
Disallow: /
```

Kelompok pertama berlaku bagi *user agent* umum, termasuk *crawler* yang diba-

ngun pada tugas akhir ini, dan mengizinkan seluruh jalur, sedangkan sembilan kelompok sesudahnya melarang *crawler* bernama yang mengumpulkan data untuk melatih model bahasa. Arahan Content-Signal pada kelompok pertama menyatakan bahwa *indexing* untuk pencarian diizinkan, pelatihan model tidak diizinkan, dan pemanfaatan isinya dibatasi sebagai rujukan. Portal tidak mencantumkan satu pun arahan jeda antar-*request*, baik *crawl-delay* maupun *request-rate*, sejalan dengan catatan pada Subsubbab II.3.2 bahwa kedua arahan itu memang bukan bagian baku protokol pengecualian robot. Portal juga tidak mencantumkan batas laju *request* pada dokumentasi mana pun. Kebijakan laju karena itu sepenuhnya menjadi tanggung jawab perancang *crawler*, dan titik operasi yang aman hanya dapat ditemukan melalui percobaan: menebak terlalu rendah memperlambat pengumpulan, sedangkan menebak terlalu tinggi berisiko membebani sumbernya (Olston & Najork, 2010; Kolobov dkk., 2019). Penghormatan atas kebijakan sumber, termasuk pembatasan laju yang tidak dinyatakan secara eksplisit, merupakan bagian dari praktik pengumpulan data web yang bertanggung jawab (Krotov dkk., 2020; Brown dkk., 2024).

III.1.2.2 Google Drive

Berbeda dengan portal peraturan, Google Drive menyediakan kontrak yang terdokumentasi dan diberi nomor versi, sehingga bentuk data yang diterima tidak berubah tanpa pemberitahuan. Aksesnya dilakukan melalui Google Drive API v3 dengan kredensial *service account* yang *scope*-nya dibatasi pada pembacaan berkas (Google, 2025). Penelusuran satu folder dilakukan melalui metode `files.list`, sedangkan pengunduhan berkasnya melalui `files.get_media`, dengan parameter berikut.

```
files.list(  
    q          = "'{folder_id}' in parents and trashed=false",  
    fields     = "nextPageToken, files(id, name, mimeType,  
                                     modifiedTime, size, webViewLink)",  
    pageSize  = {n},  
    pageToken  = {token},  
    supportsAllDrives      = True,  
    includeItemsFromAllDrives = True,  
)
```



```
files.get_media(
    fileId = "{file_id}",
    supportsAllDrives = True,
)
```

Parameter `q` berisi *query* dalam *query language* milik Drive. Klausula `'{folder_id}' in parents` memilih berkas yang induknya adalah folder yang dituju, sedangkan `trashed=false` menyingkirkan berkas yang sudah dipindahkan ke tempat sampah. Penelusuran dengan demikian hanya menjangkau satu tingkat folder dan tidak menurun ke subfolder di dalamnya. Parameter `supportsAllDrives` dan `includeItemsFromAllDrives` harus diaktifkan bersama-sama, sebab tanpa keduanya folder yang berada pada *shared drive* akan terbaca kosong meskipun berisi.

Parameter `fields` menyatakan secara eksplisit atribut apa saja yang diminta. ID berkas Google Drive menjadi *source identifier* dokumen dan tetap sama meskipun berkasnya dinamai ulang atau dipindahkan. Nama berkas, *MIME type*, ukuran, dan waktu perubahan terakhir menyertai setiap entri pada daftar yang sama. *Pagination* dinyatakan melalui `nextPageToken` pada *response*: selama *token* itu dikembalikan, penelusuran berlanjut dengan mengirimkannya kembali sebagai `pageToken`, dan penelusuran berhenti ketika *token* itu tidak lagi ada. Mode kegagalannya juga jelas, karena layanan mengembalikan kode *error* HTTP yang terdokumentasi, sehingga *error* yang layak diulang seperti 429 dan 5xx dapat dibedakan dari *error* permanen tanpa menebak.

Batas lajunya pun dinyatakan secara terbuka. Google menetapkan kuota dalam satuan *quota unit*, dengan batas 1.000.000 satuan per menit per proyek dan 325.000 satuan per menit per pengguna pada satu proyek (Google, 2026). Biaya setiap pemanggilan juga dicantumkan menurut kategorinya: pendaftaran isi folder seperti `files.list` bernilai 100 satuan, sedangkan pengunduhan isi berkas, yaitu kategori tempat `files.get_media` berada, bernilai 200 satuan. Batas per pengguna itu dengan demikian setara dengan sekitar 1.600 pengunduhan berkas per menit, jauh di atas laju yang dibutuhkan tugas akhir ini, sehingga pengaturan laju pada sumber ini menjadi persoalan yang sudah terjawab sejak awal.

III.1.2.3 Perbandingan dan Konsekuensi Rancangan

Perbandingan kedua sumber pada aspek yang menyentuh rancangan akuisisi disajikan pada Tabel III.3.

Tabel III.3: Perbandingan portal peraturan BPK dan Google Drive pada aspek yang menyentuh rancangan akuisisi

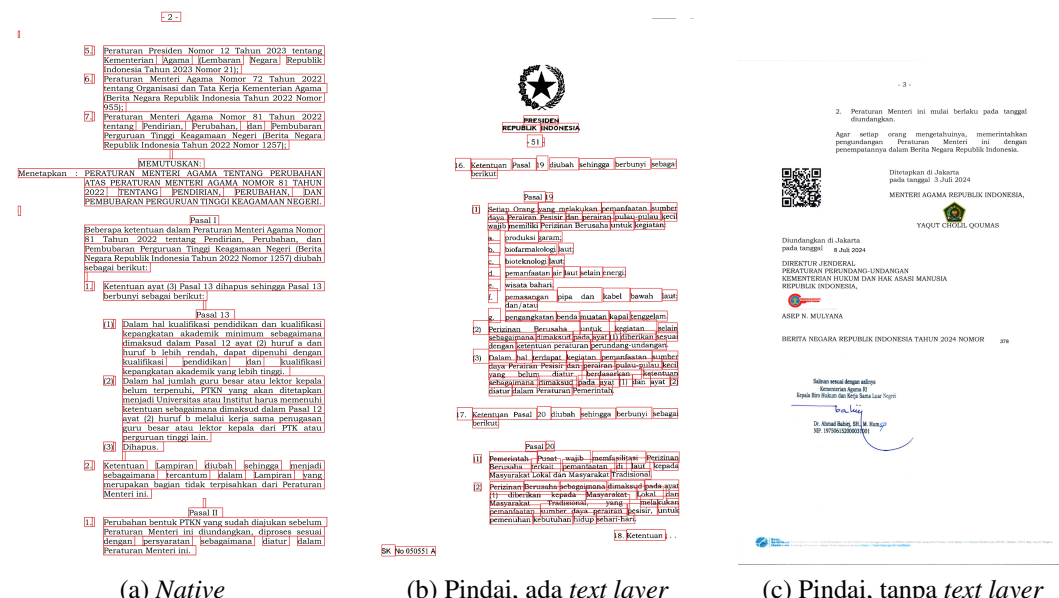
Aspek	Portal peraturan BPK	Google Drive
Mekanisme enumerasi	Halaman hasil pencarian HTML (<i>server-side rendering</i>)	Metode <code>files.list</code> pada API v3
Penanda halaman	Nomor halaman pada parameter <code>query p</code>	<code>nextPageToken</code>
Penanda akhir daftar <i>Source identifier</i>	Halaman tanpa hasil Tautan rincian <code>/Details/{id}/{slug}</code>	Tidak ada <code>nextPageToken</code> id berkas Google Drive
Cara mengenali jenis berkas	Dari nama berkas pada halaman rincian; <i>server</i> mengembalikan <code>application/octet-stream</code>	Dari <code>mimeType</code> yang menyertai entri pada daftar
Letak metadata	Hanya pada halaman rincian, menuntut satu <i>request</i> tambahan	Menyertai entri pada daftar
Batas laju	Tidak dinyatakan	Kuota terdokumentasi dalam satuan <i>quota unit</i>
Mode kegagalan	Struktur HTML berubah, berkas belum tersedia, tautan mengembalikan <i>error 404</i>	Kode <i>error</i> HTTP yang terdokumentasi

Tabel tersebut memperlihatkan bahwa kedua sumber hanya berbagi satu hal, yaitu polanya: menghasilkan aliran entri dokumen, lalu mengambil berkas dari setiap entri. Cara melakukannya berbeda pada setiap baris tabel. Pengetahuan tentang bentuk sumber, dengan demikian, harus ditempatkan pada komponen yang dapat ditukar, sementara penyimpanan, pemeriksaan duplikasi, pengaturan laju, kebijakan penghentian, dan pembukuan *run* harus dimiliki oleh inti agar berlaku sama bagi sumber mana pun. Pemeriksaan duplikasi tidak dapat hanya mengandalkan *source identifier*; pemeriksaan isi berkas juga diperlukan karena portal peraturan dapat menghadirkan berkas identik dengan *source identifier* berbeda. Laju *request* dan tingkat *concurrency* harus menjadi parameter yang dapat dikonfigurasi dan diuji, bukan tetapan yang tertanam di dalam kode, karena hanya salah satu sumber yang menyatakan batasnya.

III.1.3 Analisis Dokumen Peraturan Perundang-undangan

Seluruh dokumen peraturan pada tugas akhir ini diambil dari satu portal yang sama, tetapi cara pembuatan, tata penulisan, dan kualitas berkasnya berbeda-beda. Perbedaan itu tidak terlihat dari daftar hasil pencarian maupun dari metadatanya, dan baru muncul setelah berkasnya dibuka satu per satu. Tiga persoalan lahir dari keadaan tersebut, yaitu keterbacaan halamannya, pemulihan hierarkinya, dan keragaman korpusnya.

Persoalan pertama adalah keterbacaan halamannya. Berkas PDF peraturan memuat tiga tipe halaman, dan ketiganya diperlihatkan pada Gambar III.6 dengan kotak merah yang menandai setiap *text span* yang dapat diekstraksi langsung dari berkas. Tipe pertama adalah halaman *native*, yaitu halaman yang dihasilkan langsung oleh perangkat lunak pengolah dokumen sebagaimana diuraikan pada Subsubbab II.4.1. Seluruh isinya tertutup kotak, dan teks yang keluar sama persis dengan yang tercetak. Tipe kedua adalah halaman hasil pindai yang sudah diberi *text layer* oleh perangkat lunak pemindainya. Halaman ini tetap berupa satu citra utuh, tetapi kotaknya sama penuhnya dengan tipe pertama. Tipe ketiga adalah halaman hasil pindai tanpa *text layer*, yang tidak menghasilkan satu kotak pun.



(a) Native

(b) Pindai, ada *text layer*

(c) Pindai, tanpa *text layer*

Gambar III.6: Tiga tipe halaman; kotak merah menandai *text span* yang dapat diekstraksi langsung

Tipe-tipe itu dapat bercampur di dalam satu berkas. Panel (a) dan (c) berasal dari dokumen yang sama, yaitu Peraturan Menteri Agama Nomor 13 Tahun 2024 (peraturan.bpk.go.id/Details/298032). Panel (a) adalah halaman batang tubuhnya, sedangkan panel (c) adalah halaman penutupnya, satu-satunya halaman yang dipindai di dokumen itu.

Panel (b) diambil dari Undang-undang Nomor 11 Tahun 2020 (peraturan.bpk.go.id/Details/149750), yang seluruh halamannya berupa citra hasil pindai dan seluruhnya menyertakan *text layer*. Isi *text layer* itu berasal dari OCR perangkat lunak pemindai, dan mutunya tidak dijamin. Teks yang diekstraksi dari panel (b) memperlihatkan hal itu.

FRESIDEN

REPUBUK INDONESIA

-51 -

16. Ketentuan Pasal i9 diubah sehingga berbunyi sebagai berikut:

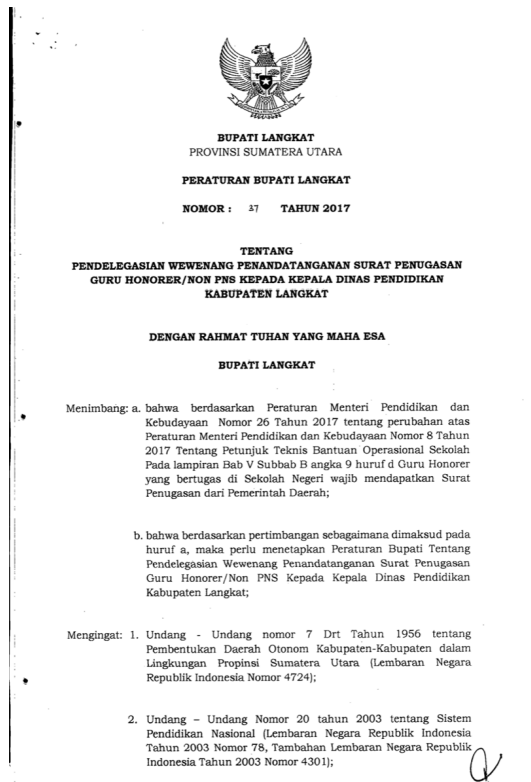
...

a. produksi gararn;

Kerusakan pada kata seperti REPUBUK dan gararn masih dapat ditoleransi, karena maknanya tetap terbaca dan pemulihan hierarki tidak bergantung padanya. Kerusakan pada penanda struktur jauh lebih berbahaya. Penanda seperti bab, bagian, dan pasal itulah satu-satunya sumber hierarki bagi dokumen peraturan, sehingga penanda yang rusak tidak akan cocok dengan pola apa pun dan hierarki yang dihasilkan menjadi salah. Halaman bertipe kedua dengan demikian menyediakan teks yang tampak lengkap tetapi bisa menyesatkan.

Halaman pindaian juga tidak selalu serapi ketiga panel tersebut. Gambar III.7 menampilkan halaman pertama sebuah peraturan bupati yang seluruh halamannya merupakan hasil pindai, lengkap dengan kemiringan pindaian, bintik-bintik, bekas lubang penjilidan, serta tanda tangan yang ditulis tangan.

Konsekuensi rancangannya ada dua. Pertama, penentuan jalur ekstraksi harus diambil per halaman, karena satu berkas dapat memuat lebih dari satu tipe halaman. Inilah celah yang tertinggal pada Subsubbab II.4.4, karena heuristik yang dilaporkan di sana menilai satu berkas PDF secara utuh. Kedua, penentuan itu tidak boleh



Gambar III.7: Halaman pertama Peraturan Bupati Langkat Nomor 37 Tahun 2017 (peraturan.bpk.go.id/Details/100120)

bersandar pada ada atau tidaknya *text span*, karena kriteria tersebut menggolongkan tipe kedua sebagai halaman *native*. Sinyal yang memisahkan keduanya adalah luas halaman yang tertutup citra, dan sinyal itu dapat dibaca dari struktur berkas tanpa menjalankan OCR terlebih dahulu.

Persoalan kedua adalah pemulihan hierarkinya. Berkas PDF peraturan tidak menyimpan susunan bagiannya di mana pun, karena tidak ada penanda semantik maupun *document outline* yang menyatakan bahwa sebuah baris merupakan judul bab atau awal pasal. Satu-satunya sumber hierarki adalah penanda yang tercetak di dalam teksnya, seperti BAB III, Bagian Kesatu, dan Pasal 5, yang bentuknya sudah dibakukan sebagaimana diuraikan pada Subsubbab II.5.7. Mengenali bentuk penanda seperti itu mudah, sedangkan menentukan kedudukannya tidak. Baris yang bentuknya persis sama dapat muncul sebagai pembuka bagian baru, sebagai kutipan atas peraturan lain, atau sebagai materi yang sedang diubah di dalam bagian lain, dan ketiganya hanya dapat dibedakan dari letak kemunculannya di dalam dokumen.

Konsekuensi rancangannya, pengenalan bentuk penanda dan penetapan peran hie-

rarkisnya harus dipisahkan. Pengenalan bentuk dapat diselesaikan dengan *pattern matching* sebagaimana diuraikan pada Subsubbab II.5.3, sedangkan penetapan peran menuntut mekanisme yang menyimpan konteks, yaitu urutan bagian dokumen pada Subsubbab II.5.4 dan kedalaman bagian yang belum tertutup pada Subsubbab II.5.5.

Persoalan ketiga adalah keragaman korpusnya. Dokumen yang terkumpul mulai dari peraturan tingkat pusat sampai peraturan bupati dan wali kota, terbit lintas puluhan tahun, dan panjangnya bervariasi dari beberapa halaman sampai ratusan halaman. Pada rentang seluas itu terdapat dokumen yang menyimpang dari tata penulisan baku, misalnya penomoran yang melompat atau bagian yang tidak lengkap, dan penyimpangan itu ada pada dokumen sumbernya sehingga tidak dapat diperbaiki oleh sistem yang membacanya. Konsekuensi rancangannya, aturan ekstraksi harus berlaku umum bagi seluruh korpus, dan temuan atas keutuhan struktur harus dilaporkan sebagai peringatan yang tetap menyertakan hasil ekstraksinya, bukan sebagai kegagalan yang membuang dokumennya.

III.1.4 Analisis Dokumen Manual Produk

Manual produk tidak memiliki penomoran baku seperti peraturan. Susunan bagusnya ditentukan oleh penulisnya dan berbeda antarproduk sebagaimana dicatat pada Subsubbab II.5.9, sehingga tidak ada penanda di dalam teksnya yang dapat dibaca untuk memulihkan hierarkinya. Hierarki manual karena itu hanya dapat diambil dari *document outline* berkas, yang sudah memuat judul setiap bagian beserta kedalamannya sebagaimana diuraikan pada Subsubbab II.4.2. Manual yang tidak menyediakan *outline* tidak memiliki sumber hierarki sama sekali, sehingga lebih tepat ditolak daripada diratakan menjadi teks tanpa struktur.

Outline memberi judul dan kedalaman setiap bagian, tetapi tidak menyatakan di mana isi bagian itu dimulai pada halaman. Setiap *outline item* karena itu masih harus ditautkan ke baris judulnya di halaman. Destinasi sebuah *outline item* berupa titik pada halaman, dan titik itu belum tentu jatuh pada baris judul yang dimaksud. Gambar III.8 memperlihatkan keadaan tersebut pada awal sebuah bab dalam manual NETGEAR R7000 (NETGEAR, 2019). Titik destinasinya jatuh jauh di atas judul

bab, sehingga baris pertama di bawahnya justru angka penanda bab.

destinasi bookmark

2 baris pertama di bawah destinasi

Connect to the Network and Access the Router

baris judul yang sebenarnya

Gambar III.8: Kedudukan destinasi sebuah *outline item* terhadap baris judul pada awal bab manual NETGEAR R7000

Konsekuensi rancangannya, pemulihan hierarki manual bersandar sepenuhnya pada *outline*, dan langkah yang menentukan keberhasilannya adalah penautan setiap *outline item* ke baris judulnya.

III.1.5 Analisis Kebutuhan Ekstensibilitas

Dua sumber data dan dua domain dokumen yang dibahas di atas masih akan bertambah. Sistem RAG multidomain memang dibangun untuk menerima domain baru, dan setiap penambahan membawa portal, format, serta aturan strukturnya sendiri. Bila penambahan itu harus dikerjakan dengan mengubah kode yang sudah ada, biaya perluasannya menumpuk dan *pipeline* yang sudah berjalan ikut berisiko rusak, sebagaimana diperingatkan pada Subsubbab II.2.2. Konsekuensi rancangannya, komponen yang berbeda-beda isinya harus mengikuti kontrak yang sama, agar penambahan sumber maupun domain tidak menyentuh kode inti.

III.1.6 Kebutuhan Fungsional dan Non-Fungsional

Analisis pada subbab-subbab sebelumnya dirangkum menjadi kebutuhan yang harus dipenuhi komponen akuisisi dan ekstraksi. Kebutuhan fungsional menyatakan apa yang harus dilakukan sistem, sedangkan kebutuhan non-fungsional menyatakan mutu yang harus dipenuhi ketika sistem melakukannya. Sebagian kebutuhan berlaku bagi seluruh sumber maupun domain, sedangkan sebagian lain hanya berlaku

pada sumber atau domain tertentu karena lahir dari sifat khas sumber atau dokumennya.

Kebutuhan fungsional akuisisi dokumen disajikan pada Tabel III.4.

Tabel III.4: Kebutuhan fungsional akuisisi dokumen

Kode	Kebutuhan
KF-01	Sistem menelusuri daftar dokumen yang disediakan sumber melalui <i>pagination</i> dan menghasilkan entri dokumen beserta <i>source identifier</i> yang tetap sama antar- <i>run</i> .
KF-02	Sistem mengunduh berkas setiap entri yang lolos <i>filter</i> jenis berkas, lalu menyimpannya beserta metadata yang menyertainya dari sumber.
KF-03	Sistem mengenali dokumen yang sudah tersimpan, baik melalui <i>source identifier</i> maupun melalui isi berkasnya, dan tidak menyimpannya lebih dari satu kali.
KF-04	Sistem menjalankan akuisisi secara terjadwal maupun atas permintaan.
KF-05	Sistem mencatat riwayat setiap <i>run</i> beserta status dan penyebab kegagalan setiap entrinya.
KF-06	Sistem menghentikan penelusuran menurut kebijakan penghentian yang dikonfigurasi, tanpa harus menelusuri seluruh daftar pada setiap <i>run</i> .
KF-07	Sistem menyediakan jeda antar- <i>request</i> dan batas <i>concurrency</i> yang dapat dikonfigurasi pada setiap sumber.

Kebutuhan fungsional ekstraksi dokumen disajikan pada Tabel III.5.

Tabel III.5: Kebutuhan fungsional ekstraksi dokumen

Kode	Kebutuhan
KF-08	Sistem menentukan jalur ekstraksi setiap halaman dokumen peraturan, yaitu pembacaan teks <i>native</i> atau OCR, pada tingkat halaman.
KF-09	Sistem mengekstraksi halaman <i>native</i> dokumen peraturan langsung dari berkas dan halaman hasil pindainya melalui layanan OCR.
KF-10	Sistem memulihkan hierarki dokumen peraturan dari tata penomorannya, termasuk menetapkan peran struktural penanda yang bentuknya sama menurut konteks kemunculannya.
KF-11	Sistem memulihkan hierarki manual produk dari <i>outline</i> berkas dengan menautkan setiap <i>outline item</i> ke baris judul yang dimaksud.
KF-12	Sistem menolak manual produk yang tidak menyediakan <i>outline</i> , alih-alih meratakannya menjadi teks tanpa struktur.
KF-13	Sistem mencatat temuan atas keutuhan struktur sebagai peringatan yang tetap menyertakan hasil ekstraksinya.
KF-14	Sistem menghasilkan satu bentuk representasi hierarkis yang sama untuk seluruh dokumen.

Kebutuhan non-fungsional disajikan pada Tabel III.6.

Tabel III.6: Kebutuhan non-fungsional

Kode	Kebutuhan
KNF-01	<i>Throughput</i> akuisisi, yaitu jumlah dokumen baru yang berhasil disimpan per satuan waktu, setinggi mungkin selama tidak mengorbankan keandalan.
KNF-02	Keandalan <i>run</i> , yaitu kegagalan pada satu entri tidak menghentikan <i>run</i> secara keseluruhan.
KNF-03	Kesantunan terhadap portal peraturan BPK, yaitu laju <i>request</i> yang sampai ke portal dapat ditahan pada tingkat yang tidak membebani.
KNF-04	Ekstensibilitas, yaitu penambahan sumber maupun domain baru tidak menuntut perubahan pada kode inti.
KNF-05	Determinisme hasil, yaitu keluaran dan rekapitulasi <i>run</i> tidak bergantung pada urutan selesainya operasi yang dijalankan secara <i>concurrent</i> .

Kebutuhan yang dirumuskan di atas menjadi dasar penilaian atas pilihan solusi yang dibahas pada subbab berikutnya.

III.2 Analisis Solusi

Analisis solusi ini berfokus pada hubungan antara arsitektur berbasis *plugin*, *pipeline* akuisisi dokumen, dan *pipeline* ekstraksi dokumen. Uraian mengenai pembagian kelas, struktur penyimpanan, dan urutan pemanggilan komponen secara lebih rinci disajikan pada bagian rancangan solusi.

III.2.1 Arsitektur

III.2.1.1 Arsitektur *Plugin*

Sistem RAG multidomain menghadapi variasi pada cara sumber eksternal menyediakan dokumen dan pada cara setiap domain menyatakan struktur dokumennya. Variasi tersebut tidak seharusnya ditangani oleh komponen inti melalui percabangan khusus untuk setiap sumber atau domain.

Dalam solusi ini, *plugin* adalah komponen yang memuat aturan khusus sumber atau domain dan dijalankan oleh komponen inti melalui kontrak yang sama. Variasi sumber eksternal ditempatkan pada *scraper plugin*, sedangkan variasi struktur dokumen ditempatkan pada *parser plugin*. Komponen inti bertugas mengorkestrasikan pemuatan dan eksekusi *plugin* serta menjalankan alur umum yang tidak bergantung pada sumber atau domain.

Setiap *plugin* dikenali oleh nama, versi, dan tipe komponennya pada registri. Registri digunakan untuk menemukan implementasi *plugin* dan skema konfigurasi yang sesuai sebelum proses dijalankan. Skema konfigurasi menyatakan atribut, tipe data, nilai bawaan, dan batasan nilai yang diperlukan oleh *plugin*. Konfigurasi yang dikirim dari antarmuka maupun yang tersimpan pada basis data divalidasi menggunakan skema tersebut.

Pemisahan ini membuat penambahan sumber baru cukup dilakukan dengan mengimplementasikan *scraper plugin* yang memenuhi kontrak tanpa mengubah komponen inti. Penambahan domain baru cukup dilakukan dengan mengimplementasikan *parser plugin* yang memenuhi kontrak tanpa mengubah komponen inti maupun tahap hilir.

III.2.1.2 Arsitektur Pipeline

Profil RAG merupakan konfigurasi yang menentukan keseluruhan *pipeline* RAG. Profil memilih *datasource* sebagai sumber dokumen dan mengonfigurasi setiap *node* yang menyusun *pipeline* tersebut. Jenis *node* yang tersedia mencakup *parser*, *chunker*, *indexer*, *retrieval orchestrator*, serta *knowledge graph* apabila digunakan. Setiap node menyimpan referensi *plugin* dan konfigurasi yang digunakan oleh komponen tersebut.

Dalam lingkup tugas akhir ini, pembahasan pipeline dibatasi sampai node *parser*. Node *parser* menerima dokumen mentah dari *datasource* dan menghasilkan representasi dokumen kanonik. Tahap setelah parser, termasuk *chunking*, *indexing*, *retrieval*, *generation*, dan pembangunan *knowledge graph*, berada di luar lingkup implementasi dan hanya menjadi konsumen keluaran parser.

III.2.2 Solusi Datasource dan Scraper Plugin

III.2.2.1 Datasource dan Kontrak Scraper

Datasource diperlakukan sebagai penyimpanan dokumen yang berdiri sendiri dari profil RAG. *Datasource* memiliki identitas, nama, deskripsi, status, daftar ekstensi yang diterima, konfigurasi *scraper* bila tersedia, serta riwayat proses *scrapping*. *Datasource* tanpa *scraper* tetap dapat digunakan sebagai tempat unggah manual,

sedangkan *datasource* dengan *scraper* dapat menerima dokumen dari sumber eksternal secara manual maupun terjadwal.

Kontrak *scraper* menyamakan bentuk interaksi dengan sumber data. *Plugin* melakukan enumerasi secara *lazy* per halaman, sehingga inti sistem dapat berhenti berdasarkan kebijakan yang dikonfigurasi pada *datasource*. Setiap entri membawa *source identifier* yang stabil, URL sumber bila tersedia, nama file, dan *metadata* sumber. Ketika entri diproses, *plugin* mengembalikan isi berkas serta dapat memberikan nama file atau *metadata* tambahan yang diperoleh saat pengunduhan.

Implementasi BPK dan Google Drive menunjukkan bagaimana kontrak yang sama mengisolasi perbedaan sumber. *Scraper* BPK mengubah halaman hasil pencarian menjadi entri berdasarkan tautan rincian peraturan. Pada saat pengunduhan, *scraper* membaca halaman rincian untuk memperoleh *metadata* dan tautan berkas, kemudian mengambil isi berkas tersebut. *Scraper* Google Drive mengubah hasil *listing* berkas melalui API menjadi entri dengan ID berkas Google Drive, nama file, tipe MIME, waktu perubahan, dan tautan tampilan, kemudian mengambil isi berkas melalui API pengunduhan media. Keduanya mengembalikan hasil dalam bentuk kontrak *scraper* yang sama kepada inti akuisisi.

III.2.2.2 Pemisahan Enumerasi, Pengunduhan, dan Ingestion

Proses akuisisi dibagi menjadi enumerasi, pengunduhan, validasi, deduplikasi, penyimpanan, dan pengiriman ke profil. Enumerasi bertugas menghasilkan halaman dan entri dari sumber. Pengunduhan mengambil isi berkas untuk satu entri serta, bila diperlukan, memperoleh nama berkas atau *metadata* tambahan. *Ingestion* memvalidasi dan mematerialisasikan isi berkas menjadi dokumen *datasource*.

Pemisahan ini memastikan aturan untuk berinteraksi dengan setiap sumber tetap ditangani oleh *plugin* yang bersangkutan. *Scraper* tidak perlu mengetahui cara menyimpan berkas pada MinIO atau cara membuat proses *pipeline* untuk setiap profil. Sebaliknya, layanan *ingestion* tidak perlu mengetahui apakah *byte* dokumen berasal dari halaman HTML, API, atau unggahan manual.

Unggah manual dan pengunduhan *scraper* diteruskan melalui jalur *ingestion* yang sama setelah isi berkas tersedia. Keduanya menghasilkan kandidat dokumen yang

memuat isi, nama berkas, asal, tipe isi, serta, bila tersedia, *source identifier*, URL sumber, dan *metadata*. Jalur bersama tersebut menempatkan validasi tipe dan isi berkas, perhitungan *hash*, penyimpanan, dan *fan-out* pada layanan yang sama, sehingga aturan inti akuisisi diterapkan secara konsisten pada dokumen unggahan maupun dokumen hasil *scraping*.

III.2.2.3 Validasi, Deduplikasi, dan Penyimpanan

Pada dokumen hasil *scraping*, validasi dilakukan pada dua tahap, yaitu sebelum dan sesudah pengunduhan. Sebelum pengunduhan, sistem membandingkan *source identifier* eksternal dengan dokumen yang sudah diketahui oleh *datasource*. Jika *source identifier* tersebut sudah tersimpan, entri langsung dilewati tanpa mengunduh ulang berkasnya. Jika belum ditemukan, nama berkas dibandingkan dengan daftar ekstensi yang diterima *datasource* agar berkas yang jelas tidak didukung tidak perlu diambil. Setelah isi berkas tersedia, sistem memeriksa kesesuaian antara nama dan *byte* berkas. Pemeriksaan berbasis *signature* pada bagian awal isi digunakan untuk mendeteksi keadaan seperti berkas yang bernama PDF tetapi sebenarnya berisi HTML atau format lain.

Setelah lolos validasi, nilai *hash SHA-256* dihitung dari isi berkas. Nilai *hash* digunakan untuk memeriksa kesamaan isi berkas, termasuk pada unggahan manual yang tidak memiliki *source identifier* eksternal. Pemeriksaan ini juga menangani entri dengan *source identifier* berbeda tetapi isi berkas yang sama. Dokumen yang telah ada tidak dibuat ulang dan entri *scraping* dicatat sebagai *skipped*, bukan sebagai kegagalan.

Penyimpanan isi dan *metadata* dipisahkan sesuai tanggung jawab infrastrukturnya. *Byte* dokumen disimpan pada penyimpanan objek, sedangkan informasi dokumen, *hash*, *metadata*, asal dokumen, ukuran, dan status disimpan pada basis data.

III.2.2.4 Penjadwalan, Concurrency, Pembatalan, dan Status Run

Akuisisi dapat dimulai melalui dua jalur, yaitu pemicu manual dari antarmuka atau pemicu terjadwal. Baik pemicu manual maupun pemicu terjadwal menjalankan alur akuisisi yang sama, yaitu memastikan *datasource* aktif, membuat *scrape run*,

menjaga hanya satu *run* aktif, dan menerbitkan pesan ke *worker queue*.

Scheduler menyusun jadwal berdasarkan *datasource* yang mengaktifkan *scraper* dan memiliki jadwal *cron* yang valid. Perubahan *cron*, zona waktu, pengaktifan, penonaktifan, atau pengarsipan *datasource* diselaraskan kembali oleh *scheduler* tanpa mengharuskan proses *scheduler* dimulai ulang. Jika sebuah *datasource* sudah memiliki *run* aktif atau sedang dibatalkan, pemicu terjadwal tidak membuat *run* kedua.

Scraper worker menjalankan enumerasi secara *serial* pada batas halaman, tetapi dapat mengunduh beberapa entri dalam satu halaman secara konkuren sesuai konfigurasi *datasource*. Jeda enumerasi dan pengunduhan diatur melalui konfigurasi *pacing* pada *datasource*, sehingga *plugin* dapat menerapkan jeda pada *request* yang sesuai. Kebijakan penghentian dapat meniadakan batas, membatasi jumlah halaman, atau menghentikan enumerasi setelah sejumlah halaman berturut-turut yang seluruh entrinya telah diketahui oleh *datasource*.

Setiap *run* memiliki status dan penghitung total entri, entri baru, entri yang dilewati, serta entri yang gagal. Setiap entri juga memiliki status dan riwayat transisi sendiri, mulai dari *pending*, *downloading*, dan *downloaded* hingga *completed*, *skipped*, atau *failed*. Kegagalan pengunduhan satu entri dicatat pada entri tersebut dan tidak menghentikan entri lain pada halaman yang sama. Sebaliknya, kegagalan enumerasi yang membuat halaman sumber tidak dapat dibaca dicatat sebagai kegagalan tingkat *run*. Pembatalan dilakukan secara kooperatif: entri pada halaman yang sedang diproses dapat diselesaikan, lalu *worker* menghentikan enumerasi sebelum mengambil halaman berikutnya.

III.2.2.5 Fan-out ke Profil dan Sinkronisasi Dokumen

Setelah dokumen tersimpan, layanan *fan-out* mencari profil aktif yang berlangganan *datasource* tersebut. Untuk setiap profil yang konfigurasi node *parser*-nya sesuai dengan dokumen, sistem menyertakan rujukan lokasi berkas mentah pada *object storage* agar *parser* dapat membacanya, kemudian menerbitkan permintaan *parsing* ke *parser queue*. Profil yang konfigurasi *parser*-nya tidak dapat diterapkan atau tidak mendukung ekstensi dokumen dilewati tanpa menghambat pemrosesan profil lain. Setiap profil memproses dokumen dengan konfigurasi node *parser*-nya sendiri.

Jika suatu dokumen sudah memiliki *run* pada profil tertentu, sistem tidak membuat *pipeline run* kedua untuk pasangan profil dan dokumen tersebut. Sebaliknya, profil yang baru berlangganan dapat meminta sinkronisasi dokumen yang telah tersimpan pada *datasource*.

III.2.3 Solusi *Parser Plugin*

III.2.3.1 Kontrak *Parser Plugin* dan Dokumen Kanonik

Setiap *parser plugin* menerima dokumen mentah, konfigurasi node *parser* pada profil, dan konteks layanan. Kontrak tersebut menetapkan bentuk *input* dan *output* tanpa memaksakan cara ekstraksi yang sama. Dengan demikian, *legal parser* dan *generic outline parser* dapat menggunakan strategi berbeda tetapi tetap dijalankan melalui alur *parser* yang sama.

Keluaran setiap *parser* adalah dokumen kanonik berbentuk pohon blok. Setiap blok memiliki pengenalan, tipe, teks, *metadata*, dan daftar anak, sedangkan dokumen kanonik menyimpan identitas dokumen, blok, dan *metadata* dokumen. Bentuk bersama ini menjadi batas antara *parser* dan tahap *chunking*; tahap hilir tidak perlu mengetahui sumber atau strategi yang digunakan untuk membentuk blok.

Konfigurasi dan konteks layanan memiliki tanggung jawab yang berbeda. Konfigurasi mengatur kebutuhan yang bergantung pada jenis *parser*, seperti pembacaan sumber, OCR, deteksi *layout*, atau pembacaan *outline*. Konteks layanan menyediakan kemampuan eksternal yang diperlukan, seperti pemeriksaan ketersediaan, pengenalan teks, deteksi *region*, dan penyimpanan artefak *debug*. Pada *generic outline parser*, konteks tersebut tidak digunakan karena *parser* ini tidak menggunakan OCR atau deteksi *layout*.

III.2.3.2 Ekstraksi *Legal Parser* per Halaman

Routing per halaman hanya digunakan oleh *legal parser*. PDF dibuka oleh komponen pembaca sumber yang menyediakan geometri halaman, kotak citra, fragmen teks *native*, dan kemampuan untuk mengubah halaman menjadi citra piksel. Setiap halaman kemudian diklasifikasikan secara terpisah. Halaman ditandai sebagai *SCAN* apabila cakupan citranya melewati ambang yang dikonfigurasi atau apabi-

la halaman memiliki citra tetapi jumlah teks *native*-nya berada di bawah ambang. Halaman lain masuk ke jalur *VECTOR*. Pemisahan per halaman ini memungkinkan satu PDF yang memuat halaman teks dan halaman pindai untuk menggunakan jalur ekstraksi yang berbeda.

Pada jalur *VECTOR*, fragmen teks dikelompokkan berdasarkan kedekatan posisi vertikal untuk membentuk baris. Fragmen dalam setiap baris diurutkan berdasarkan posisi horizontal, lalu spasi disisipkan apabila jarak antarfragmen menunjukkan adanya pemisah kata. Hasilnya berupa baris teks yang tetap membawa posisi halaman dan jenis jalur ekstraksinya.

III.2.3.3 OCR dan Deteksi *Layout* pada *Legal Parser*

Halaman *SCAN* diubah menjadi citra piksel pada resolusi yang dikonfigurasi sebelum dikirim ke jalur OCR. Sebelum ekstraksi dimulai, ketersediaan layanan diperiksa apabila dokumen memiliki halaman *SCAN*. Jika deteksi *layout* tidak diaktifkan, seluruh citra halaman dikirim sebagai satu permintaan OCR. Jika deteksi *layout* diaktifkan, layanan mengembalikan *region* beserta label, skor, kotak, dan urutan baca. *Region* yang *label*-nya termasuk dalam daftar pengecualian dibuang, sedangkan *region* lainnya diurutkan menurut urutan baca, dipotong dari citra halaman, dan dikirim sebagai permintaan OCR terpisah.

OCR dan deteksi *layout* ditempatkan di luar *parser* melalui konteks layanan. Konfigurasi *legal parser* mendukung dua model pengenalan teks, yaitu GLM-OCR dan PaddleOCR-VL 1.6, yang dipilih melalui konfigurasi *parser*. Kedua model tersebut dijalankan oleh server oMLX yang dihubungi langsung oleh *parser*. Klien layanan pada sisi *parser* memilih model dan parameter pengenalan teks, memeriksa ketersediaan model, serta menangani batas waktu dan percobaan ulang.

Deteksi *layout* bersifat opsional dan mendukung model PP-DocLayoutV3. Berbeda dari model OCR yang dilayani oleh oMLX, model ini dimuat dan dijalankan oleh layanan *layout* yang dibangun pada penelitian ini. Layanan tersebut menangani pemuatan model, perangkat komputasi, *concurrency*, validasi citra, serta penyaringan hasil deteksi sebelum *region* dikembalikan. Urutan baca berasal dari prediksi model; layanan hanya menomorinya ulang secara rapat atas *region* yang lolos pe-

nyaringan. Pemisahan ini membuat *parser* tidak bergantung langsung pada proses inferensi atau implementasi model tertentu. Hasil OCR dibersihkan dari penanda format yang dikeluarkan model dan dinormalisasi untuk memperbaiki penanda yang menempel pada teks. Setiap baris hasil OCR tetap menyimpan posisi halaman dan kotak *region* apabila ekstraksi dilakukan melalui deteksi *layout*.

III.2.3.4 Pemrosesan *Legal Parser* Berbasis Penanda

Setelah baris dari jalur *native* dan OCR tersedia, *legal parser* memprosesnya melalui analisis leksikal, segmentasi, *parsing* bagian, pembentukan blok, dan verifikasi. Analisis leksikal mengenali pola penanda hukum dan memberikan anotasi pada baris. Segmentasi mengelompokkan baris menjadi bagian seperti pembukaan, batang tubuh, penutup, penjelasan, dan lampiran. *Parser* bagian kemudian membentuk blok berdasarkan jenis bagian dan keadaan struktur yang sedang dibaca.

Pengenalan pola penanda dipisahkan dari pemulihan hierarki. Analisis leksikal hanya menentukan apakah sebuah baris memiliki pola penanda tertentu, sedangkan *parser* bagian menentukan peran penanda berdasarkan urutan penanda dan struktur yang masih terbuka. Pemisahan ini diperlukan karena bentuk penanda yang sama dapat berperan sebagai pembuka struktur, bagian dari ketentuan perubahan, atau kutipan terhadap struktur lain.

Blok yang dihasilkan menyimpan tipe penanda, nilai penomoran, operasi perubahan bila ada, dan lokasi halaman. Informasi tersebut disimpan sebagai *metadata* blok agar dokumen kanonik tetap menyediakan konteks yang diperlukan oleh tahap pemrosesan berikutnya.

III.2.3.5 *Generic Outline Parser* Berbasis *Bookmark*

Generic outline parser menggunakan asumsi yang berbeda dari *legal parser*. *Parser* ini membaca PDF yang memiliki teks digital dan pohon *bookmark* sebagai sumber hierarki. *Parser* ini tidak menggunakan OCR, deteksi *layout*, atau penebakan hierarki dari isi teks. Teks halaman dibaca sebagai baris berposisi, sementara setiap *bookmark* membawa judul, tingkat kedalaman, halaman tujuan, dan posisi tujuan bila tersedia.

Karena posisi tujuan *bookmark* tidak selalu tepat berada pada baris judul, judul dicari pada jendela baris di sekitar dan setelah posisi tujuan. Hasil pencarian tersebut dihubungkan dengan *bookmark*, kemudian disusun menjadi pohon bagian berdasarkan tingkat *bookmark*. Isi setiap bagian dikumpulkan dari baris yang berada di antara judulnya dan judul bagian berikutnya. Judul bagian yang dikecualikan oleh konfigurasi, seperti bagian daftar isi, dibuang bersama turunannya sebelum pohon kanonik dibentuk.

PDF tanpa *bookmark outline* ditolak karena *parser* ini tidak memiliki sumber hierarki. PDF yang memiliki *outline* tetapi tidak menghasilkan teks isi juga ditolak karena *parser* ini tidak ditujukan untuk dokumen hasil pindai. Ketidaktepatan pencocokan judul, lompatan tingkat *bookmark*, bagian kosong, dan rentang halaman yang tidak valid tetap dikumpulkan sebagai peringatan apabila dokumen masih dapat dibentuk.

III.2.3.6 Verifikasi, Peringatan, dan Kegagalan Parser

Penanganan hasil *parser* membedakan ketidakteraturan struktur dari kegagalan memproses dokumen. Peringatan digunakan untuk temuan yang masih memungkinkan dokumen kanonik dibentuk, seperti ketidakteraturan penanda dan rentang halaman pada *legal parser* atau masalah pencocokan *bookmark* dan struktur blok pada *generic outline parser*. Peringatan tidak mengubah isi sumber dan tidak membatalkan dokumen yang berhasil dibentuk.

Diagnostik digunakan untuk mencatat kondisi sumber, halaman, layanan, dan tahapan pemrosesan, termasuk lokasi temuan bila tersedia. Kegagalan membuka PDF, layanan OCR atau *layout* yang tidak tersedia, konfigurasi *plugin* yang tidak valid, dan keluaran yang melanggar kontrak *parser* diperlakukan sebagai kegagalan pemrosesan. Kegagalan tersebut dicatat bersama tahap, sumber, pesan, dan sifat dapat-diulanginya sehingga dapat dibedakan dari dokumen yang hanya memerlukan peninjauan struktur.

Kode peringatan pada *legal parser* menyatakan kondisi struktur yang ditemukan setelah dokumen kanonik dibentuk. Kode tersebut menjelaskan jenis temuan, tetapi belum menentukan klasifikasi *advisory* atau *severe* yang digunakan pada analisis

eksperimen. Makna setiap kode peringatan dirangkum pada Tabel III.7.

Tabel III.7: Kode peringatan dan kondisi yang ditandai oleh *legal parser*

Kode	Kondisi yang ditandai
sequence_gap	Urutan penanda hukum melewati nilai yang diharapkan, misalnya nomor pasal atau ayat tidak berurutan.
BODY_STRUCTURAL_MARKER_IN_REVISION	Penanda struktur muncul di dalam konteks perubahan dan diperlakukan sebagai isi perubahan, bukan sebagai struktur utama dokumen.
pembukaan_incomplete	Bagian pembukaan tidak memuat salah satu unsur yang diperiksa, seperti konsiderans, dasar hukum, atau perintah menetapkan.
empty_leaf	Blok daun tidak memiliki teks maupun anak sehingga bagian struktur yang dihasilkan tidak memiliki isi.
revise_pasal_no_children	Blok pasal perubahan tidak memiliki anak perubahan yang diharapkan, seperti angka perubahan atau pasal.
revise_pasal_count	Jumlah blok pasal perubahan pada batang tubuh melebihi batas yang ditetapkan parser.
location_invalid	Rentang halaman suatu blok tidak valid karena halaman awal lebih besar daripada halaman akhir.
empty_batang_tubuh	Bagian batang tubuh tidak berhasil dibentuk atau tidak memiliki isi yang dapat diproses.

Pada *generic outline parser*, PDF tanpa *bookmark outline* atau tanpa teks yang dapat diekstraksi ditolak sebagai kegagalan sumber, bukan dicatat sebagai peringatan. Bagian yang dikecualikan secara sengaja oleh konfigurasi, seperti *Contents*, juga dicatat sebagai *metadata* dan bukan sebagai peringatan. Kode peringatan *generic outline parser* dirangkum pada Tabel III.8.

Tabel III.8: Kode peringatan dan kondisi yang ditandai oleh *generic outline parser*

Kode	Kondisi yang ditandai
<code>empty_leaf</code>	<i>Heading</i> tidak memiliki isi maupun subseksi, atau <i>text block</i> tidak memiliki teks.
<code>location_invalid</code>	Rentang halaman suatu blok terbalik karena halaman awal lebih besar daripada halaman akhir.
<code>location_out_of_range</code>	Rentang halaman suatu blok berada di luar jumlah halaman dokumen.
<code>location_not_nested</code>	Rentang halaman suatu bagian anak tidak berada di dalam rentang halaman bagian induknya.
<code>outline.empty_section</code>	Suatu bagian hasil <i>bookmark</i> tidak memiliki teks maupun subseksi setelah proses perakitan.
<code>outline.anchor_not_found</code>	Tidak ditemukan baris yang dapat digunakan pada atau setelah tujuan <i>bookmark</i> , sehingga bagian tersebut menjadi kosong.
<code>outline.anchor_out_of_order</code>	Baris judul yang cocok berada pada atau sebelum bagian sebelumnya sehingga hasil pencocokan dibuang untuk menjaga urutan bagian.
<code>outline.page_out_of_range</code>	Tujuan <i>bookmark</i> menunjuk halaman di luar dokumen sehingga <i>bookmark</i> dibuang.
<code>outline.anchor_unmatched_title</code>	Judul <i>bookmark</i> tidak cocok dengan baris pada jendela pencarian sehingga baris pertama setelah tujuan digunakan sebagai <i>fallback</i> .
<code>outline.destination_missing</code>	<i>Bookmark</i> tidak memiliki tujuan yang dapat digunakan sehingga awal bagiannya ditempatkan pada bagian atas halaman.
<code>outline.level_skipped</code>	Tingkat <i>bookmark</i> melompat lebih dari satu tingkat sehingga kedalamannya dirapatkan ke tingkat berikutnya yang valid.
<code>outline.page_rotated</code>	Halaman memiliki rotasi sehingga konversi koordinat tujuan dapat menghasilkan posisi judul yang tidak tepat.

III.2.3.7 Materialisasi Dokumen Kanonik dan *Handoff*

Pada saat pesan pemrosesan diterima, *parser worker* mengambil nama *plugin* dan konfigurasi *parser* dari profil, bukan mempercayainya dari pesan. Konfigurasi tersebut divalidasi terhadap skema *plugin* sebelum *parser* dijalankan. Berkas mentah dibaca dari penyimpanan objek berdasarkan rujukan lokasinya, kemudian *hash* aktualnya dibandingkan dengan nilai yang dikirim pada pesan. Pemeriksaan ini memastikan *parser* memproses berkas yang sama dengan berkas yang diterima pada tahap akuisisi.

Setelah *parser* menghasilkan keluaran, dokumen tersebut divalidasi sebagai dokumen kanonik, digabungkan dengan *metadata* sumber, dan disimpan sebagai artefak dokumen kanonik. Artefak tersebut menyimpan identitas dokumen, profil, proses, pembuat, *hash*, dan jumlah blok.

Dalam alur *pipeline* penuh, setelah artefak berhasil disimpan, *parser worker* mener-

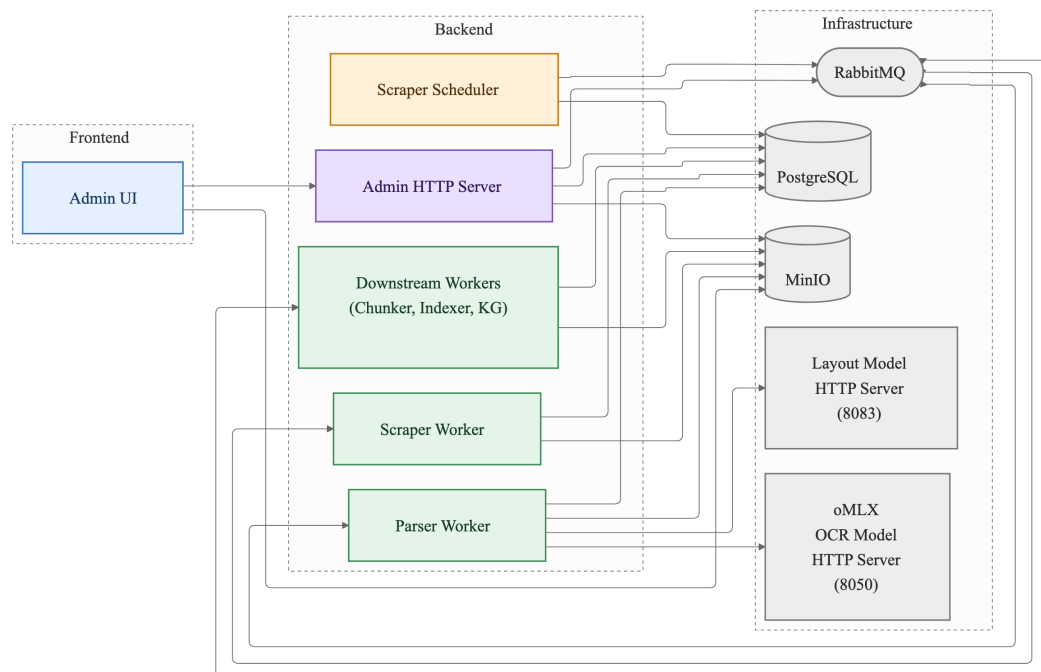
bitkan pesan ke *chunker queue* agar *chunker* dapat melanjutkan pemrosesan dokumen. Dengan demikian, tahap hilir hanya bergantung pada dokumen kanonik dan tidak perlu mengetahui apakah isi dokumen berasal dari teks *native*, OCR, penanda hukum, atau *bookmark outline*.

III.3 Rancangan Solusi

Bab ini menerjemahkan solusi konseptual pada bagian sebelumnya menjadi rancangan implementasi yang digunakan pada sistem OMNI-RAG. Rancangan yang dibahas mencakup *service*, modul, penyimpanan, kontrak data, *lifecycle*, alur pesan, dan antarmuka administrasi yang berkaitan dengan *datasource*, *scraper*, *parser*, dan OCR. Nama komponen, tabel, *queue*, serta *field* kontrak ditampilkan untuk menjelaskan bentuk konkret sistem tanpa menampilkan *source code* implementasi.

III.3.1 Rancangan Arsitektur Implementasi

III.3.1.1 Arsitektur *Deployment* dan *Service*



Gambar III.9: Arsitektur *deployment* dan *service* pada alur *datasource* sampai *parser*

Gambar III.9 memperlihatkan pembagian komponen implementasi ke dalam tiga

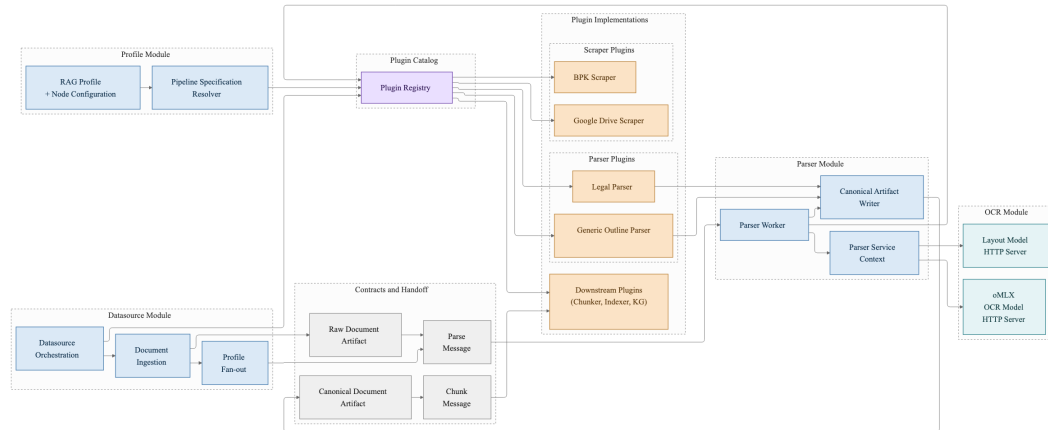
kelompok. Kelompok *Frontend* berisi *Admin UI* untuk mengelola konfigurasi dan proses. Kelompok *Backend* berisi *Admin HTTP Server*, *Scraper Scheduler*, *Scraper Worker*, *Parser Worker*, serta konteks *Downstream Workers* yang mencakup *chunker*, *indexer*, dan *KG worker*. Kelompok *Infrastructure* berisi PostgreSQL, RabbitMQ, MinIO, *oMLX OCR Model HTTP Server*, dan *Layout Model HTTP Server*.

Admin UI berkomunikasi dengan *Admin HTTP Server* untuk mengelola konfigurasi dan proses. *Admin HTTP Server* menggunakan PostgreSQL sebagai basis data, MinIO sebagai penyimpanan berkas dan artefak, serta RabbitMQ sebagai perantara pesan. Untuk membuka atau mengunduh dokumen, *Admin UI* meminta URL akses kepada *Admin HTTP Server*, kemudian membaca berkas secara langsung dari MinIO menggunakan *presigned URL*.

Scraper Scheduler membaca jadwal dari PostgreSQL dan mengirimkan permintaan akuisisi ke RabbitMQ. RabbitMQ kemudian mengirimkan permintaan tersebut kepada *Scraper Worker*. *Scraper Worker* menjalankan *scraper plugin*, menyimpan informasi *run* dan *entry* pada PostgreSQL, serta menyimpan berkas mentah pada MinIO. Setelah dokumen melewati *ingestion* dan *fan-out*, permintaan parsing dikirimkan melalui RabbitMQ kepada *Parser Worker*. *Parser Worker* mencatat proses *pipeline* pada PostgreSQL, membaca berkas mentah dari MinIO, menjalankan *parser plugin*, lalu menyimpan artefak dokumen kanonik pada MinIO. Hasil tersebut kemudian diteruskan ke tahap *downstream* melalui RabbitMQ.

Jalur OCR hanya digunakan apabila dibutuhkan oleh *parser plugin*. *Legal parser* menghubungi *oMLX OCR Model HTTP Server* apabila dokumen memiliki halaman pindai, sedangkan deteksi *layout* dilakukan melalui *Layout Model HTTP Server* jika diaktifkan pada konfigurasi. *Generic outline parser* tidak menggunakan kedua layanan tersebut karena memanfaatkan teks digital dan *bookmark outline*. *oMLX OCR Model HTTP Server* menjalankan model OCR, sedangkan *Layout Model HTTP Server* menjalankan PP-DocLayoutV3. Arah panah pada gambar menunjukkan alur komunikasi tingkat layanan, sedangkan detail struktur pesan dan urutan konsumsi setiap *queue* dibahas pada diagram *sequence*.

III.3.1.2 Arsitektur Modul dan Komponen



Gambar III.10: Komponen modul dan kontrak pada *datasource*, *profile*, dan *parser*

Gambar III.10 memetakan komponen yang membentuk alur akuisisi dan ekstraksi. Komponen inti dibagi menjadi *Datasource Module*, *Profile Module*, dan *Parser Module*. *Datasource Module* terdiri atas *Datasource Orchestration*, *Document Ingestion*, dan *Profile Fan-out*. *Datasource Orchestration* mengatur penggunaan *scraper plugin*, *Document Ingestion* memvalidasi serta menyimpan dokumen mentah, sedangkan *Profile Fan-out* meneruskan dokumen yang berhasil disimpan kepada *profile* yang berlangganan.

Profile Module menyimpan *RAG Profile* beserta konfigurasi *pipeline node*-nya. Konfigurasi tersebut mencakup *datasource* dan *node* untuk *parser*, *chunker*, *indexer*, *retrieval orchestrator*, serta *knowledge graph* apabila digunakan. *Pipeline Specification Resolver* membaca konfigurasi tersebut, memeriksa kecocokan antar-*node*, dan meminta *Plugin Registry* untuk me-*resolve plugin* yang dipilih beserta konfigurasi dan *schema*-nya.

Plugin Registry berfungsi sebagai katalog implementasi, bukan tempat kode *plugin* dijalankan. *Plugin Registry* menyimpan identitas *plugin*, versi, *manifest*, *schema* konfigurasi, dan referensi implementasinya. Implementasi BPK dan Google Drive ditempatkan sebagai *scraper plugin*, sedangkan *legal parser* dan *generic outline parser* ditempatkan sebagai *parser plugin*. *Downstream Plugins* ditampilkan secara agregat karena implementasinya berada di luar fokus pembahasan, tetapi tetap menggunakan katalog yang sama.

Parser Module terdiri atas *Parser Worker*, *Parser Service Context*, dan *Canonical Artifact Writer*. *Parser Worker* menerima *Parse Message*, meminta spesifikasi *pipeline* dari *resolver*, lalu menjalankan *parser plugin* yang sesuai. *Parser Service Context* menyediakan akses ke kemampuan OCR dan deteksi *layout* tanpa menempatkan implementasi layanan tersebut di dalam *plugin*. *Canonical Artifact Writer* menyimpan keluaran *plugin* sebagai *Canonical Document Artifact* yang dapat digunakan oleh tahap berikutnya.

Kelompok *Contracts and Handoff* memperlihatkan bentuk pertukaran antarkomponen. *Raw Document Artifact* menjadi masukan bagi *Parse Message*, sedangkan *Canonical Document Artifact* menjadi masukan bagi *Chunk Message*. *OCR Module* menghubungkan dua layanan terpisah; pengenalan teks dijalankan oleh *oMLX OCR Model HTTP Server*, sedangkan PP-DocLayoutV3 dijalankan oleh *Layout Model HTTP Server* untuk deteksi *layout*.

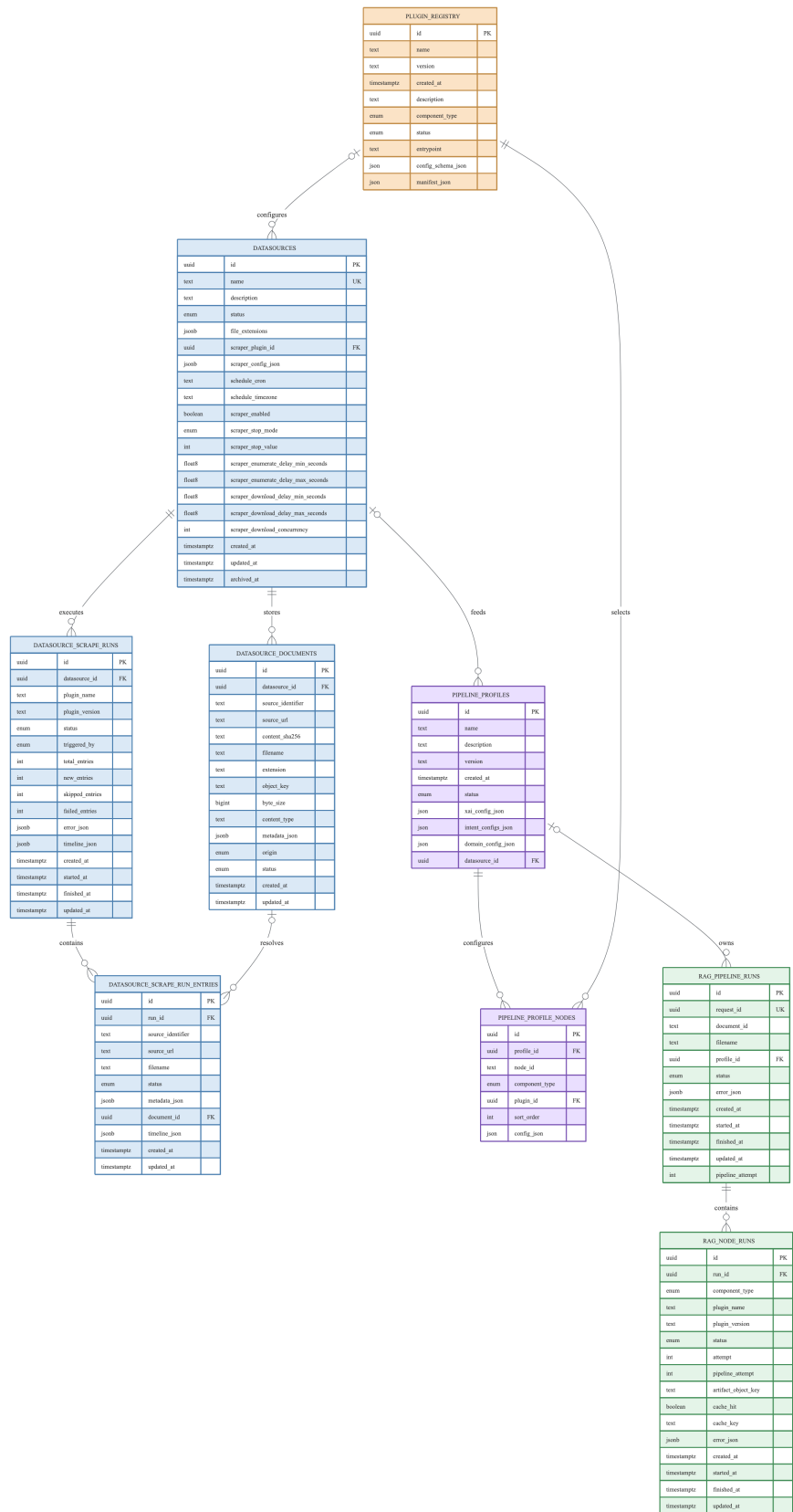
III.3.2 Rancangan Data, Schema, dan Kontrak

III.3.2.1 ERD Basis Data

Gambar III.11 menunjukkan struktur fisik sembilan tabel yang digunakan untuk menyimpan *datasource*, konfigurasi *profile*, *plugin registry*, serta riwayat pemrosesan. Warna biru digunakan untuk kelompok *datasource*, warna ungu untuk kelompok *profile*, warna jingga untuk *plugin registry*, dan warna hijau untuk *monitoring pipeline*.

Datasource menjadi induk bagi dokumen yang tersimpan dan riwayat *scrape run*. Satu *scrape run* memiliki banyak *scrape entry*, sedangkan *entry* dapat menunjuk pada dokumen yang berhasil disimpan atau dokumen yang sudah ada karena deduplikasi. *Datasource* dapat menggunakan satu *scraper plugin*; hubungan ini bersifat opsional karena *datasource* yang hanya menerima unggahan tidak memiliki *scraper plugin*.

Profile dapat memilih satu *datasource* dan memiliki banyak *pipeline node*. Setiap *node* memilih satu baris pada *plugin registry*. *RAG pipeline run* dapat merujuk pada satu *profile* dan memiliki banyak *node run*.



Gambar III.11: ERD entitas *datasource*, *profile*, *plugin*, dan *monitoring pipeline*

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
-------	-----------------	----------------	-----------

Tabel III.9: *Schema* tabel *datasources*

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
<i>id</i>	<i>uuid</i>	PK, NOT NULL	Identitas <i>datasource</i> .
<i>name</i>	<i>text</i>	UNIQUE, NOT NULL	Nama <i>datasource</i> .
<i>description</i>	<i>text</i>	NULL	Deskripsi <i>datasource</i> .
<i>status</i>	<i>datasource_status</i>	NOT NULL	Status <i>active</i> atau <i>archived</i> .
<i>file_extensions</i>	<i>jsonb</i>	NOT NULL	Daftar ekstensi berkas yang diterima.
<i>scraper_plugin_id</i>	<i>uuid</i>	FK ke <i>plugin_registry.id</i> , NULL	<i>Scraper plugin</i> yang digunakan; kosong untuk <i>upload-only datasource</i> .
<i>scraper_config_json</i>	<i>jsonb</i>	NULL	Konfigurasi <i>scraper plugin</i> .
<i>schedule_cron</i>	<i>text</i>	NULL	Ekspresi jadwal <i>cron</i> .
<i>schedule_timezone</i>	<i>text</i>	NULL	Zona waktu jadwal.
<i>scraper_enabled</i>	<i>boolean</i>	NULL	Penanda apakah jadwal <i>scraper</i> dijalankan.
<i>scraper_stop_mode</i>	<i>datasource_scrape_stop_mode</i>	NULL	Kebijakan penghentian: <i>none</i> , <i>max_pages</i> , atau <i>known_pages</i> .
<i>scraper_stop_value</i>	<i>integer</i>	NULL	Nilai batas halaman untuk mode penghentian selain <i>none</i> .
<i>scraper_enumerate_delay_min_seconds</i>	<i>doubleprecision</i>	NULL	Jeda minimum antarpermintaan enumerasi.
<i>scraper_enumerate_delay_max_seconds</i>	<i>doubleprecision</i>	NULL	Jeda maksimum antarpermintaan enumerasi.
<i>scraper_download_delay_min_seconds</i>	<i>doubleprecision</i>	NULL	Jeda minimum antarpermintaan pengunduhan.
<i>scraper_download_delay_max_seconds</i>	<i>doubleprecision</i>	NULL	Jeda maksimum antarpermintaan pengunduhan.
<i>scraper_download_concurrency</i>	<i>integer</i>	NULL	Jumlah pengunduhan konkuren yang diizinkan.
<i>created_at</i>	<i>timestampz</i>	NOT NULL	Waktu pembuatan baris.
<i>updated_at</i>	<i>timestampz</i>	NOT NULL	Waktu perubahan terakhir.
<i>archived_at</i>	<i>timestampz</i>	NULL	Waktu pengarsipan <i>datasource</i> .

Constraint tabel ini memastikan nama *datasource* unik, daftar ekstensi tidak kosong, serta konfigurasi *scraper_config_json*, *schedule_timezone*, *scraper_enabled*, *scraper_stop_mode*, *pacing*, dan *download concurrency* diisi atau dikosongkan bersama *scraper_plugin_id*. Kebijakan penghentian *scraper* dengan mode *none* tidak menetapkan batas halaman, sedangkan mode lain memerlukan nilai minimal satu. Rentang *pacing* harus bernilai nonnegatif danurut, sedangkan *download concurrency* minimal bernilai satu.

Tabel III.10: *Schema* tabel `datasource_documents`

<i>Field</i>	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
<code>id</code>	<code>uuid</code>	PK, NOT NULL	Identitas dokumen di dalam <i>datasource</i> .
<code>datasource_id</code>	<code>uuid</code>	FK ke <code>datasources.id</code> , NOT NULL	<i>Datasource</i> pemilik dokumen.
<code>source_identifier</code>	<code>text</code>	NULL	<i>Source identifier</i> dokumen pada sumber eksternal; kosong untuk unggahan manual.
<code>source_url</code>	<code>text</code>	NULL	URL asal dokumen jika tersedia.
<code>content_sha256</code>	<code>text</code>	NOT NULL	Hash SHA-256 isi berkas dalam bentuk heksadesimal.
<code>filename</code>	<code>text</code>	NOT NULL	Nama berkas yang disimpan sebagai <i>metadata</i> .
<code>extension</code>	<code>text</code>	NOT NULL	Ekstensi berkas yang telah dinormalisasi.
<code>object_key</code>	<code>text</code>	NOT NULL	Lokasi berkas pada <i>object storage</i> .
<code>byte_size</code>	<code>bigint</code>	NOT NULL	Ukuran isi berkas dalam <i>byte</i> .
<code>content_type</code>	<code>text</code>	NOT NULL	Tipe media berkas.
<code>metadata_json</code>	<code>jsonb</code>	NOT NULL	<i>Metadata</i> tambahan dari sumber atau proses <i>ingestion</i> .
<code>origin</code>	<code>datasource_document_origin</code>	NOT NULL	Asal dokumen: <i>scraper</i> atau <i>upload</i> .
<code>status</code>	<code>datasource_document_status</code>	NOT NULL	Status dokumen: <i>active</i> atau <i>archived</i> .
<code>created_at</code>	<code>timestampz</code>	NOT NULL	Waktu dokumen disimpan.
<code>updated_at</code>	<code>timestampz</code>	NOT NULL	Waktu perubahan terakhir.

Constraint unik pada pasangan `datasource_id` dan `source_identifier` mencegah pengulangan dokumen dari sumber yang sama. Pasangan `datasource_id` dan `content_sha256` mencegah penyimpanan ulang isi yang sama. Nilai `source_identifier` yang NULL tetap diperbolehkan untuk banyak unggahan manual, sedangkan `content_sha256` harus berupa 64 karakter heksadesimal huruf kecil.

Tabel III.11: *Schema* tabel `datasource_scrape_runs`

<i>Field</i>	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
<code>id</code>	<code>uuid</code>	PK, NOT NULL	Identitas satu proses <i>scrape</i> .
<code>datasource_id</code>	<code>uuid</code>	FK ke <code>datasources.id</code> , NOT NULL	<i>Datasource</i> yang diproses.
<code>plugin_name</code>	<code>text</code>	NOT NULL	Nama <i>scraper plugin</i> yang digunakan.

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
plugin_version	text	NOT NULL	Versi <i>scraper plugin</i> yang digunakan.
status	datasource_scrape_run_status	NOT NULL	Status: <i>queued</i> , <i>running</i> , <i>cancelling</i> , <i>completed</i> , <i>failed</i> , atau <i>cancelled</i> .
triggered_by	datasource_scrape_trigger	NOT NULL	Pemicu proses: <i>manual</i> atau <i>schedule</i> .
total_entries	integer	NOT NULL	Jumlah seluruh entri yang ditemukan.
new_entries	integer	NOT NULL	Jumlah entri yang menghasilkan dokumen baru.
skipped_entries	integer	NOT NULL	Jumlah entri yang dilewati.
failed_entries	integer	NOT NULL	Jumlah entri yang gagal.
error_json	jsonb	NULL	Informasi kegagalan pada tingkat <i>run</i> .
timeline_json	jsonb	NOT NULL	Riwayat perubahan status <i>run</i> .
created_at	timestampz	NOT NULL	Waktu <i>run</i> dibuat.
started_at	timestampz	NULL	Waktu <i>worker</i> mulai menjalankan <i>run</i> .
finished_at	timestampz	NULL	Waktu <i>run</i> mencapai status <i>terminal</i> .
updated_at	timestampz	NOT NULL	Waktu perubahan terakhir.

Indeks unik parsial pada *datasource_id* membatasi satu *run* berstatus aktif, yaitu *queued*, *running*, atau *cancelling*, untuk setiap *datasource*.

Tabel III.12: *Schema* tabel *datasource_scrape_run_entries*

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas satu entri hasil enumerasi.
run_id	uuid	FK ke <i>datasource_scrape_runs.id</i> , NOT NULL	<i>Scrape run</i> yang menghasilkan entri.
source_identifier	text	NOT NULL	<i>Source identifier</i> entri pada sumber eksternal.
source_url	text	NULL	URL sumber entri jika tersedia.
filename	text	NOT NULL	Nama berkas yang ditemukan atau diunduh.
status	datasource_scrape_entry_status	NOT NULL	Status: <i>pending</i> , <i>downloading</i> , <i>downloaded</i> , <i>completed</i> , <i>skipped</i> , atau <i>failed</i> .
metadata_json	jsonb	NOT NULL	<i>Metadata</i> yang dibawa dari sumber eksternal.

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
document_id	uuid	FK ke datasource_ documents.id, NULL	Dokumen yang menjadi hasil resolusi entri.
timeline_json	jsonb	NOT NULL	Riwayat perubahan status entri.
created_at	timestampz	NOT NULL	Waktu entri dibuat.
updated_at	timestampz	NOT NULL	Waktu perubahan terakhir.

Constraint unik pada pasangan run_id dan source_identifier mencegah satu sumber tercatat dua kali pada *run* yang sama.

Tabel III.13: *Schema* tabel pipeline_profiles

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas <i>profile</i> .
name	text	NOT NULL	Nama <i>profile</i> .
description	text	NULL	Deskripsi <i>profile</i> .
version	text	NOT NULL	Versi konfigurasi <i>profile</i> yang disimpan oleh implementasi.
created_at	timestampz	NOT NULL	Waktu <i>profile</i> dibuat.
status	pipeline_profile_ status	NOT NULL	Status: draft, active, atau disabled.
xai_config_json	json	NOT NULL	Konfigurasi kemampuan <i>explainability</i> .
intent_configs_json	json	NOT NULL	Konfigurasi aksi dan strategi <i>intent</i> .
domain_config_json	json	NOT NULL	Konfigurasi domain dan perilaku <i>pipeline</i> .
datasource_id	uuid	FK ke datasources.id, NULL	<i>Datasource</i> yang menjadi sumber dokumen <i>profile</i> .

Constraint unik pada pasangan name dan version mencegah dua *profile* dengan identitas konfigurasi yang sama.

Tabel III.14: *Schema* tabel pipeline_profile_nodes

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas <i>node</i> pada penyimpanan.
profile_id	uuid	FK ke pipeline_ profiles.id, NOT NULL	<i>Profile</i> pemilik <i>node</i> .
node_id	text	NOT NULL	Identitas <i>node</i> dalam urutan <i>pipeline</i> .

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
component_type	component_type	NOT NULL	Jenis komponen: <i>scraper</i> , <i>parser</i> , <i>chunker</i> , <i>indexer</i> , <i>kg</i> , <i>retrieval_orchestrator</i> , <i>xai_intent</i> , <i>xai_retrieval</i> , atau <i>xai_reranking</i> .
plugin_id	uuid	FK ke plugin_registry.id, NOT NULL	<i>Plugin</i> yang dipilih untuk <i>node</i> .
sort_order	integer	NOT NULL	Urutan eksekusi <i>node</i> .
config_json	json	NOT NULL	Konfigurasi <i>node</i> yang divalidasi oleh <i>plugin schema</i> .

Constraint unik pada pasangan *profile_id* dan *node_id* mencegah *node* bernama sama dalam satu *profile*. Pasangan *profile_id* dan *sort_order* juga harus unik agar satu posisi hanya ditempati satu *node*.

Tabel III.15: *Schema* tabel plugin_registry

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas implementasi <i>plugin</i> .
name	text	NOT NULL	Nama <i>plugin</i> .
version	text	NOT NULL	Versi <i>plugin</i> .
created_at	timestampz	NOT NULL	Waktu baris <i>plugin</i> dibuat.
description	text	NULL	Deskripsi fungsi <i>plugin</i> .
component_type	component_type	NOT NULL	Jenis komponen yang diimplementasikan: <i>scraper</i> , <i>parser</i> , <i>chunker</i> , <i>indexer</i> , <i>kg</i> , <i>retrieval_orchestrator</i> , <i>xai_intent</i> , <i>xai_retrieval</i> , atau <i>xai_reranking</i> .
status	plugin_status	NOT NULL	Status: <i>active</i> atau <i>disabled</i> .
entrypoint	text	NOT NULL	Referensi implementasi yang di-resolve oleh sistem.
config_schema_json	json	NOT NULL	<i>Schema</i> konfigurasi <i>plugin</i> .
manifest_json	json	NOT NULL	<i>Metadata manifest plugin</i> .

Constraint unik pada *component_type*, *name*, dan *version* memastikan satu implementasi tidak terdaftar dua kali pada tipe komponen yang sama.

Tabel III.16: *Schema* tabel rag_pipeline_runs

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas satu proses <i>RAG pipeline</i> .

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
request_id	uuid	UNIQUE, NOT NULL	Identitas permintaan pemrosesan.
document_id	text	NULL	Referensi logis ke dokumen yang diproses.
filename	text	NOT NULL	Nama berkas yang terkait dengan proses.
profile_id	uuid	FK ke pipeline_profiles.id, NULL	Profile yang menjalankan proses.
status	rag_run_status	NOT NULL	Status: <i>queued</i> , <i>running</i> , <i>succeeded</i> , <i>failed</i> , atau <i>cancelled</i> .
error_json	jsonb	NULL	Informasi kegagalan proses.
created_at	timestampz	NOT NULL	Waktu proses dibuat.
started_at	timestampz	NULL	Waktu proses dimulai.
finished_at	timestampz	NULL	Waktu proses selesai atau berhenti.
updated_at	timestampz	NOT NULL	Waktu perubahan terakhir.
pipeline_attempt	integer	NOT NULL	Nomor percobaan keseluruhan <i>pipeline</i> .

Constraint unik pada pasangan *profile_id* dan *document_id* mencegah satu dokumen memiliki lebih dari satu *pipeline run* pada *profile* yang sama. *Constraint* tersebut menggunakan *NULLSNOTDISTINCT*, sehingga nilai *NULL* juga diperlakukan sebagai bagian dari keunikan.

Tabel III.17: *Schema* tabel *rag_node_runs*

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
id	uuid	PK, NOT NULL	Identitas proses satu <i>node</i> .
run_id	uuid	FK ke rag_pipeline_runs.id, NOT NULL	<i>Pipeline run</i> pemilik <i>node</i> .
component_type	rag_component_type	NOT NULL	Jenis tahap: <i>parser</i> , <i>chunker</i> , <i>indexer</i> , atau <i>kg</i> .
plugin_name	text	NULL	Nama <i>plugin</i> yang dijalankan.
plugin_version	text	NULL	Versi <i>plugin</i> yang dijalankan.
status	rag_run_status	NOT NULL	Status proses <i>node</i> .
attempt	integer	NOT NULL	Nomor percobaan <i>node</i> .
pipeline_attempt	integer	NOT NULL	Nomor percobaan <i>pipeline</i> yang menaungi <i>node</i> .
artifact_object_key	text	NULL	Lokasi <i>artifact</i> keluaran pada <i>object storage</i> .
cache_hit	boolean	NULL	Penanda bahwa keluaran diperoleh dari <i>cache</i> .
cache_key	text	NULL	Kunci pencarian <i>cache</i> .
error_json	jsonb	NULL	Informasi kegagalan <i>node</i> .

Bersambung ke halaman berikutnya

(lanjutan)

Field	Tipe PostgreSQL	Kunci dan NULL	Deskripsi
created_at	timestampz	NOT NULL	Waktu proses <i>node</i> dibuat.
started_at	timestampz	NULL	Waktu proses <i>node</i> dimulai.
finished_at	timestampz	NULL	Waktu proses <i>node</i> selesai.
updated_at	timestampz	NOT NULL	Waktu perubahan terakhir.

Constraint unik pada *run_id*, *component_type*, dan *attempt* menjaga agar satu tahap dan satu nomor percobaan tidak tercatat dua kali pada *pipeline run*.

III.3.2.2 Skema *Message* dan Artefak *Pipeline*

Subsubbab ini menjelaskan kontrak yang menghubungkan permintaan pemrosesan dengan data yang diproses. *Message* pada RabbitMQ meminta suatu *stage* kepada *service* yang sesuai, sedangkan *artifact* menyimpan data yang dibaca atau dihasilkan oleh *stage* tersebut. Isi berkas tidak disisipkan ke dalam *message*; berkas disimpan pada MinIO dan *message* hanya membawa identitas, metadata, serta rujukan lokasi.

Konfigurasi *plugin* juga tidak dikirim melalui RabbitMQ. *Scraper worker* mengambil konfigurasi dari *datasource*, sedangkan *parser worker* mengambil konfigurasi dari *profile* ketika menangani *message*. Dengan demikian, isi *message* berfokus pada identitas pekerjaan dan data yang diperlukan untuk menemukan *artifact*, sementara konfigurasi yang digunakan tetap mengikuti sumber konfigurasi pada basis data.

Pesan akuisisi menggunakan skema *ScrapeEventEnvelope* untuk mengidentifikasi satu *scrape run*. Pesan *parser* menggunakan skema *NodeEventEnvelope* untuk mengaitkan permintaan dengan *pipeline node*, *stage*, dan percobaan tertentu.

Tabel III.18: Skema *ScrapeEventEnvelope*

Field	Tipe atau nilai	Fungsi
event_id	String	Identitas unik <i>event</i> .
schema_version	Literal "2"	Versi kontrak <i>envelope</i> .
run_id	String	Identitas <i>scrape run</i> .
requested_at	<i>Datetime</i>	Waktu permintaan dibuat.

Tabel III.19: Skema NodeEventEnvelope

Field	Tipe atau nilai	Fungsi
event_id	String	Identitas unik <i>event</i> .
schema_version	Literal "2"	Versi kontrak <i>envelope</i> .
run_id	String	Identitas <i>pipeline run</i> .
requested_at	Datetime	Waktu permintaan dibuat.
request_id	String	Identitas permintaan tahap.
node_id	String	Identitas <i>node</i> yang dituju.
attempt	Integer, minimal 1	Nomor percobaan node.

Tabel III.20: Skema RagScrapeRequestedMessage

Field	Tipe atau nilai	Fungsi
idempotency_key	String, bawaan kosong	Kunci untuk mencegah pemrosesan permintaan yang sama.
event	ScrapeEventEnvelope	Struktur yang membawa identitas <i>scrape run</i> .
datasource_id	String	Identitas <i>datasource</i> yang diproses.
scrape_run_id	String	Identitas <i>scrape run</i> yang telah dibuat pada basis data.

Tabel III.21: Skema RagParseRequestedMessage

Field	Tipe atau nilai	Fungsi
idempotency_key	String, bawaan kosong	Kunci idempotensi untuk tahap <i>parser</i> .
event	NodeEventEnvelope	Struktur yang membawa identitas <i>pipeline run</i> , <i>node</i> , dan percobaan.
datasource_id	String	Identitas <i>datasource</i> asal dokumen.
filename	String	Nama berkas mentah.
document_metadata	Object, bawaan {}	Metadata dokumen dari <i>datasource</i> .
source_artifact	ArtifactReference	Rujukan berkas mentah yang dibaca oleh <i>parser</i> .

Pada pesan *parser*, *source_artifact* membawa rujukan berkas mentah dan menggunakan tipe *ArtifactReference*, sehingga *message* membawa rujukan serta metadata integritas tanpa membawa isi berkas. Kontrak untuk tahap setelah *parser*, termasuk *chunking*, tidak dirinci pada subsubbab ini karena berada di luar *scope* tugas akhir dan diimplementasikan pada *capstone* lain.

ArtifactReference adalah struktur rujukan yang menghubungkan *message* dengan *artifact* pada *object storage*. Pembahasan pada subsubbab ini berfokus pada raw-document sebagai masukan *parser* dan canonical-document sebagai kelu-
arannya.

Tabel III.22: Skema ArtifactReference

<i>Field</i>	Tipe atau nilai	NULL	Fungsi
<code>contract</code>	Pilihan kontrak	NOT NULL	Membedakan jenis <i>artifact</i> yang dirujuk.
<code>schema_version</code>	Literal "1"	NOT NULL	Versi <i>schema artifact</i> .
<code>object_key</code>	String	NOT NULL	Lokasi <i>artifact</i> pada <i>object storage</i> .
<code>document_id</code>	String	NOT NULL	Identitas dokumen yang menjadi dasar <i>artifact</i> .
<code>profile_id</code>	String	Ya	Identitas <i>profile</i> pemilik hasil proses.
<code>run_id</code>	String	NOT NULL	Identitas <i>pipeline run</i> yang menghasilkan atau menggunakan <i>artifact</i> .
<code>producer</code>	<i>Plugin reference</i>	Ya	Nama dan versi <i>plugin</i> pembuat <i>artifact</i> .
<code>item_count</code>	Integer, minimal 0	Ya	Jumlah item yang dihasilkan tahap tersebut.
<code>sha256</code>	String 64 karakter	NOT NULL	<i>Checksum payload artifact</i> .
<code>content_sha256</code>	String 64 karakter	Ya	<i>Hash</i> isi semantik <i>artifact</i> apabila tersedia.

Tabel III.23: Skema dokumen kanonik

Elemen	<i>Field</i>	Deskripsi
Akar dokumen	<code>document_id</code>	Identitas dokumen kanonik.
Akar dokumen	<code>blocks</code>	Daftar <i>block</i> yang membentuk representasi hierarkis.
Akar dokumen	<code>metadata</code>	Metadata dokumen yang digabungkan dari sumber dan <i>parser</i> .
<i>Block</i>	<code>block_id</code>	Identitas unik <i>block</i> .
<i>Block</i>	<code>type</code>	Jenis <i>block</i> , misalnya <code>HEADING</code> atau <code>TEXT_BLOCK</code> .
<i>Block</i>	<code>text</code>	Isi teks <i>block</i> ; dapat bernilai <code>NULL</code> .
<i>Block</i>	<code>metadata</code>	Metadata struktur, lokasi halaman, <i>marker</i> , atau operasi perubahan.
<i>Block</i>	<code>children</code>	Daftar <i>block</i> turunan untuk membentuk hierarki.

Setelah *parser plugin* menghasilkan keluaran, *parser* memvalidasi struktur tersebut sebagai dokumen kanonik sebelum menyimpannya. Keluaran yang lolos validasi disimpan sebagai *payload canonical-document* dan diberi metadata *provenance* pada batas penyimpanan.

Tabel III.24: Metadata *provenance* pada *canonical document artifact*

Bagian	Field	Fungsi
<i>Provenance</i>	<code>profile_id</code> , <code>run_id</code>	Menunjukkan konteks <i>profile</i> dan <i>pipeline run</i> .
<i>Provenance</i>	<code>parser.name</code> , <code>parser.version</code> , <code>parser.output_revision</code>	Identitas versi <i>parser</i> yang menghasilkan keluaran.
<i>Parser cache</i>	<code>cache_key</code>	Kunci yang digunakan untuk mencari hasil <i>parser</i> yang dapat digunakan kembali.
<i>Parser cache</i>	<code>content_sha256</code>	<i>Hash</i> semantik dokumen kanonik.
<i>Parser cache</i>	<code>reused</code>	Penanda apakah keluaran diambil dari <i>cache</i> .
<i>Provenance</i>	<code>materialized_at</code>	Waktu <i>payload</i> kanonik disimpan sebagai <i>artifact</i> .

III.3.2.3 Skema Konfigurasi *Plugin*

Setiap *plugin* memiliki *manifest* dan *JSON Schema* konfigurasi. *Manifest* dicatat pada *plugin registry* untuk mengidentifikasi nama, versi, tipe komponen, dan implementasi *plugin*, sedangkan *schema* konfigurasi digunakan untuk membentuk konfigurasi dan memvalidasi nilai sebelum *plugin* dijalankan. *Schema* menggunakan `additionalProperties: false`, sehingga atribut di luar kontrak tidak diterima.

Tabel III.25: Field *manifest plugin*

Field	Tipe atau nilai	Kewajiban	Fungsi
<code>name</code>	String	Wajib	Nama <i>plugin</i> .
<code>version</code>	String	Wajib	Versi <i>plugin</i> .
<code>type</code>	<i>Component type</i>	Wajib	Tipe komponen yang diimplementasikan.
<code>entrypoint</code>	String	Wajib	Nama implementasi yang di- <i>resolve</i> .
<code>config_schema</code>	Nama berkas	Wajib	Lokasi <i>JSON Schema</i> konfigurasi.
<code>description</code>	String	Wajib	Penjelasan fungsi <i>plugin</i> .
<code>supported_extensions</code>	Array String	<i>Parser</i>	Ekstensi berkas yang dapat dibaca <i>plugin parser</i> .
<code>output_revision</code>	String	<i>Parser</i>	Revisi keluaran yang menjadi bagian dari identitas <i>parser cache</i> .

Tabel III.26: Skema konfigurasi BPK *scraper*

<i>Field</i>	Tipe	Nilai bawaan atau batasan	Fungsi
<code>base_url</code>	String	Bawaan URL portal BPK	Alamat dasar portal peraturan.
<code>keywords</code>	String	Bawaan kosong	Pencarian bebas pada portal.
<code>tentang</code>	String	Bawaan kosong	Filter berdasarkan subjek peraturan.
<code>nomor</code>	String	Bawaan kosong	Filter berdasarkan nomor peraturan.
<code>tahun</code>	Array Integer	Minimal 1945; <i>default</i> []	Tahun peraturan yang disertakan.
<code>entitas</code>	Array pilihan	<i>default</i> []	Identitas entitas pemerintahan yang dipilih.
<code>jenis</code>	Array pilihan	<i>default</i> []	Jenis peraturan yang dipilih.
<code>tag</code>	Array pilihan	<i>default</i> []	Tema atau tag peraturan yang dipilih.

Tidak ada *field* BPK yang diwajibkan oleh *schema*; nilai bawaan digunakan ketika filter tidak diisi.

Tabel III.27: Skema konfigurasi Google Drive *scraper*

<i>Field</i>	Tipe	Nilai bawaan atau batasan	Fungsi
<code>folder_id</code>	String	Wajib, minimal satu karakter	Identitas folder Google Drive yang diproses secara nonrekursif.
<code>service_account_json_env</code>	String	Wajib, minimal satu karakter	Nama <i>environment variable</i> yang menyimpan kredensial <i>service account</i> .
<code>filename_regex</code>	String	Opsional	Pola nama berkas yang harus cocok sebelum berkas diunduh.
<code>page_size</code>	Integer	1–1000; bawaan 1000	Jumlah berkas dalam satu permintaan <i>listing</i> ke API Google Drive.

Tabel III.28: Skema konfigurasi *legal parser*

<i>Field</i>	Tipe	Nilai bawaan atau batasan	Fungsi
<code>source.dpi</code>	Integer	<i>default</i> 150; lebih besar dari 0	Resolusi rasterisasi halaman pindai.

Bersambung ke halaman berikutnya

(lanjutan)

<i>Field</i>	<i>Tipe</i>	<i>Nilai bawaan atau batasan</i>	<i>Fungsi</i>
<code>source.scan_image_coverage_threshold</code>	Number	0–1; <i>default</i> 0.5	Ambang cakupan citra untuk klasifikasi halaman pindai.
<code>source.scan_min_text_threshold</code>	Integer	Minimal 0; <i>default</i> 100	Batas jumlah karakter <i>native text</i> sebelum halaman diperlakukan sebagai pindai.
<code>source.merge_y_tolerance</code>	Number	Minimal 0; <i>default</i> 6.0	Toleransi vertikal penggabungan fragmen teks menjadi baris.
<code>source.merge_x_tolerance</code>	Number	Minimal 0; <i>default</i> 0.5	Toleransi jarak horizontal penggabungan fragmen teks.
<code>layout.model</code>	Pilihan model	<code>none</code> atau <code>pp-doclayout-v3</code> ; <i>default</i> <code>none</code>	Model deteksi <i>region</i> sebelum OCR.
<code>layout.score_threshold</code>	Number	0–1; <i>default</i> 0.3	Skor minimum <i>region</i> yang diterima.
<code>layout.excluded_region_labels</code>	Array String	<i>default</i> []	Label <i>region</i> yang tidak dikirim ke OCR.
<code>layout.render_debug_overlays</code>	Boolean	<i>default</i> <code>false</code>	Penanda untuk menyimpan <i>overlay region</i> sebagai kebutuhan <i>debug</i> .
<code>ocr.model</code>	Pilihan model	<code>glm-ocr</code> atau <code>paddleocr-v1-1.6</code> ; <i>default</i> <code>glm-ocr</code>	Model pengenalan teks.
<code>ocr.base_prompt</code>	String	Bawaan <code>Text Recognition:</code>	Instruksi yang dikirim bersama citra.
<code>ocr.max_tokens</code>	Integer	Lebih besar dari 0; <i>default</i> 4096	Batas <i>token</i> keluaran satu permintaan OCR.
<code>ocr.max_concurrent_requests</code>	Integer	1–32; <i>default</i> 4	Jumlah permintaan OCR atau <i>layout</i> konkuren.
<code>ocr.temperature</code>	Number	0–2; <i>default</i> 0	Temperatur <i>sampling</i> OCR.
<code>ocr.repetition_penalty</code>	Number	1–2; <i>default</i> 1.1	Penalti pengulangan <i>token</i> .
<code>ocr.top_p</code>	Number	Lebih besar dari 0 sampai 1; <i>default</i> 0.00001	Batas <i>nucleus sampling</i> .
<code>ocr.top_k</code>	Integer	Minimal 0; <i>default</i> 1	Batas <i>top-k sampling</i> .

Schema legal parser mewajibkan kelompok `source`, `layout`, dan `ocr`, serta seluruh *field* yang ditentukan dalam kelompok tersebut.

Tabel III.29: Skema konfigurasi *generic outline parser*

<i>Field</i>	<i>Tipe</i>	<i>Nilai bawaan atau batas-an</i>	<i>Fungsi</i>
<code>outline.header_margin_ratio</code>	Number	0–0.5; <i>default</i> 0.075	Proporsi tinggi halaman yang diperlakukan sebagai <i>header</i> .
<code>outline.footer_margin_ratio</code>	Number	0.5–1; <i>default</i> 0.9	Proporsi halaman tempat <i>footer</i> dimulai.
<code>outline.anchor_search_window_points</code>	Number	Lebih besar dari 0; <i>default</i> 150.0	Jendela pencarian judul di sekitar tujuan <i>bookmark</i> .
<code>outline.line_merge_tolerance_points</code>	Number	Minimal 0; <i>default</i> 4.0	Toleransi vertikal penggabungan fragmen menjadi satu baris.
<code>outline.text_decoration</code>	Boolean	<i>default</i> <code>true</code>	Penanda untuk mempertahankan dekorasi teks.
<code>outline.excluded_section_titles</code>	Array String	Bawaan <code>contents</code> dan <code>table of contents</code>	Judul <i>bookmark</i> beserta turunannya dikecualikan.

Schema generic outline parser mewajibkan kelompok `outline` dan seluruh *field*-nya.

III.3.2.4 Kontrak API

Admin UI berkomunikasi dengan *Admin HTTP Server* sebagai *control plane* untuk mengelola *datasource*, *scraper*, dan *profile*, serta memantau pemrosesan dokumen. Subsubbab ini merangkum endpoint yang digunakan langsung pada alur akuisisi, konfigurasi *parser*, pemantauan *pipeline*, dan OCR/deteksi *layout*. Endpoint di luar cakupan subsubbab ini tidak dibahas.

Tabel III.30: Kontrak API utama untuk akuisisi dan parsing

<i>Metode</i>	<i>Path</i>	<i>Otorisasi</i>	<i>Fungsi</i>
<i>Datasource</i> dan akuisisi			
POST	<code>/api/v1/datasources</code>	Admin	Membuat <i>datasource</i> dan mengikat <i>scraper plugin</i> bila diberikan.
GET	<code>/api/v1/datasources</code>	Admin	Menampilkan daftar <i>datasource</i> dengan filter dan <i>pagination</i> .

Bersambung ke halaman berikutnya

(lanjutan)

Metode	Path	Otorisasi	Fungsi
GET	/api/v1/datasources/create-options	Admin	Menyediakan daftar <i>scraper plugin</i> dan <i>config schema</i> untuk formulir administrasi.
GET	/api/v1/datasources/<datasource_id>	Admin	Menampilkan detail <i>datasource</i> , profil terkait, dan <i>scrape run</i> terakhir.
PATCH	/api/v1/datasources/<datasource_id>	Admin	Memperbarui metadata, ekstensi berkas, atau binding <i>scraper</i> .
POST	/api/v1/datasources/<datasource_id>/archive	Admin	Mengarsipkan <i>datasource</i> ; operasi ditolak jika masih dipakai profil aktif.
POST	/api/v1/datasources/<datasource_id>/unarchive	Admin	Mengaktifkan kembali <i>datasource</i> yang diarsipkan.
GET	/api/v1/datasources/<datasource_id>/documents	Admin	Menampilkan daftar dokumen dengan filter status, asal, ekstensi, dan pencarian.
POST	/api/v1/datasources/<datasource_id>/documents	Admin	Menerima unggahan multipart, menyimpan dokumen, dan menjalankan <i>fan-out</i> .
GET	/api/v1/datasources/<datasource_id>/documents/<document_id>	Admin	Menampilkan detail dokumen dan statusnya pada profil yang berlangsung.
GET	/api/v1/datasources/<datasource_id>/documents/<document_id>/download-url	Admin	Menghasilkan <i>presigned URL</i> sementara untuk membaca berkas dari MinIO.
POST	/api/v1/datasources/<datasource_id>/documents/<document_id>/archive	Admin	Mengarsipkan dokumen pada <i>datasource</i> .
POST	/api/v1/datasources/<datasource_id>/documents/<document_id>/unarchive	Admin	Mengaktifkan kembali dokumen pada <i>datasource</i> .
GET	/api/v1/datasources/<datasource_id>/scrape-runs	Admin	Menampilkan daftar <i>scrape run</i> dengan filter.
POST	/api/v1/datasources/<datasource_id>/scrape-runs	Admin	Membuat dan mengantrikan <i>scrape run</i> manual.
GET	/api/v1/datasources/<datasource_id>/scrape-runs/<run_id>	Admin	Menampilkan status, penghitung, kesalahan, dan riwayat transisi <i>run</i> .
POST	/api/v1/datasources/<datasource_id>/scrape-runs/<run_id>/cancel	Admin	Meminta pembatalan <i>scrape run</i> secara kooperatif.
GET	/api/v1/datasources/<datasource_id>/scrape-runs/<run_id>/entries	Admin	Menampilkan <i>scrape entry</i> beserta statusnya.
GET	/api/v1/datasources/<datasource_id>/scrape-runs/<run_id>/entries/<entry_id>	Admin	Menampilkan detail satu <i>scrape entry</i> dan riwayat transisinya.
<i>Profile dan konfigurasi pipeline</i>			
GET	/api/v1/profiles	Admin	Menampilkan daftar ringkas <i>profile</i> dengan pencarian dan <i>pagination</i> .

Bersambung ke halaman berikutnya

(lanjutan)

Metode	Path	Otorisasi	Fungsi
POST	/api/v1/profiles	Admin	Membuat <i>profile</i> dan konfigurasi <i>pipeline node</i> .
GET	/api/v1/profiles/create-options	Admin	Menyediakan pilihan <i>plugin</i> , <i>config schema</i> , dan kompatibilitas untuk <i>profile wizard</i> .
GET	/api/v1/profiles/<profile_id>	Admin	Menampilkan konfigurasi <i>datasource</i> dan <i>pipeline node</i> .
PATCH	/api/v1/profiles/<profile_id> /ingestion-config	Admin	Memperbarui mode ingesti dan mode komponen <i>profile</i> .
POST	/api/v1/profiles/<profile_id>/sync	Admin	Mengirim dokumen <i>datasource</i> yang sudah tersimpan ke <i>profile</i> .
Pemantauan dan pengendalian pemrosesan			
POST	/api/v1/profiles/<profile_id>/runs/ cancel	Admin	Membatalkan seluruh <i>pipeline run</i> yang masih menunggu atau berjalan.
GET	/api/v1/profiles/<profile_id> /documents	Admin	Menampilkan dokumen pada <i>profile</i> beserta status <i>pipeline</i> .
GET	/api/v1/profiles/<profile_id> /documents/<document_id>	Admin	Menampilkan status pemrosesan, <i>pipeline run</i> , dan tautan artefak.
GET	/api/v1/profiles/<profile_id> /documents/<document_id>/runs	Admin	Menampilkan riwayat <i>pipeline run</i> untuk satu dokumen.
GET	/api/v1/profiles/<profile_id> /documents/<document_id>/runs/<run_ id>/events	Admin	Mengalirkan perubahan status <i>pipeline run</i> melalui SSE.
POST	/api/v1/profiles/<profile_id> /documents/<document_id>/runs/<run_ id>/retry	Admin	Mengantri ulang <i>pipeline run</i> , dengan pilihan tahap awal.
POST	/api/v1/profiles/<profile_id> /documents/retry	Admin	Mengantri ulang dokumen <i>profile</i> yang dipilih atau sesuai filter.
<i>Plugin</i>			
GET	/api/v1/plugins	Admin	Menampilkan daftar <i>plugin</i> yang tersedia.
GET	/api/v1/plugins/<name>/<version>	Admin	Menampilkan manifest, <i>config schema</i> , dan berkas <i>plugin</i> .
GET	/api/v1/plugins/<name>/<version> /files?path=<relative_path>	Admin	Menampilkan isi satu berkas teks <i>plugin</i> yang dipilih.

Untuk halaman pindai, *parser worker* menghubungi *oMLX OCR Model HTTP Server* secara langsung. Ketersediaan *oMLX* diperiksa melalui endpoint `/v1/models`, sedangkan pengenalan teks dilakukan melalui `/v1/chat/completions`. Kedua endpoint tersebut dirangkum pada tabel berikut.

Tabel III.31: Kontrak API oMLX yang digunakan oleh *parser*

Metode	Path	Otorisasi	Fungsi
GET	/v1/models	Bearer <i>OCR API key</i>	Memeriksa ketersediaan API dan model OCR sebelum parsing.
POST	/v1/chat/completions	Bearer <i>OCR API key</i>	Mengirim citra halaman, instruksi, dan parameter <i>sampling</i> untuk memperoleh teks OCR.

Layout Model HTTP Server menyediakan antarmuka HTTP pada port bawaan 8083 untuk menjalankan PP-DocLayoutV3. Seluruh endpoint layanan ini dilindungi *Bearer service API key* yang dikirim pada header Authorization. Endpoint yang digunakan oleh *parser* dirangkum pada tabel berikut.

Tabel III.32: Kontrak *Layout Model HTTP Server*

Metode	Path	Otorisasi	Fungsi
GET	/health	Bearer <i>service API key</i>	Memeriksa kesiapan layanan dan model deteksi <i>layout</i> .
GET	/v1/layout/models	Bearer <i>service API key</i>	Mengembalikan alias model, model sumber, dan perangkat eksekusinya.
POST	/v1/layout/detections	Bearer <i>service API key</i>	Menerima citra dan ambang skor, lalu mengembalikan <i>region</i> yang telah dinormalisasi beserta urutan baca.

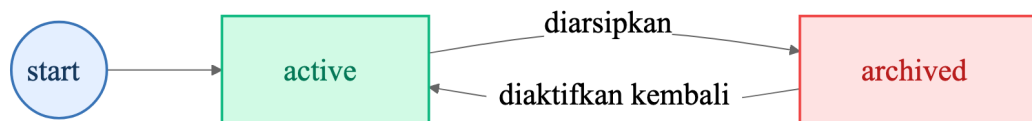
III.3.3 Rancangan *Lifecycle* dan *State*

Lifecycle pada alur akuisisi dan ekstraksi dirancang sebagai *state* yang dapat diamati oleh API dan *Admin UI*. Subbab ini memisahkan *state datasource*, *scrape run*, *scrape entry*, dan *pipeline run* agar setiap transisi serta kondisi terminal dapat dibaca secara terpisah.

III.3.3.1 *Lifecycle Datasource*

Datasource dibuat dalam *state active* dan dapat diarsipkan menjadi *archived*. Pengarsipan merupakan *soft archive*: data dan riwayat proses tetap disimpan, tetapi

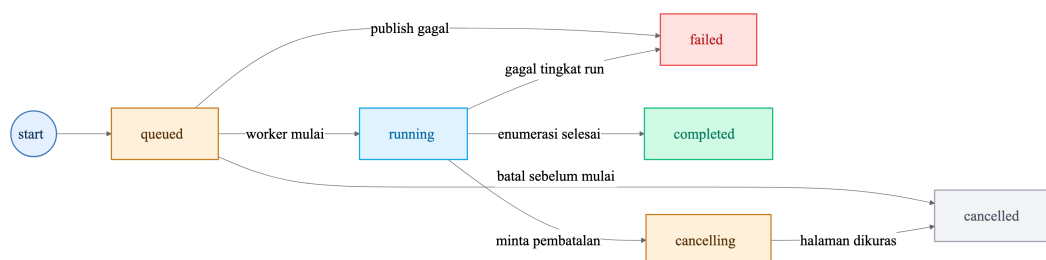
datasource tidak dapat menerima perubahan atau proses akuisisi baru. Pengarsipan hanya dapat dilakukan ketika tidak ada *profile* aktif yang berlangganan, tidak ada *profile* nonaktif yang masih memiliki *pipeline run* terkait, dan tidak ada *scrape run* aktif. *Datasource* yang telah diarsipkan dapat dikembalikan ke *active* melalui operasi *unarchive*. Karena alasan historis, implementasi saat ini menggunakan *soft archive* dan belum menyediakan operasi *hard delete* melalui API *datasource*.



Gambar III.12: *Lifecycle datasource*

III.3.3.2 *Lifecycle Scrape Run*

Scrape run dibuat dalam *state queued* oleh *manual trigger* maupun *scheduled trigger*. *Scraper worker* mengubahnya menjadi *running* ketika berhasil mengambil pekerjaan. Kegagalan saat *publish* ketika masih *queued* atau *run-level failure* ketika sedang berjalan mengarah ke *failed*. *Cancellation request* saat *queued* langsung menghasilkan *cancelled*, sedangkan saat *running* melewati *cancelling* agar halaman yang sedang diproses dapat diselesaikan terlebih dahulu. Setelah enumerasi dan pemrosesan selesai, *run* berakhir sebagai *completed*; setelah halaman yang sedang diproses dikuras, *run* yang dibatalkan berakhir sebagai *cancelled*.

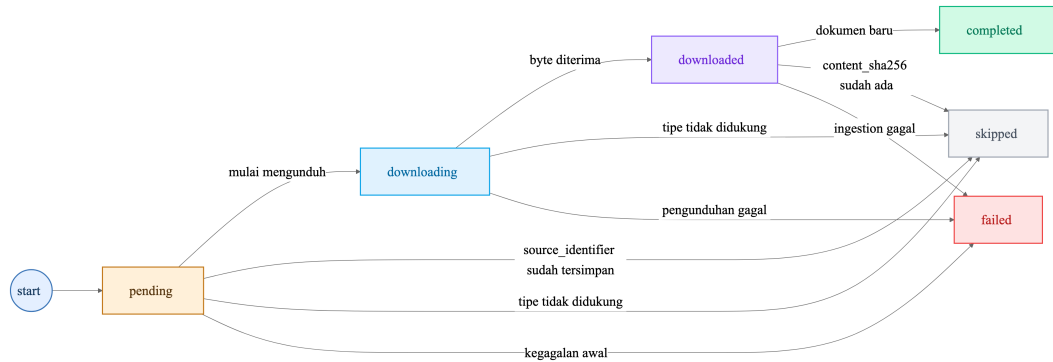


Gambar III.13: *Lifecycle scrape run*

III.3.3.3 *Lifecycle Scrape Entry*

Setiap *entry* yang ditemukan oleh *scraper plugin* dimulai dalam *state pending*. Entri dengan *source_idenfifier* yang sudah tersimpan dapat langsung berakhir sebagai *skipped* tanpa *download*. Entri dengan tipe berkas yang tidak didukung juga

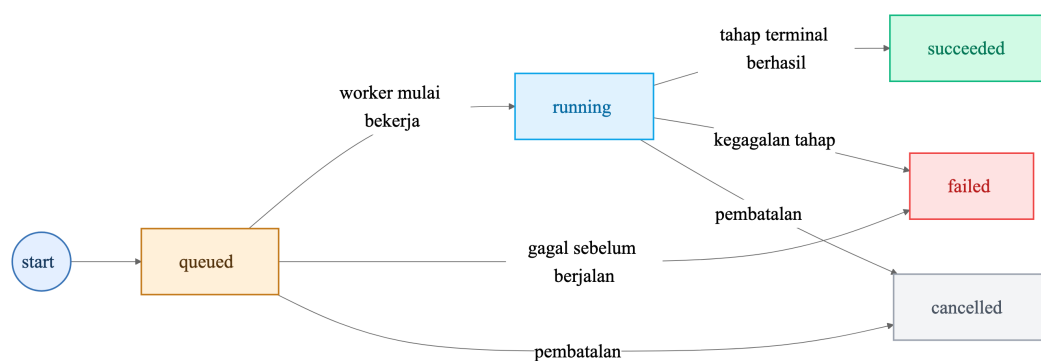
berakhir sebagai *skipped* tanpa disimpan. Entri yang akan di-*download* berpindah ke *downloading*, lalu ke *downloaded* setelah *byte* diterima. Hasil *ingestion* yang menyimpan dokumen baru berakhir sebagai *completed*, sedangkan dokumen yang memiliki *content_sha256* yang sudah tersimpan berakhir sebagai *skipped*. Kegagalan *download* atau *ingestion* berakhir sebagai *failed*.



Gambar III.14: *Lifecycle scrape entry*

III.3.3.4 Lifecycle Pipeline Run

Pipeline run merupakan status agregat pemrosesan dokumen pada *pipeline*, termasuk tahap *parser*. Permintaan yang diterima dibuat dalam *state queued* dan berpindah ke *running* ketika *worker* mulai menjalankannya. *Run* berakhir sebagai *succeeded* setelah tahap terminal berhasil, sebagai *failed* ketika terjadi kegagalan tahap, atau sebagai *cancelled* ketika pekerjaan dibatalkan.



Gambar III.15: *Lifecycle pipeline run*

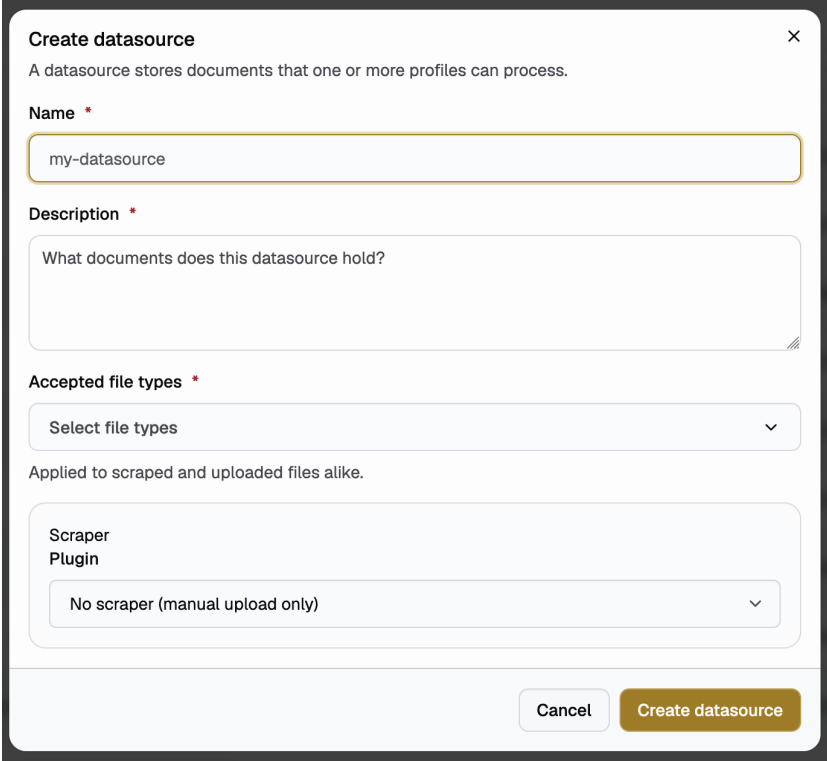
III.3.4 Rancangan Alur Interaksi

Subbab ini menjelaskan urutan interaksi antara *Admin UI*, *Admin HTTP Server*, *Scraper Scheduler*, *Scraper Worker*, *Parser Worker*, dan komponen infrastruktur.

Detail *endpoint* yang digunakan dirangkum pada Tabel III.30, sedangkan perubahan *state* yang menjadi hasil interaksi dirangkum pada Subsubbab III.3.3.

III.3.4.1 Pembuatan *Datasource*

Pembuatan *datasource* dimulai ketika administrator membuka formulir pada *Admin UI*. *Admin UI* meminta pilihan *scraper plugin* dan *config schema* melalui `/api/v1/datasources/create-options` agar formulir dapat dibentuk dari kontrak *plugin* yang tersedia. Setelah administrator mengisi *metadata*, ekstensi berkas, dan konfigurasi *scraper* opsional, *Admin UI* mengirimkan POST ke `/api/v1/datasources`. *Admin HTTP Server* memvalidasi konfigurasi dan menyimpan *datasource* baru dengan status *active* pada PostgreSQL. *Datasource* tanpa *scraper binding* tetap dapat dibuat sebagai *upload-only datasource*. Contoh formulir pembuatan *datasource upload-only* pada *Admin UI* ditunjukkan pada Gambar III.16.



Create datasource ×

A datasource stores documents that one or more profiles can process.

Name *

my-datasource

Description *

What documents does this datasource hold?

Accepted file types *

Select file types

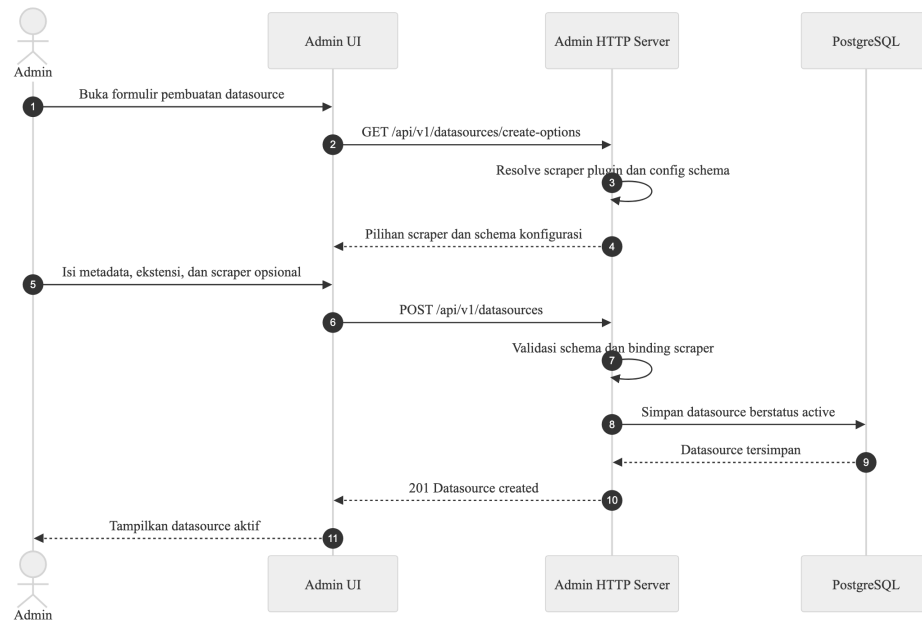
Applied to scraped and uploaded files alike.

Scraper Plugin

No scraper (manual upload only)

Cancel Create datasource

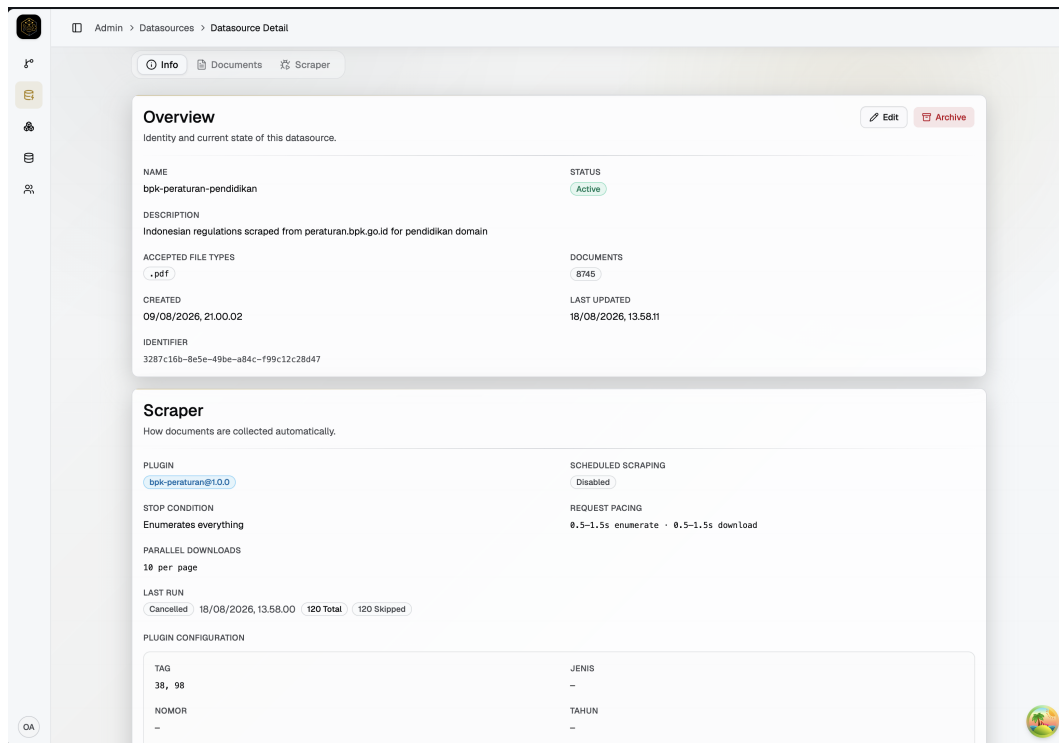
Gambar III.16: Formulir pembuatan *datasource upload-only* pada *Admin UI*



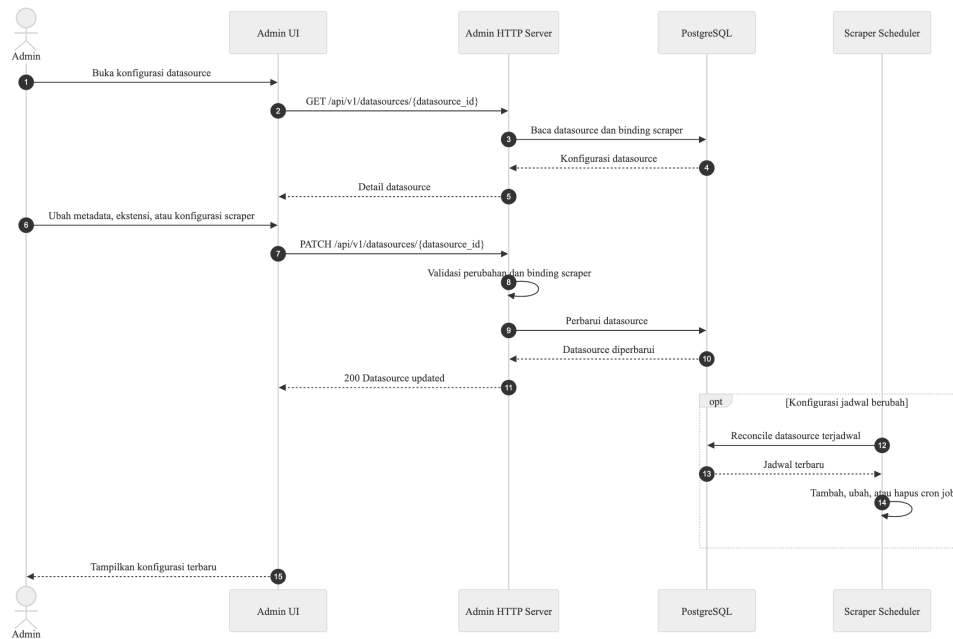
Gambar III.17: Alur interaksi pembuatan *datasource*

III.3.4.2 Perubahan Konfigurasi *Datasource*

Perubahan konfigurasi dimulai dengan pembacaan detail *datasource* melalui *endpoint* detail *datasource*. Administrator dapat memperbarui konfigurasi *datasource* melalui PATCH pada *endpoint* yang sama. *Admin HTTP Server* memvalidasi perubahan sebelum memperbarui baris *datasource* pada PostgreSQL. Dokumen yang sudah tersimpan tidak ikut berubah karena perubahan ini hanya memengaruhi konfigurasi penerimaan dan akuisisi berikutnya. Jika perubahan mencakup jadwal *scraper*, *Scraper Scheduler* membaca perubahan tersebut pada proses *reconcile* berikutnya untuk menambah, mengganti, atau menghapus *cron job*. Tampilan detail *datasource* aktif beserta kontrol perubahan dan pengarsipannya ditunjukkan pada Gambar III.18.



Gambar III.18: Detail *datasource* aktif pada Admin UI

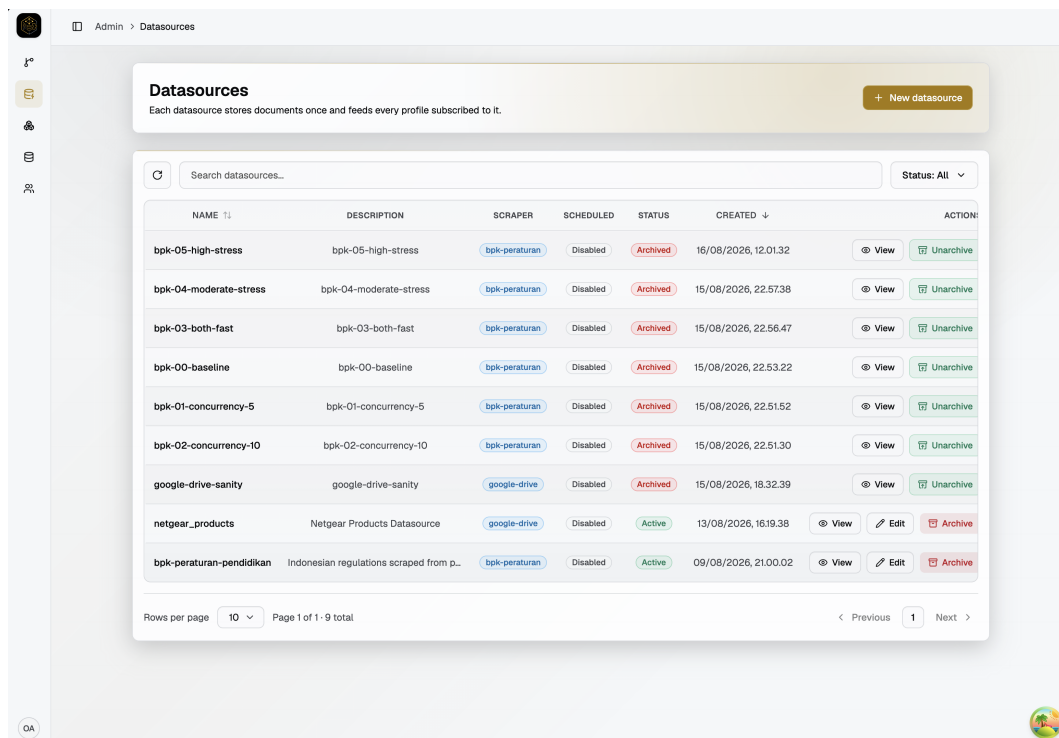


Gambar III.19: Alur interaksi perubahan konfigurasi *datasource*

III.3.4.3 Pengarsipan *Datasource*

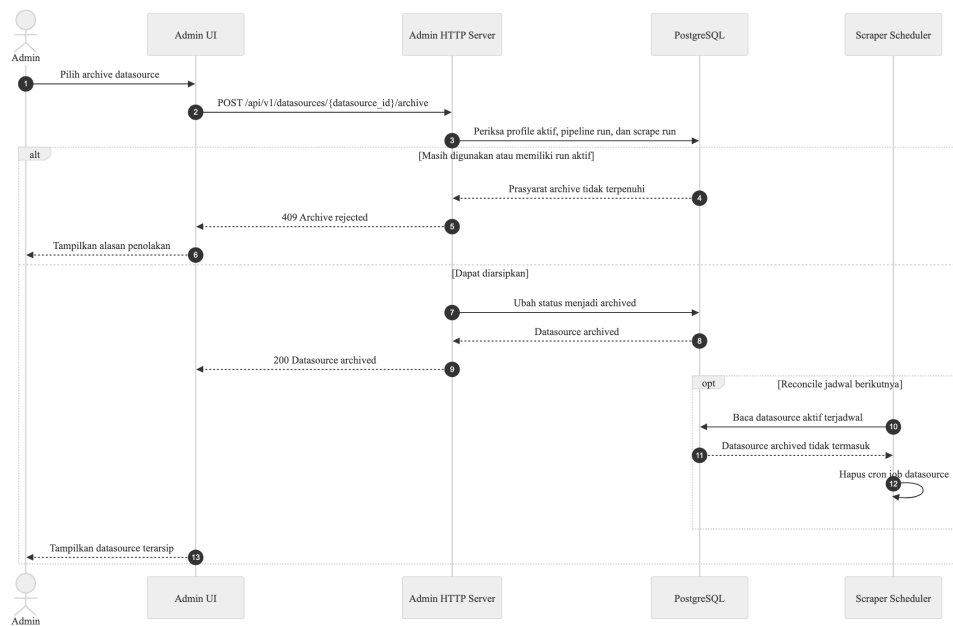
Pengarsipan dipicu administrator melalui POST pada *endpoint archive datasource*. *Admin HTTP Server* memeriksa apakah masih ada *profile* aktif, pekerjaan pem-

bersihkan *profile* yang belum selesai, atau *scrape run* aktif yang bergantung pada *datasource* tersebut. Jika salah satu prasyarat masih terpenuhi, operasi ditolak dan tidak ada perubahan data. Jika tidak ada dependensi aktif, status *datasource* diubah menjadi *archived* sebagai *soft archive*, sehingga dokumen dan riwayat proses tetap tersimpan. Pada proses *reconcile* berikutnya, *Scraper Scheduler* menghapus jadwal yang terkait dengan *datasource* terarsip. Daftar *datasource* pada *Admin UI* memperlihatkan perbedaan status *active* dan *archived* beserta aksi yang tersedia pada masing-masing status, seperti ditunjukkan pada Gambar III.20.



NAME	DESCRIPTION	SCRAPER	SCHEDULED	STATUS	CREATED	ACTION
bpk-05-high-stress	bpk-05-high-stress	bpk-peraturan	Disabled	Archived	16/08/2026, 12.01.32	View Unarchive
bpk-04-moderate-stress	bpk-04-moderate-stress	bpk-peraturan	Disabled	Archived	15/08/2026, 22.57.38	View Unarchive
bpk-03-both-fast	bpk-03-both-fast	bpk-peraturan	Disabled	Archived	15/08/2026, 22.56.47	View Unarchive
bpk-00-baseline	bpk-00-baseline	bpk-peraturan	Disabled	Archived	15/08/2026, 22.53.22	View Unarchive
bpk-01-concurrency-5	bpk-01-concurrency-5	bpk-peraturan	Disabled	Archived	15/08/2026, 22.51.52	View Unarchive
bpk-02-concurrency-10	bpk-02-concurrency-10	bpk-peraturan	Disabled	Archived	15/08/2026, 22.51.30	View Unarchive
google-drive-sanity	google-drive-sanity	google-drive	Disabled	Archived	15/08/2026, 18.32.39	View Unarchive
netgear_products	Netgear Products Datasource	google-drive	Disabled	Active	13/08/2026, 16.19.38	View Edit Archive
bpk-peraturan-pendidikan	Indonesian regulations scraped from p...	bpk-peraturan	Disabled	Active	09/08/2026, 21.00.02	View Edit Archive

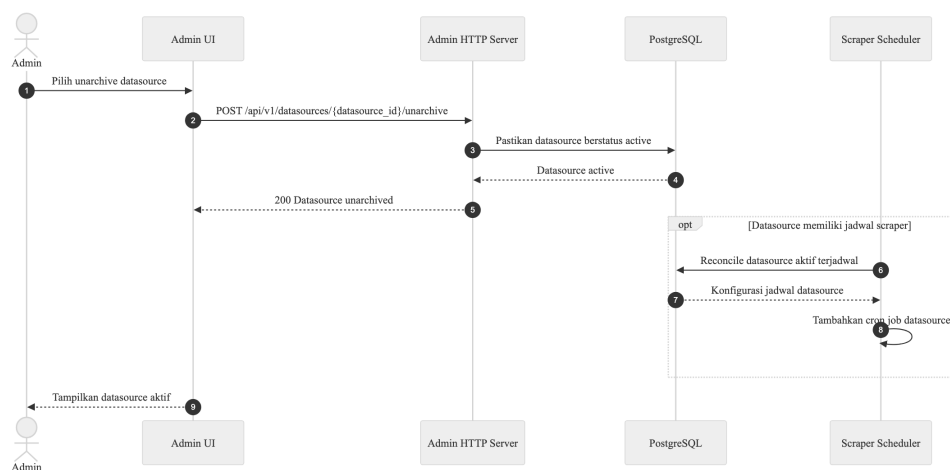
Gambar III.20: Daftar *datasource* dengan status *active* dan *archived* pada *Admin UI*



Gambar III.21: Alur interaksi pengarsipan *datasource*

III.3.4.4 Pengaktifan Kembali *Datasource*

Pengaktifan kembali dilakukan melalui POST pada *endpoint unarchive datasource*. *Admin HTTP Server* memastikan bahwa baris *datasource* berada pada status yang dapat diaktifkan, lalu menyimpannya kembali dengan status *active* pada PostgreSQL. Operasi ini tidak membuat dokumen baru dan tidak mengulang riwayat akuisisi secara otomatis. Jika konfigurasi *scraper* memiliki jadwal aktif, *Scraper Scheduler* menambahkan kembali *cron job* pada proses *reconcile* berikutnya.



Gambar III.22: Alur interaksi pengaktifan kembali *datasource*

III.3.4.5 Pembuatan *Profile* dan Pemilihan *Parser*

Pembuatan *profile* dimulai dari *profile wizard* pada *Admin UI*. *Admin UI* mengambil pilihan *plugin*, *config schema*, dan informasi kompatibilitas melalui `/api/v1/profiles/create-options`. Administrator kemudian memilih *datasource*, satu *parser node*, dan *pipeline node* lain yang diperlukan sebelum mengirimkan POST ke `/api/v1/profiles`. *Admin HTTP Server* memvalidasi keberadaan tepat satu *parser*, kompatibilitas antarnode, serta konfigurasi setiap *node* sebelum menyimpan *profile* dan *pipeline nodes* pada PostgreSQL. Pembuatan *profile* tidak langsung memproses dokumen lama; pemrosesan dokumen yang sudah ada dilakukan melalui *flow* sinkronisasi pada Subsubsubbab III.3.4.12. *Profile wizard* menampilkan pemilihan *datasource* dan *parser* secara bertahap, seperti ditunjukkan pada Gambar III.23.

(a) Pemilihan *datasource*

(b) Pemilihan *parser*

Gambar III.23: Pemilihan *datasource* dan *parser* pada *profile wizard*

Konfigurasi *generic outline parser*, termasuk batas pencarian judul, penggabungan

baris, dekorasi teks, dan judul bagian yang dikecualikan, ditampilkan pada Gambar III.24.

generic-outline@1.0.0 configuration

Adjust advanced settings here. Declared schema defaults are loaded automatically.

Outline *

Header margin ratio *

0.075

Fraction of page height treated as the running header. Lines ending above it are dropped.

Footer margin ratio *

0.9

Fraction of page height where the running footer begins. Lines starting below it are dropped.

Anchor search window points *

150

How far below a bookmark destination, in points, its heading may be found.

Line merge tolerance points *

4

Vertical distance, in points, within which separately drawn runs are treated as one line. Rejoins bullet glyphs with the text they introduce.

Text decoration *

Record bold, italic and monospaced runs as markdown in the block text. Turn off for the plainest possible text.

Excluded section titles *

Type a value, then press Enter

contents x table of contents x

Bookmark titles dropped along with everything nested under them, compared case-insensitively.

Cancel Apply configuration

Gambar III.24: Konfigurasi *generic outline parser* pada *profile wizard*

Konfigurasi khusus *legal parser*, termasuk deteksi halaman pindai, *layout*, dan OCR, dapat diatur melalui dialog konfigurasi *plugin* sebagaimana ditunjukkan pada Gambar III.25.

legal-parser@1.0.0 configuration ✕

Adjust advanced settings here. Declared schema defaults are loaded automatically.

Source *

Dpi *

Resolution used to render scanned pages before OCR.

Scan image coverage threshold *

Image coverage ratio above which a page is treated as scanned rather than vector.

Scan min text threshold *

Pages carrying fewer native-text characters than this may be treated as scanned.

Merge y tolerance *

Vertical tolerance for grouping native text spans into a single line. Above roughly 13 the lines of a tight paragraph merge into one.

Merge x tolerance *

Horizontal gap below which adjacent native text spans join without a space. Above roughly 3 real word spaces are swallowed.

Layout *

Model *

Layout model used to split a scanned page into regions.

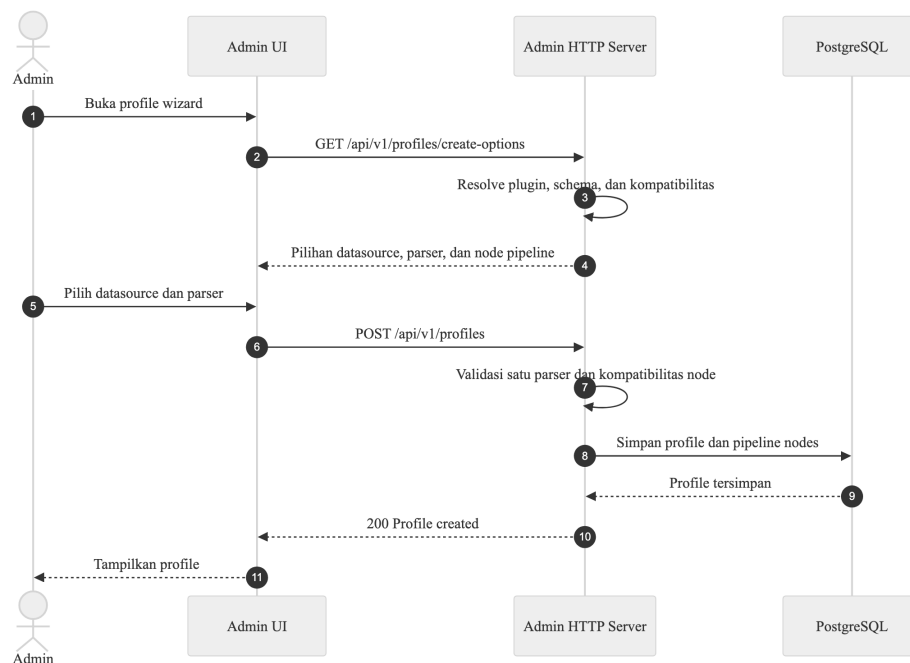
Ocr *

Model *

Vision-language model used for text recognition.

Cancel ✓ Apply configuration 

Gambar III.25: Konfigurasi *legal parser* pada *profile wizard*



Gambar III.26: Alur interaksi pembuatan *profile* dan pemilihan *parser*

III.3.4.6 Unggah Dokumen Manual

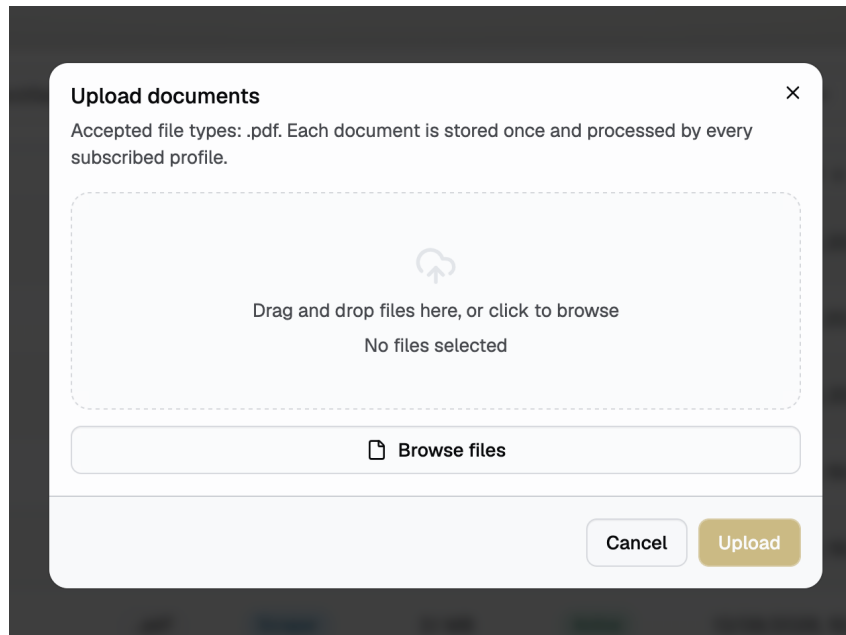
Unggah manual dimulai ketika administrator mengirim satu atau lebih berkas melalui *Admin UI*. *Admin UI* mengirimkan *multipart request* ke *endpoint* unggah dokumen *datasource*, dan *Admin HTTP Server* memproses setiap berkas secara terpisah. Untuk setiap berkas, sistem memeriksa ekstensi, menghitung *SHA-256*, dan mencari dokumen yang telah disimpan berdasarkan *source identifier* atau *content hash*. Berkas baru disimpan pada MinIO dan *metadata* dokumennya ditulis pada PostgreSQL, sedangkan berkas duplikat menggunakan dokumen yang sudah ada. Setelah dokumen tersedia, *profile fan-out* dibuat secara internal oleh *Admin HTTP Server* untuk setiap *profile* aktif yang sesuai. *Fan-out* membuat *pipeline run* dan *parser node* pada PostgreSQL, lalu menerbitkan `rag.parse.requested` ke RabbitMQ. Respons unggah mengembalikan hasil setiap berkas, termasuk dokumen baru, duplikat, berkas yang ditolak, dan kegagalan individual. Tab *Documents* menampilkan dokumen yang tersimpan, asal dokumen, status pemrosesan, dan tombol untuk memulai unggah manual, seperti ditunjukkan pada Gambar III.27.

FILENAME	TYPE	ORIGIN	SIZE	STATUS	ADDED	ACTIONS
PERATURAN_MENAG_8_2026.pdf	.pdf	Scraper	2.1 MB	Active	16/08/2026, 20.02:11	Detail Archive
PERWALI_17_2024.pdf	.pdf	Scraper	2.1 MB	Active	13/08/2026, 20.47:36	Detail Archive
PERWALI_11_2025.pdf	.pdf	Scraper	530.0 KB	Active	13/08/2026, 20.47:17	Detail Archive
PERBUP_5_2026.pdf	.pdf	Scraper	1006.8 KB	Active	13/08/2026, 19.09:46	Detail Archive
PERBUP_9_2026.pdf	.pdf	Scraper	690.4 KB	Active	13/08/2026, 19.09:41	Detail Archive
PERBUP_8_2026.pdf	.pdf	Scraper	3.1 MB	Active	13/08/2026, 19.09:41	Detail Archive
UU_11_2020.pdf	.pdf	Scraper	14.3 MB	Active	13/08/2026, 18.59:10	Detail Archive
UU_11_2019.pdf	.pdf	Scraper	3.2 MB	Active	13/08/2026, 18.59:09	Detail Archive
UU_23_2022.pdf	.pdf	Scraper	1.6 MB	Active	13/08/2026, 18.59:08	Detail Archive
UU_18_2019.pdf	.pdf	Scraper	1.9 MB	Active	13/08/2026, 18.59:08	Detail Archive

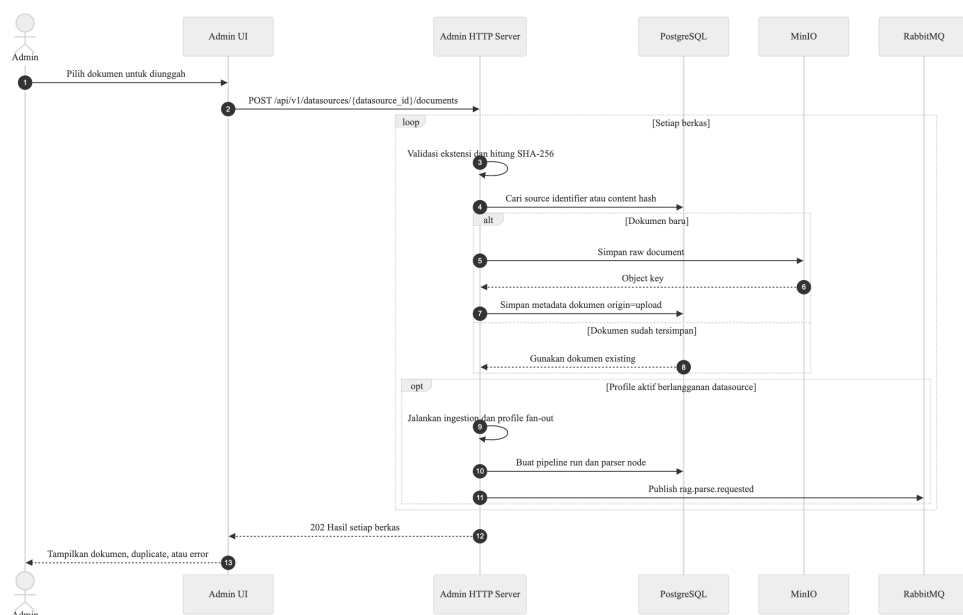
Gambar III.27: Tab *Documents* pada detail *datasource*

Dialog unggah pada *Admin UI* menyatakan tipe berkas yang diterima serta bahwa

setiap dokumen diproses oleh seluruh *profile* yang berlangganan, seperti ditunjukkan pada Gambar III.28.



Gambar III.28: Dialog unggah dokumen pada *Admin UI*



Gambar III.29: Alur interaksi unggah dokumen manual sampai *fan-out*

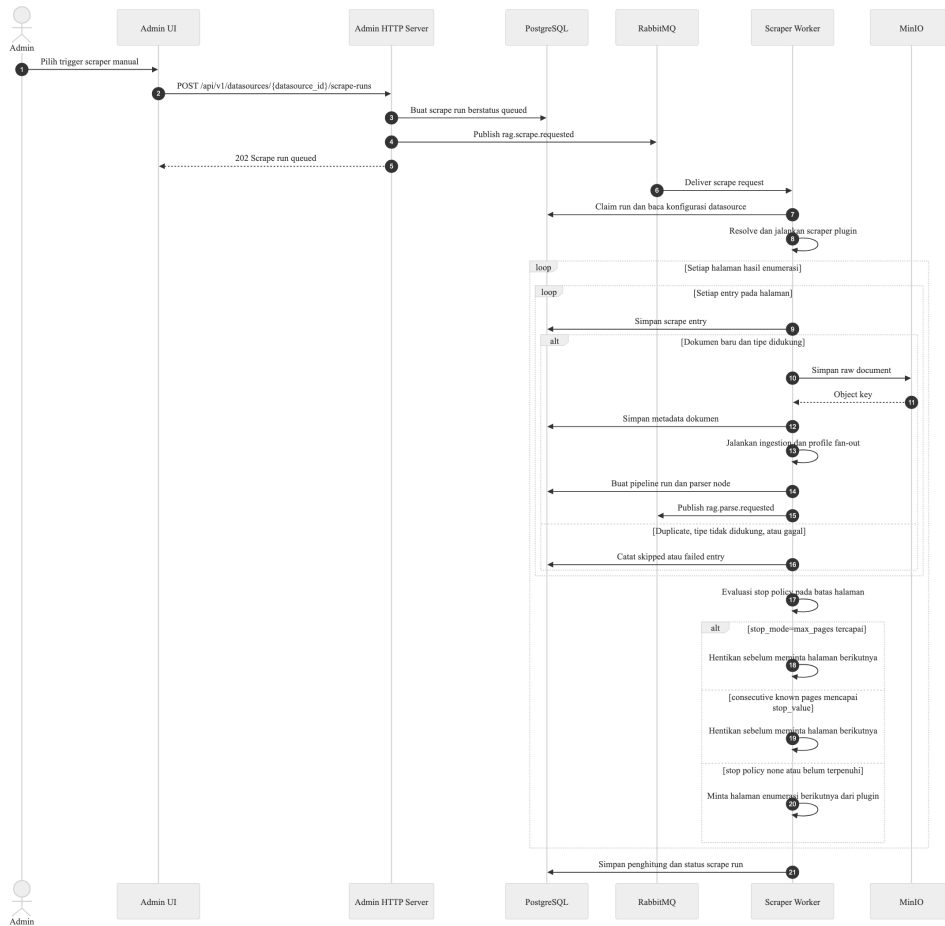
III.3.4.7 Pemicu *Scraper* Manual

Pemicu manual dimulai dari aksi administrator pada *Admin UI* dan dikirim melalui POST pada *trigger endpoint scrape run*. *Admin HTTP Server* membuat *scrape run*

berstatus *queued*, menerbitkan `rag.scrape.requested` ke RabbitMQ, dan segera mengembalikan identitas *run* kepada antarmuka. RabbitMQ mengantarkan pesan tersebut kepada *Scraper Worker*, yang mengambil konfigurasi terbaru dari PostgreSQL sebelum menjalankan *scrape session*. *Scrape session* memproses hasil enumerasi per halaman, menjalankan validasi dan deduplikasi untuk setiap *entry*, lalu menyimpan dokumen baru pada MinIO dan PostgreSQL. Pada batas setiap halaman, *scrape session* mengevaluasi *stop policy*: *none* melanjutkan hingga *plugin* selesai, *max pages* berhenti setelah batas halaman, dan *known pages* berhenti setelah jumlah halaman berturut-turut yang seluruh *entry*-nya telah dikenal tercapai. Dokumen baru diteruskan melalui *profile fan-out*, yang membuat *parser node* dan menerbitkan `rag.parse.requested` ke RabbitMQ. Tab *Scraper* menampilkan tombol pemicu manual dan riwayat *scrape run*, seperti ditunjukkan pada Gambar III.30.

STATUS	TRIGGERED BY	SUMMARY	CREATED	STARTED	FINISHED	ACTIONS
Cancelled	Schedule	120 Total 120 Skipped	18/08/2026, 13.58.00	18/08/2026, 13.58.00	18/08/2026, 13.58.20	Detail
Cancelled	Schedule	540 Total 540 Skipped	18/08/2026, 13.56.00	18/08/2026, 13.56.00	18/08/2026, 13.57.34	Detail
Cancelled	Manual	240 Total 240 Skipped	18/08/2026, 13.52.36	18/08/2026, 13.52.36	18/08/2026, 13.53.06	Detail
Cancelled	Manual	90 Total 90 Skipped	18/08/2026, 13.52.19	18/08/2026, 13.52.19	18/08/2026, 13.52.31	Detail
Cancelled	Manual	340 Total 340 Skipped	18/08/2026, 13.49.47	18/08/2026, 13.49.47	18/08/2026, 13.50.35	Detail
Cancelled	Manual	360 Total 360 Skipped	18/08/2026, 13.27.17	18/08/2026, 13.27.17	18/08/2026, 13.28.10	Detail
Cancelled	Manual	100 Total 100 Skipped	18/08/2026, 13.24.08	18/08/2026, 13.24.08	18/08/2026, 13.24.24	Detail
Cancelled	Manual	60 Total 60 Skipped	18/08/2026, 13.22.02	18/08/2026, 13.22.02	18/08/2026, 13.22.12	Detail
Completed	Manual	8822 Total 1 New 8806 Skipped 15 Failed	16/08/2026, 19.37.38	16/08/2026, 19.37.39	16/08/2026, 20.04.23	Detail
Failed	Manual	0 Total	13/08/2026, 22.19.08	13/08/2026, 22.19.08	13/08/2026, 22.19.09	Detail

Gambar III.30: Tab *Scraper* dengan pemicu manual dan riwayat *scrape run*



Gambar III.31: Alur interaksi pemicu *scraper* manual sampai *fan-out*

III.3.4.8 Pemicu *Scraper* Terjadwal

Pemicu terjadwal dikelola oleh *Scraper Scheduler*, bukan oleh *request* langsung dari *Admin UI*. *Scraper Scheduler* melakukan *reconcile* terhadap konfigurasi jadwal pada PostgreSQL, memasang *cron job*, lalu pada waktu yang ditentukan memanggil jalur *trigger* yang sama dengan pemicu manual. Perbedaannya hanya terdapat pada sumber pemicu dan nilai `triggered_by=schedule` ketika *scrape run* dibuat. Setelah pesan `rag.scrape.requested` diterbitkan, *Scraper Worker* menjalankan enumerasi, pemrosesan *entry*, evaluasi *stop policy*, penyimpanan dokumen, dan *fan-out* dengan mekanisme yang sama seperti pada Subsubsubbab III.3.4.7. Konfigurasi jadwal dan *stop condition* pada *scraper* ditampilkan pada Gambar III.32.

Scraper

Plugin

bpk-peraturan@1.0.0

Scheduled scraping

Runs automatically on the schedule below.

Schedule (cron) *

0 0 * * *

minute hour day month weekday

Timezone *

Asia/Jakarta

Crawl policy

How far each run goes, and how fast it requests.

Stop condition

Enumerate everything

Enumerate delay min (s) *

1

Enumerate delay max (s) *

3

Download delay min (s) *

1

Download delay max (s) *

3

Parallel downloads *

5

Each request pauses a random interval between the min and max. Parallel downloads fetch that many entries of a page at once, raising the rate reaching the source; 5 is the recommended default.

Plugin configuration

5 settings ready

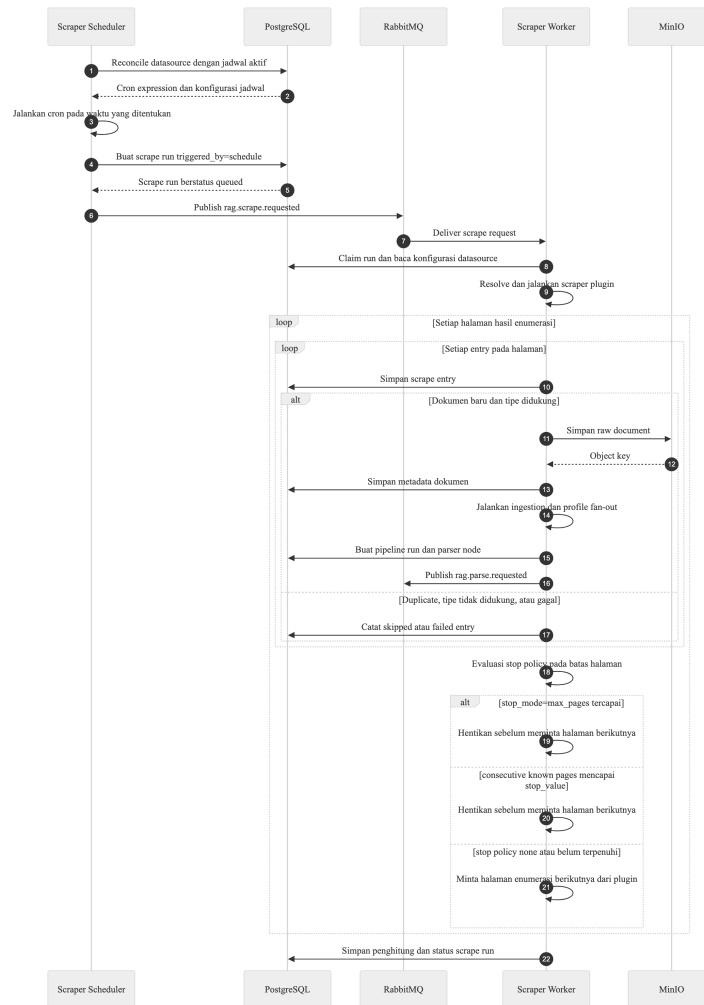
Configure

5

Cancel

Create datasource

Gambar III.32: Konfigurasi jadwal dan *stop condition scraper* pada *Admin UI*

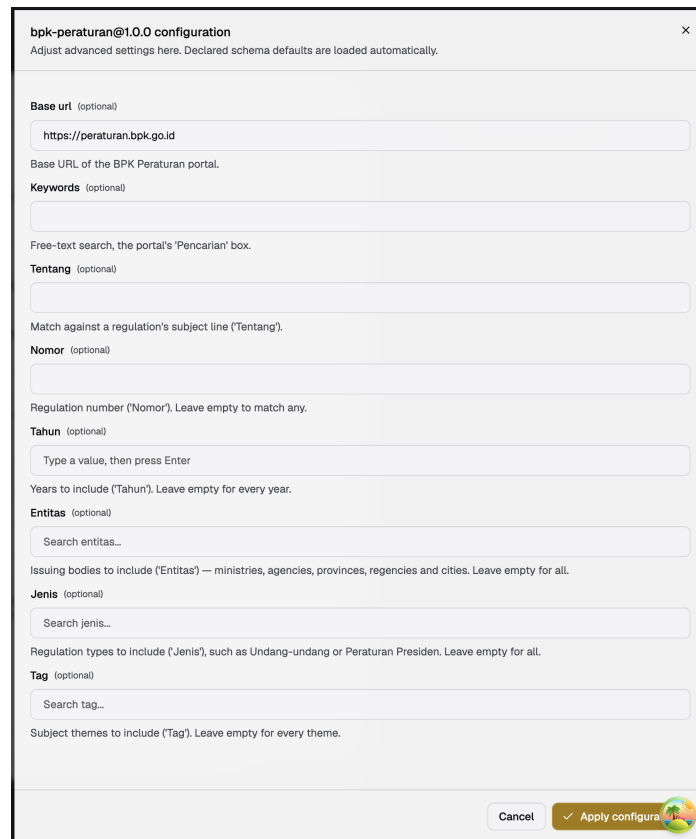


Gambar III.33: Alur interaksi pemicu *scraper* terjadwal sampai *fan-out*

III.3.4.9 Eksekusi *BPK Peraturan Scraper*

BPK Peraturan Scraper mengimplementasikan kontrak *scraper* untuk portal peraturan BPK yang dianalisis pada Subsubsubbab III.1.2.1. Pada tahap enumerasi, *plugin* membangun URL `/Search` dari *filter datasource* dan nomor halaman `p`, mengambil *HTML server-side rendered*, lalu mengekstrak tautan detail berbentuk `/Details/<id>/<slug>` sebagai *source identifier*. Pengecekan apakah *source identifier* sudah tersimpan dilakukan oleh *scrape session* sebelum *plugin* melakukan pengunduhan. Untuk *entry* baru, *plugin* membaca halaman detail, mem-parsing tujuh belas *metadata fields* dan kartu relasi status, menemukan tautan `/Download`, mengunduh *byte*, serta menurunkan nama berkas dan ekstensi yang akan diperiksa oleh *datasource*. *Plugin* hanya mengembalikan *ScrapedPage* dan *DownloadedDocument*; validasi ekstensi, *hashing*, deduplikasi, penyimpanan, dan *fan-out* tetap

dilakukan oleh komponen inti. *stop policy* dievaluasi oleh *scrape session* pada batas halaman, sehingga *plugin* tidak mengambil keputusan penghentian berdasarkan jumlah halaman yang disimpan. Konfigurasi khusus *BPK Peraturan Scraper*, seperti URL dasar dan filter penelusuran, ditampilkan pada Gambar III.34.



bpk-peraturan@1.0.0 configuration ×

Adjust advanced settings here. Declared schema defaults are loaded automatically.

Base url (optional)

Base URL of the BPK Peraturan portal.

Keywords (optional)

Free-text search, the portal's 'Pencarian' box.

Tentang (optional)

Match against a regulation's subject line ('Tentang').

Nomor (optional)

Regulation number ('Nomor'). Leave empty to match any.

Tahun (optional)

Years to include ('Tahun'). Leave empty for every year.

Entitas (optional)

Issuing bodies to include ('Entitas') — ministries, agencies, provinces, regencies and cities. Leave empty for all.

Jenis (optional)

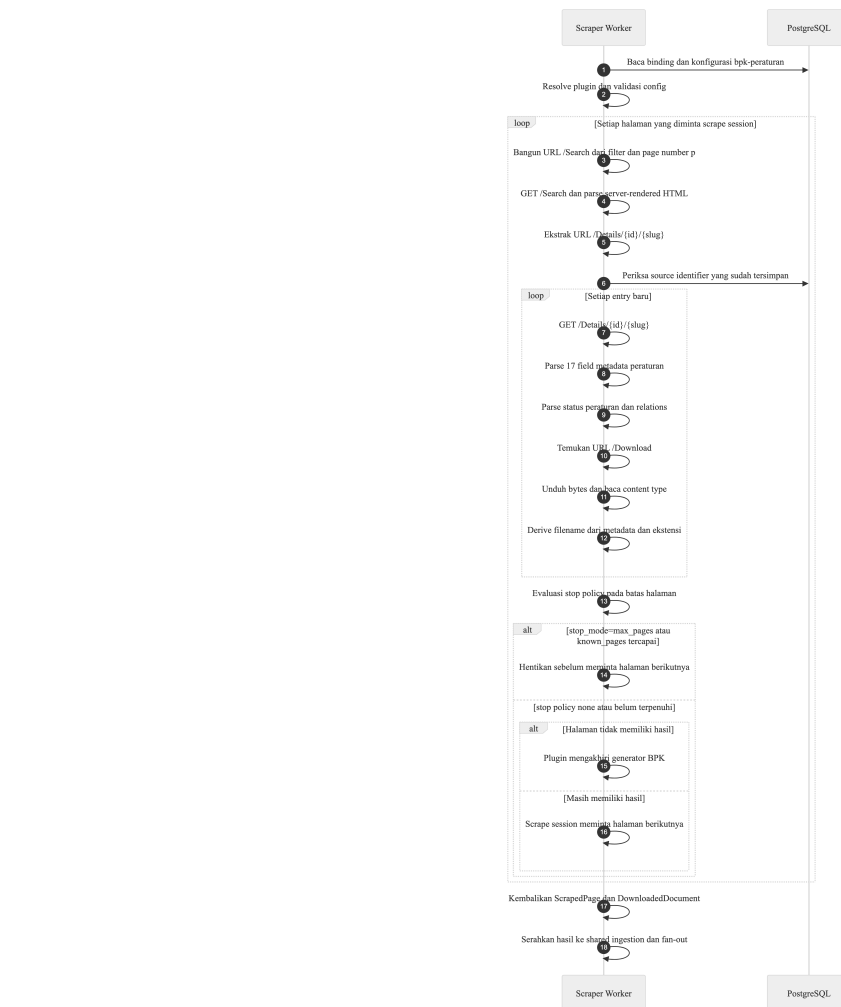
Regulation types to include ('Jenis'), such as Undang-undang or Peraturan Presiden. Leave empty for all.

Tag (optional)

Subject themes to include ('Tag'). Leave empty for every theme.

Cancel ✓ Apply configuration 

Gambar III.34: Konfigurasi *BPK Peraturan Scraper* pada *Admin UI*



Gambar III.35: Alur internal *BPK Peraturan Scraper* pada *Scraper Worker*

III.3.4.10 Eksekusi *Google Drive Scraper*

Google Drive Scraper mengakses satu folder Google Drive dengan kredensial *service account* untuk akses baca. Sebelum enumerasi, *plugin* memverifikasi bahwa *folder_id* dapat diakses dan benar-benar menunjuk pada folder, termasuk folder pada *shared drive*. Enumerasi dilakukan melalui *files.list* dengan *query* induk folder, *filter* *trashed=false*, parameter *pageToken*, serta *flag shared drive*. Setiap hasil *listing* menghasilkan *entry* dengan *source identifier* berupa *file ID* dan *metadata* seperti nama, *MIME type*, waktu perubahan, ukuran, dan tautan tampilan. Setelah *scrape session* menyingkirkan *file ID* yang sudah tersimpan, *plugin* mengunduh *entry* baru melalui *files.get_media* dan mengembalikan *byte* beserta *metadata* tersebut sebagai *DownloadedDocument*. Berbeda dari *BPK*, *metadata* Google Drive sudah tersedia sejak enumerasi dan tidak memerlukan halaman detail

tambahan. *stop policy* tetap diputuskan oleh *scrape session* pada batas halaman, sedangkan *plugin* berhenti sendiri ketika Google Drive tidak lagi mengembalikan *nextPageToken*. Konfigurasi *Google Drive Scraper* memuat folder tujuan, jadwal, dan kebijakan penelusuran, seperti ditunjukkan pada Gambar III.36.

The screenshot shows a configuration window for a 'Scraper Plugin'. At the top, the 'Plugin' is set to 'google-drive@1.0.0'. Below this, there is a 'Scheduled scraping' section with a toggle switch turned on, indicating it runs automatically on the schedule below. The 'Schedule (cron)' is set to '0 0 * * *' and the 'Timezone' is set to 'Asia/Jakarta'. A 'Crawl policy' section explains how far each run goes and how fast it requests. Under 'Stop condition', 'Enumerate everything' is selected. There are input fields for 'Enumerate delay min (s)' (1), 'Enumerate delay max (s)' (3), 'Download delay min (s)' (1), and 'Download delay max (s)' (3). A 'Parallel downloads' field is set to 5. A note states: 'Each request pauses a random interval between the min and max. Parallel downloads fetch that many entries of a page at once, raising the rate reaching the source; 5 is the recommended default.' At the bottom, there is a 'Plugin configuration' section showing '1 settings ready' and a 'Configure' button with a count of 1. The window has 'Cancel' and 'Create datasource' buttons at the bottom right.

Gambar III.36: Konfigurasi *Google Drive Scraper* pada *Admin UI*

Parameter khusus plugin Google Drive ditampilkan melalui dialog konfigurasi berikut.

google-drive@1.0.0 configuration

Adjust advanced settings here. Declared schema defaults are loaded automatically.

Folder id *

Google Drive folder ID to scrape (non-recursive).

Service account json env *

Name of the environment variable holding the service account JSON credentials.

Filename regex (optional)

Optional regular expression; only files whose names match are downloaded. Accepted file extensions come from the datasource, not from here.

Page size (optional)

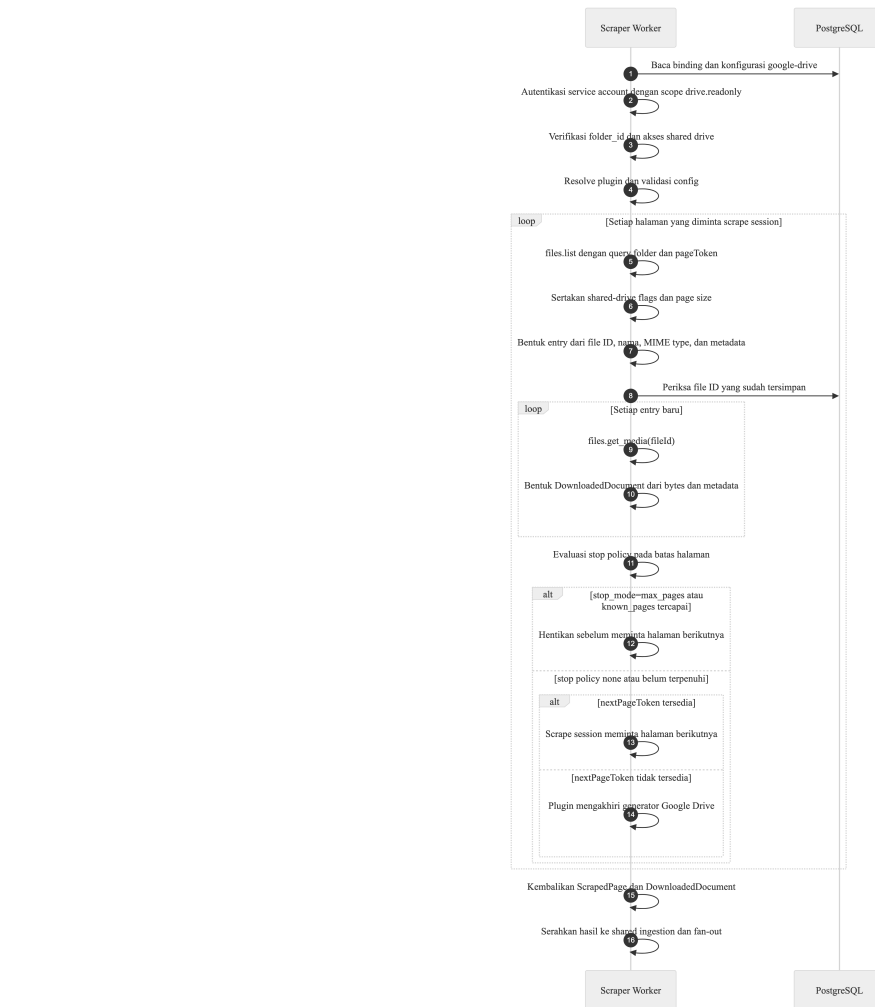
1000

Files requested per Drive listing call. The API caps this at 1000.

Cancel

✓ Apply configuration

Gambar III.37: Konfigurasi plugin *Google Drive Scraper* pada *Admin UI*

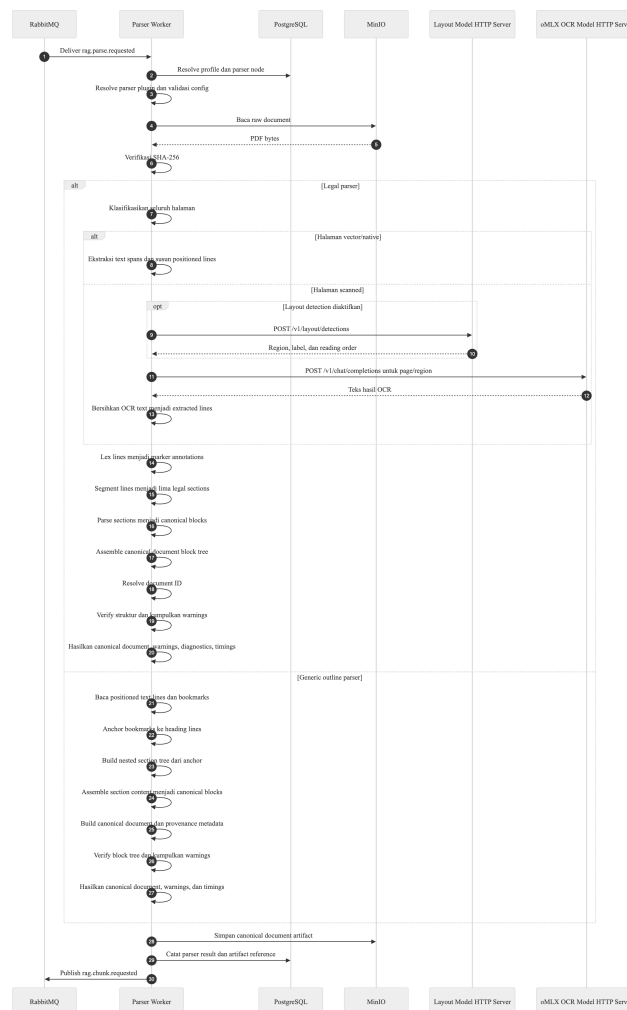


Gambar III.38: Alur internal *Google Drive Scraper* pada *Scraper Worker*

III.3.4.11 Parsing dan Handoff ke Chunker

Parser Worker menerima `rag.parse.requested` dari RabbitMQ dan memeriksa identitas *run*, *profile*, serta dokumen pada PostgreSQL. *Parser Worker* mengambil konfigurasi *parser* dari *profile*, membaca *raw document* dari MinIO, dan memverifikasi nilai *SHA-256* sebelum *plugin* dijalankan. Pada *legal parser*, halaman diklasifikasikan menjadi halaman *vector/native* dan *scanned*; halaman *vector/native* diekstraksi menjadi *positioned lines*, sedangkan halaman *scanned* dapat melewati deteksi *layout* dan OCR melalui *Layout Model HTTP Server* serta *oMLX OCR Model HTTP Server*. Baris yang dihasilkan kemudian diberi anotasi *marker*, disegmentasi menjadi bagian-bagian hukum, diproses oleh *section parser*, dan dirangkai menjadi *canonical document* berbentuk *block tree*. *Legal parser* juga menyelesaikan *document ID* serta mengumpulkan hasil verifikasi struktural, *warning*, *diagnostic*, dan

timing. Pada *generic outline parser*, *Parser Worker* membaca teks dan *bookmark*, menautkan *bookmark* ke baris judul, membangun *nested section tree*, mengisi isi setiap bagian menjadi *canonical block*, serta menambahkan *provenance metadata*. *Generic parser* memverifikasi *block tree* dan mengembalikan *warning* serta *timing* tanpa memanggil OCR atau layanan *layout*. Setelah kedua *plugin* menghasilkan *canonical document*, *Parser Worker* menyimpan *canonical artifact* pada MinIO, mencatat hasil *parser* pada PostgreSQL, dan menerbitkan *rag.chunk.requested* ke RabbitMQ.

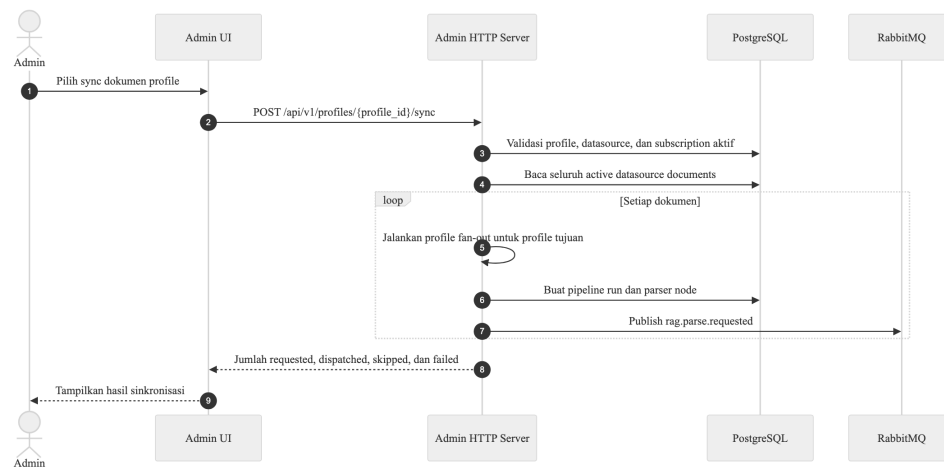


Gambar III.39: Alur interaksi *Parser Worker*, *plugin parser*, OCR/layout, dan *hand-off* ke *chunker*

III.3.4.12 Sinkronisasi Dokumen Lama ke *Profile*

Sinkronisasi digunakan ketika sebuah *profile* baru perlu memproses dokumen yang sudah tersimpan sebelum *profile* tersebut dibuat. Administrator memicu POST pada

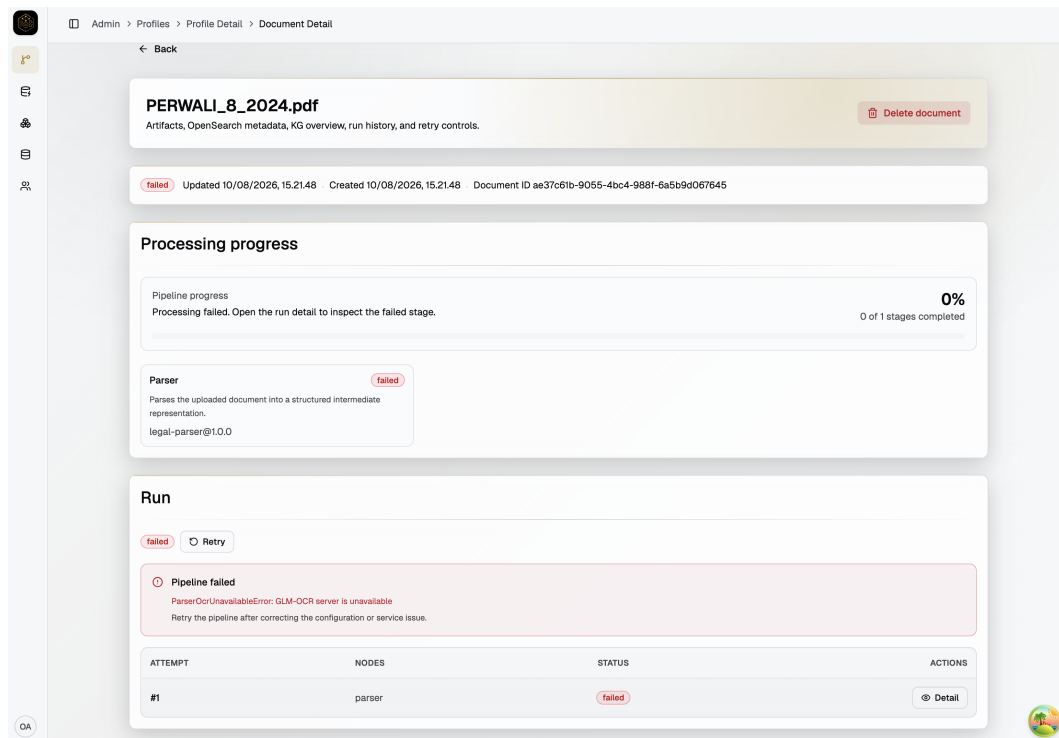
/api/v1/profiles/<profile_id>/sync melalui *Admin UI*. *Admin HTTP Server* memvalidasi bahwa *profile* terhubung dengan *datasource* aktif dan masih menjadi *subscriber* aktif, kemudian membaca seluruh dokumen aktif dari PostgreSQL. Untuk setiap dokumen, komponen internal *profile fan-out* membuat *pipeline run* dan *parser node* jika belum ada, lalu menerbitkan `rag.parse.requested` ke RabbitMQ. Tidak ada penyalinan ulang *raw document* karena pesan menunjuk pada *artifact* yang sudah disimpan di MinIO. Respons sinkronisasi mengembalikan jumlah dokumen yang diminta, dikirim, sudah tersedia, dilewati, dan gagal.



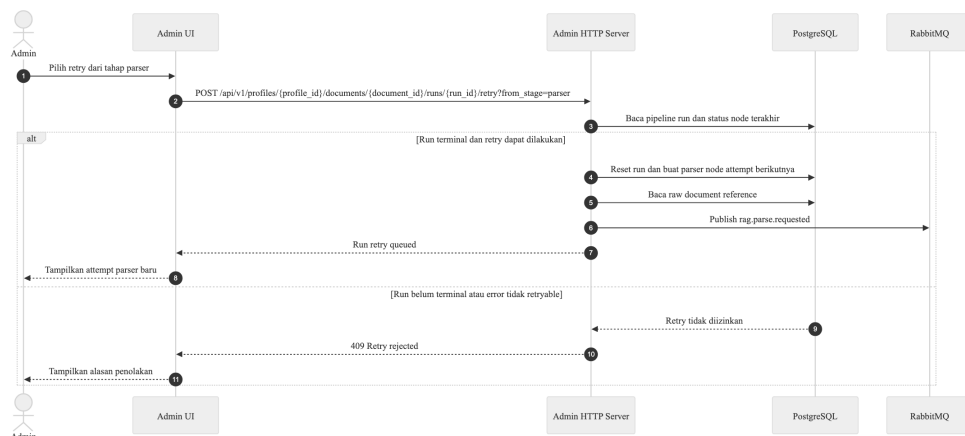
Gambar III.40: Alur interaksi sinkronisasi dokumen lama ke *profile*

III.3.4.13 Pengulangan Parsing

Pengulangan parsing digunakan untuk menjalankan kembali tahap *parser* pada *pipeline run* yang sudah berada pada status terminal dan memenuhi kebijakan *retry*. Administrator mengirimkan POST pada *retry endpoint* dokumen *profile* dengan parameter `from_stage=parser`. *Admin HTTP Server* membaca riwayat *run* dan *node* terakhir dari PostgreSQL, mereset status *run*, membuat *parser node* dengan nomor *attempt* berikutnya, dan mengambil kembali referensi *raw document*. Jika pengulangan dapat dilakukan, *Admin HTTP Server* menerbitkan `rag.parse.requested` ke RabbitMQ dan mengembalikan status bahwa *retry* telah diantri. Jika *run* belum terminal atau *error* tersimpan tidak dapat diulang, *request* ditolak tanpa membuat *attempt* baru. Detail dokumen pada *profile* menampilkan status parser, riwayat *attempt*, pesan kegagalan, dan kontrol *retry*, seperti ditunjukkan pada Gambar III.41.



Gambar III.41: Detail dokumen pada *profile* dengan kontrol *retry* parser



Gambar III.42: Alur interaksi pengulangan tahap *parser*

III.3.4.14 Pembatalan *Scrape Run*

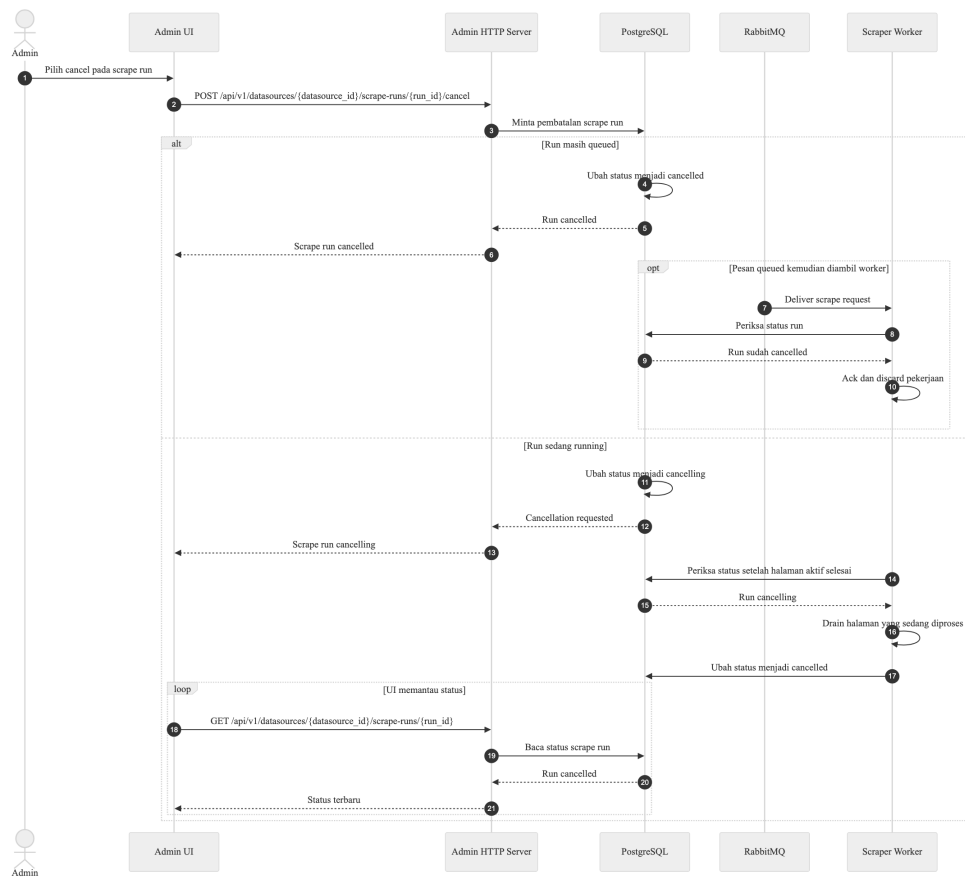
Pembatalan *scrape run* dimulai ketika administrator mengirimkan POST pada *cancel endpoint scrape run datasource*. Jika pesan masih berada pada status *queued*, PostgreSQL langsung mengubahnya menjadi *cancelled*; ketika pesan kemudian diambil *worker*, *worker* membaca status tersebut dan membuang pekerjaan tanpa menjalankan *plugin*. Jika *run* sudah *running*, *request* mengubah status menjadi *cancelling* dan mengembalikan informasi bahwa pembatalan sedang berlangsung.

Scraper Worker menyelesaikan halaman yang sedang diproses, mengevaluasi status pada batas halaman, lalu mengubah *run* menjadi *cancelled* sebelum berhenti. *Admin UI* dapat membaca status terbaru melalui *endpoint* detail *scrape run*. Transisi ini menerapkan *cooperative cancellation*; detail statusnya telah dirangkum pada Subsubsubbab III.3.3.2. Detail *scrape run* menampilkan status akhir, jumlah *entry*, dan urutan transisi *queued–running–cancelling–cancelled*, seperti ditunjukkan pada Gambar III.43.

The screenshot displays the 'Scrape Run' details in the Admin UI. The top section, 'Run summary', shows the run's status as 'Cancelled', triggered by 'Schedule', and provides timestamps for creation, start, and finish. It also indicates that 120 entries were total, with 120 skipped. The 'STATE HISTORY' section shows a sequence of states: Queued, Running, Cancelling, and finally Cancelled, each with a timestamp. The bottom section, 'Entries', lists individual scraped items with columns for status, source identifier, filename, created/updated times, and actions.

STATUS	SOURCE IDENTIFIER	FILENAME	CREATED	UPDATED	ACTIONS
Skipped	/Details/66531/pp-no-5-tahun-1980	pp-no-5-tahun-1980.pdf	18/08/2026, 13.58.20	18/08/2026, 13.58.20	Detail
Skipped	/Details/66646/pp-no-43-tahun-1980	pp-no-43-tahun-1980.pdf	18/08/2026, 13.58.20	18/08/2026, 13.58.20	Detail
Skipped	/Details/66338/pp-no-27-tahun-1981	pp-no-27-tahun-1981.pdf	18/08/2026, 13.58.20	18/08/2026, 13.58.20	Detail
Skipped	/Details/66341/pp-no-28-tahun-1981	pp-no-28-tahun-1981.pdf	18/08/2026, 13.58.20	18/08/2026, 13.58.20	Detail
Skipped	/Details/65283/pp-no-37-tahun-1982	pp-no-37-tahun-1982.pdf	18/08/2026, 13.58.20	18/08/2026, 13.58.20	Detail

Gambar III.43: Detail *scrape run* dan riwayat perubahan *state* pada *Admin UI*



Gambar III.44: Alur interaksi pembatalan *scrape run*

BAB IV

EKSPERIMEN DAN EVALUASI

Bab ini mengevaluasi komponen akuisisi dan ekstraksi dokumen yang dirancang pada Bab III. Evaluasi akuisisi mencakup *throughput*, kegagalan, deduplikasi, dan fleksibilitas terhadap sumber baru, sedangkan evaluasi ekstraksi mencakup penentuan jalur ekstraksi per halaman dan pemulihan struktur hierarkis dokumen. Urutan eksperimen ekstraksi mengikuti dependensi implementasinya. Aturan *routing* halaman ditetapkan terlebih dahulu, lalu digunakan untuk membentuk korpus evaluasi jalur *native* dan jalur OCR pada *legal parser*.

IV.1 Evaluasi *Datasource* dan *Scraper* Peraturan BPK

IV.1.1 Eksperimen *Scraper* Portal Peraturan BPK

IV.1.1.1 Tujuan dan Pertanyaan Evaluasi

Eksperimen ini mengevaluasi mekanisme akuisisi dokumen pada portal peraturan BPK. Evaluasi difokuskan pada tiga ukuran keberhasilan Tujuan 1, yaitu *throughput* ingesti, tingkat kegagalan, dan ketiadaan duplikasi penyimpanan.

Pertanyaan evaluasi yang dijawab oleh eksperimen ini adalah sebagai berikut.

1. Bagaimana perubahan nilai *concurrency* dan jeda permintaan memengaruhi durasi serta *throughput* ingesti?
2. Apakah konfigurasi dengan *throughput* lebih tinggi menghasilkan lebih banyak *failure* atau mengganggu kelengkapan enumerasi?
3. Apa penyebab *failure* yang terjadi, dan apakah penyebab tersebut berkaitan dengan konfigurasi atau dengan kondisi sumber?
4. Apakah *failure* pada satu entri dapat menghentikan pemrosesan entri lain atau keseluruhan *run*?

5. Apakah dokumen dengan isi identik hanya menghasilkan satu salinan *payload* di *storage* meskipun memiliki *source identifier* yang berbeda?

IV.1.1.2 Rancangan Eksperimen

Portal peraturan BPK dipilih sebagai sumber eksperimen utama karena menyediakan dokumen peraturan perundang-undangan yang menjadi korpus domain hukum pada tugas akhir ini. Google Drive tidak digunakan sebagai sumber eksperimen karena hanya disiapkan untuk evaluasi integrasi terhadap sumber kedua. Portal BPK juga tidak mencantumkan batas laju permintaan publik yang terdokumentasi. Keadaan tersebut membuat pengaruh *concurrency* dan jeda permintaan perlu ditentukan melalui eksperimen, bukan melalui satu nilai batas yang telah ditetapkan oleh sumber.

Korpus eksperimen diperoleh dari hasil pencarian portal dengan *tag* Pendidikan dan pendidikan dan pelatihan. Korpus tersebut digunakan untuk keenam konfigurasi sehingga perbandingan antarkonfigurasi dilakukan pada himpunan entri yang sama. *Datasource* hanya menerima berkas PDF. Enumerasi dilakukan sampai portal mengembalikan halaman kosong dengan kebijakan penghentian *stop_mode=none*.

Setiap konfigurasi dijalankan satu kali pada *datasource* baru. Seluruh *run* dijalankan secara berurutan pada lingkungan yang sama, yaitu Docker Compose dengan satu *scraper worker*, RabbitMQ dengan *prefetch* satu, PostgreSQL, dan MinIO. Tidak terdapat *profile* aktif maupun proses *pipeline* hilir selama eksperimen. Versi kode, citra Docker, kredensial, jaringan, dan mesin dipertahankan sama untuk seluruh konfigurasi. Dengan demikian, perubahan hasil dapat dikaitkan dengan parameter konfigurasi yang diuji, sementara faktor lingkungan dijaga tetap.

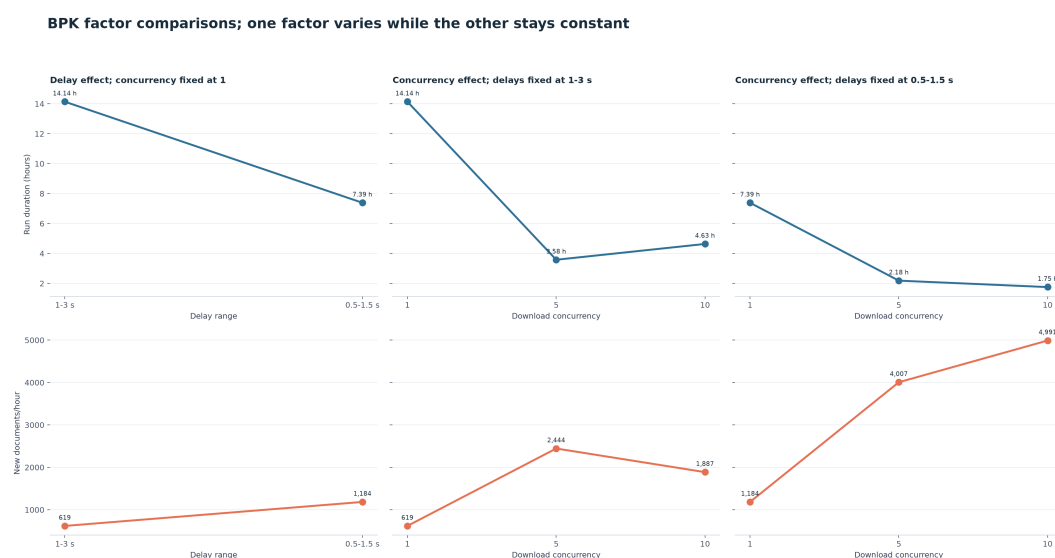
Variabel konfigurasi terdiri atas *download_concurrency*, dua parameter jeda enumerasi, dan dua parameter jeda pengunduhan. Parameter jeda enumerasi adalah *enumerate_delay_min_seconds* dan *enumerate_delay_max_seconds*. Parameter jeda pengunduhan adalah *download_delay_min_seconds* dan *download_delay_max_seconds*. Metrik yang dicatat meliputi durasi *run*, jumlah entri, jumlah dokumen baru, jumlah *skip*, jumlah *failure*, *throughput*, kategori *failure*, jalur perubahan status, dan hasil deduplikasi.

Tabel IV.1: Hasil enam konfigurasi eksperimen *scraper* BPK

Konfigurasi	Jeda enumerasi dan unduh	download_concurrency	Durasi (jam)	Dokumen/jam	Failure
bpk-00-baseline	1–3 detik	1	14,14	618,5	16
bpk-01-concurrency-5	1–3 detik	5	3,58	2 444,2	15
bpk-02-concurrency-10	1–3 detik	10	4,63	1 887,0	15
bpk-03-both-fast	0,5–1,5 detik	1	7,39	1 184,0	15
bpk-04-moderate-stress	0,5–1,5 detik	5	2,18	4 007,4	15
bpk-05-high-stress	0,5–1,5 detik	10	1,75	4 990,5	15

IV.1.1.3 Perbandingan *Throughput* dan Durasi

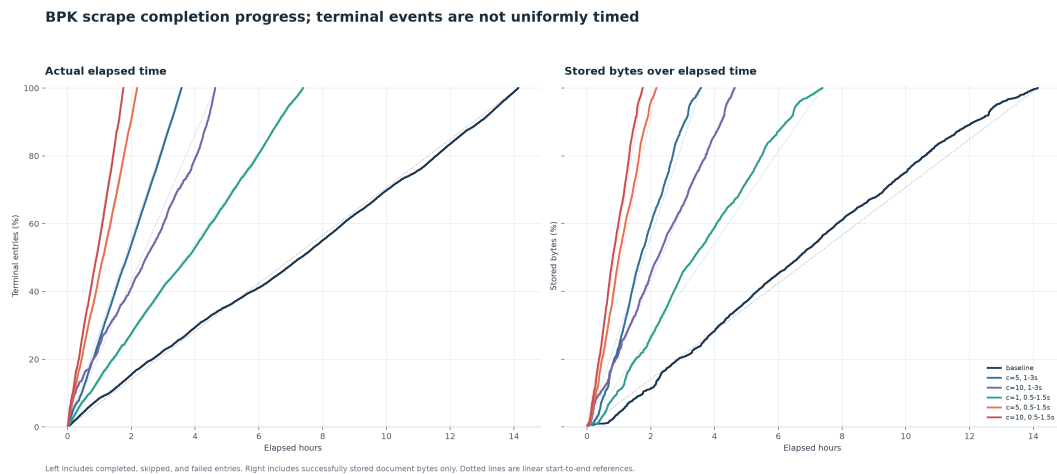
Keenam *run* memproses 8.822 entri yang sama dan masing-masing menghasilkan 8.745 dokumen baru. Perbedaan antarkonfigurasi terutama terlihat pada waktu yang diperlukan untuk mencapai hasil tersebut. Konfigurasi bpk-00-baseline membutuhkan waktu 14,14 jam dengan *throughput* 618,5 dokumen baru per jam. Sebaliknya, konfigurasi bpk-05-high-stress selesai dalam 1,75 jam dengan *throughput* 4.990,5 dokumen baru per jam.

Gambar IV.1: Perbandingan durasi dan *throughput* pada konfigurasi *scraper* BPK

Perbandingan tersebut menunjukkan bahwa pengaruh *concurrency* bergantung pada rentang jeda yang digunakan. Pada jeda 1–3 detik, *concurrency* lima menghasilkan durasi 3,58 jam dan *throughput* 2.444,2 dokumen baru per jam, sedangkan *concurrency* sepuluh menghasilkan durasi 4,63 jam dan *throughput* 1.887,0 dokumen baru per jam. Pada jeda 0,5–1,5 detik, *concurrency* sepuluh justru lebih cepat daripada *concurrency* lima, yaitu 1,75 jam dibandingkan 2,18 jam. Oleh karena itu, *concurrency* yang lebih tinggi tidak dapat dianggap selalu menghasilkan *throughput* yang

lebih tinggi.

IV.1.1.4 Bentuk Penyelesaian *Run*



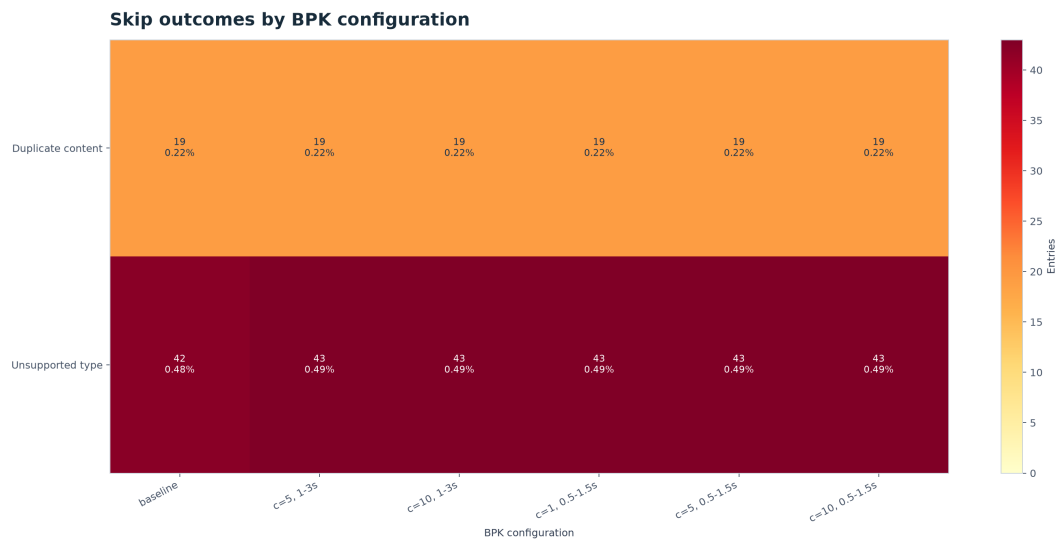
Gambar IV.2: *Progress* penyelesaian entri dan akumulasi *byte* tersimpan pada ke-nam *run* BPK

Gambar IV.2 memperlihatkan perkembangan pekerjaan berdasarkan waktu yang telah berlalu sejak setiap *run* dimulai. Panel kiri menghitung persentase entri yang telah mencapai status terminal, termasuk entri berhasil, *skip*, maupun *failure*. Panel kanan menghitung akumulasi ukuran dalam *byte* dari dokumen baru yang berhasil disimpan. Karena panel kanan hanya memasukkan dokumen yang tersimpan, entri yang berakhir sebagai *skip* atau *failure* tidak menambah nilainya. Selain itu, satu entri dapat memiliki berkas yang jauh lebih besar daripada entri lain. Akibatnya, penyelesaian sejumlah entri tidak selalu menghasilkan kenaikan jumlah *byte* yang sebanding.

IV.1.1.5 Hasil *Run* Representatif dan Kategori *Failure*

Seluruh *run* berstatus *completed* dan menyelesaikan enumerasi pada himpunan 8.822 entri yang sama. Status *completed* pada tingkat *run* berarti enumerasi telah selesai, bukan berarti setiap entri berhasil disimpan sebagai dokumen baru. Karena hasil *skip*, jalur status, dan kategori *failure* pada konfigurasi *non-baseline* berulang, satu *run* digunakan untuk menjelaskan pola umum. *Run* representatif tersebut adalah *bpk-01-concurrency-5*, yaitu konfigurasi yang kemudian direkomendasikan sebagai titik operasi.

Pada *run* representatif, 19 entri dilewati karena isi berkas identik dan 43 entri dilewati karena tipe berkas tidak didukung. Ke-43 entri tersebut terdiri atas 24 berkas .doc, 16 berkas .docx, dua berkas .rar, dan satu berkas .rtf. Setiap entri dengan tipe yang tidak didukung mengikuti jalur pending->downloading->downloaded->skipped. Hal ini menunjukkan bahwa kebijakan tipe berkas diterapkan setelah berkas diterima dan ekstensi aktualnya diketahui.

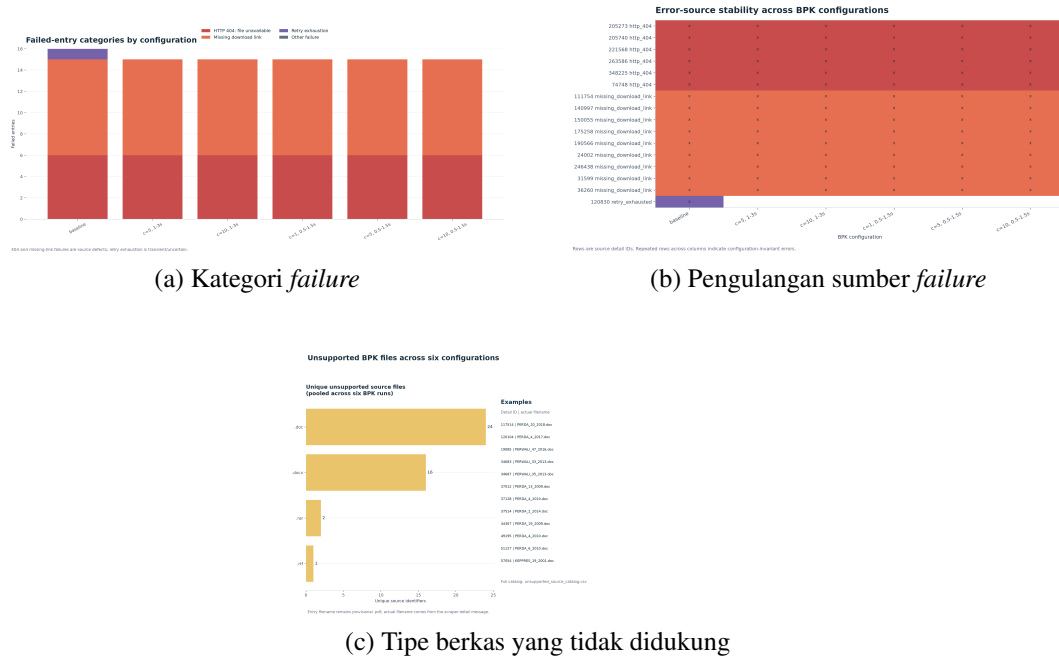


Gambar IV.3: Hasil *skip* pada *run* representatif *scraper* BPK

Pada *run* representatif terdapat 15 *failure*. Enam *failure* berupa HTTP 404 pada tautan unduh, sedangkan sembilan lainnya berupa tautan unduh yang tidak tersedia. Kelima *run* lain menunjukkan sumber dan kategori *failure* yang sama. Dengan demikian, kedua kategori tersebut lebih tepat dipahami sebagai kondisi sumber daripada akibat perbedaan *concurrency* atau jeda.

Konfigurasi *baseline* memiliki satu perbedaan, yaitu 42 *unsupported-file skip* dan 16 *failure*, sedangkan konfigurasi lain memiliki 43 *unsupported-file skip* dan 15 *failure*. Perbedaan tersebut berasal dari sumber dengan *source identifier* /Details/120830/perbup-kab-purbalingga-no-9-tahun-2019. Pada konfigurasi *baseline*, proses menghabiskan percobaan ulang sebelum menerima berkas .rtf. Pada lima konfigurasi lain, berkas yang sama berhasil diterima lalu dilewati karena tipe berkasnya tidak didukung. Tidak ditemukan HTTP 429 maupun HTTP 500 pada keenam *run*. Anomali tersebut karena itu kemungkinan besar berkaitan dengan gangguan jaringan pada sisi klien yang menyebabkan *timeout* atau *retry exhaustion*,

bukan akibat *rate limiting* dari *server*.



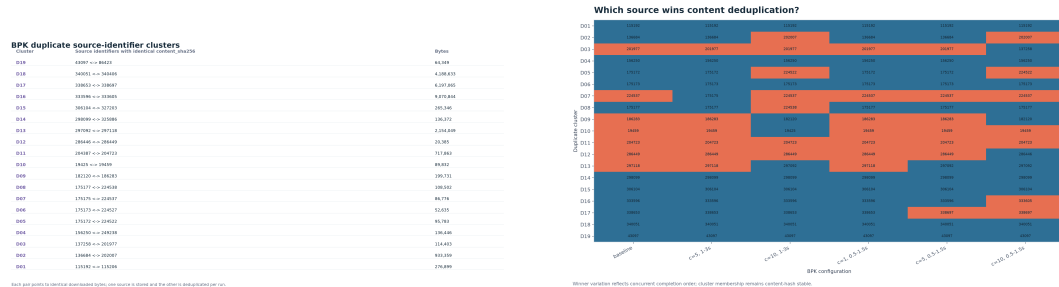
Gambar IV.4: Kategori *failure*, kestabilan sumber, dan tipe berkas yang tidak didukung pada *scraper* BPK

Tidak ditemukannya HTTP 429 pada seluruh *run* menunjukkan bahwa portal peraturan BPK tidak memperlihatkan mekanisme *rate limiting* terhadap pola permintaan yang digunakan selama eksperimen. Kesimpulan ini bersifat terbatas pada kondisi eksperimen dan tidak menyatakan bahwa portal tersebut tidak memiliki pembatasan laju pada kondisi lain. Kegagalan pada tingkat dokumen juga tidak menghentikan pemrosesan entri lain karena setiap entri diproses secara terisolasi. Hal ini berbeda dari kegagalan enumerasi terminal, yang tetap dapat menghentikan keseluruhan *run* sesuai rancangan pada Subsubsubbab III.2.2.2 dan Subsubsubbab III.2.2.4.

IV.1.1.6 Deduplikasi dan Identitas Sumber

Pada tahap awal akuisisi, *source identifier* digunakan untuk melacak entri yang diberikan portal BPK. Identitas tersebut digunakan untuk mengambil halaman rincian, menghubungkan metadata, dan merekam riwayat pemrosesan entri. Setelah berkas PDF berhasil diterima, sistem menghitung *hash* atas seluruh isi berkas. Nilai *hash* kemudian digunakan untuk menentukan apakah *payload* identik sudah pernah disimpan.

Pada *run* representatif ditemukan 19 pasang *source identifier*. Setiap pasangan memiliki nilai *content_sha256* yang sama. Keanggotaan seluruh kluster tersebut stabil pada keenam *run*, sehingga jumlah kluster dan hasil deteksi duplikasi dapat dijelaskan melalui satu *run* representatif. Deduplikasi berdasarkan *hash* memastikan bahwa isi PDF yang identik tidak disimpan sebagai dua salinan *payload*.



Gambar IV.5: Deduplikasi berdasarkan kesamaan isi pada eksperimen *scraper* BPK

Aspek yang berubah antarkonfigurasi bukan keanggotaan kluster, melainkan *winner*, yaitu *source identifier* yang lebih dahulu mencapai tahap penyimpanan. Sebelas dari 19 kluster mengalami perubahan *winner* pada setidaknya satu konfigurasi. Perubahan tersebut dapat terjadi karena urutan hasil enumerasi portal tidak dijamin stabil dan *concurrency* mengubah urutan penyelesaian pengunduhan. Dengan demikian, variasi *winner* tidak menunjukkan bahwa deduplikasi gagal. Variasi tersebut menunjukkan bahwa sistem perlu memisahkan identitas sumber dari identitas isi berkas.

IV.1.1.7 Kesimpulan Eksperimen

Kesimpulan eksperimen ini disusun untuk menjawab lima pertanyaan evaluasi pada bagian sebelumnya.

1. Perubahan *concurrency* dan jeda permintaan memengaruhi durasi serta *throughput* secara bersamaan. Konfigurasi *bpk-05-high-stress* menghasilkan *throughput* tertinggi, yaitu 4.990,5 dokumen baru per jam, tetapi hasil perbandingan menunjukkan bahwa *concurrency* lebih tinggi tidak selalu lebih cepat. Konfigurasi operasional yang dipilih adalah *download_concurrency=5* dengan jeda 1–3 detik karena memberikan *trade-off* yang lebih konservatif, dengan durasi 3,58 jam dan *throughput* 2.444,2 dokumen baru per jam.

2. *Throughput* yang lebih tinggi tidak mengurangi kelengkapan enumerasi atau meningkatkan *failure* pada keenam *run* yang diamati. Seluruh konfigurasi memproses 8.822 entri, menghasilkan 8.745 dokumen baru, dan berstatus *completed*. Jumlah *failure* juga serupa pada konfigurasi *non-baseline*; perbedaan pada *baseline* disebabkan oleh satu anomali sumber.
3. *Failure* yang berulang terutama berasal dari kondisi sumber, yaitu HTTP 404 dan tautan unduh yang tidak tersedia. Satu *retry exhaustion* pada *baseline* kemungkinan berkaitan dengan gangguan jaringan pada sisi klien karena tidak ditemukan HTTP 429 maupun HTTP 500. Tipe berkas yang tidak didukung tidak dihitung sebagai *failure*, melainkan sebagai *skip* sesuai kebijakan *data-source*.
4. *Failure* pada satu entri tidak menghentikan pemrosesan entri lain. Setiap entri diproses secara terisolasi, sehingga seluruh *run* tetap dapat menyelesaikan enumerasi meskipun sebagian entri berakhir sebagai *failure* atau *skip*.
5. Deduplikasi mencegah penyimpanan ulang terhadap isi yang identik. Setelah pemeriksaan awal berdasarkan *source identifier*, *content_sha256* menjadi sumber kebenaran akhir untuk identitas isi. Eksperimen menemukan 19 kluster duplikasi; pada setiap kluster hanya satu dokumen disimpan, sedangkan entri lain dengan hash yang sama dianggap sebagai duplikat dan berakhir sebagai *skipped*.

IV.1.2 Analisis Data Korpus BPK

IV.1.2.1 Ruang Lingkup dan Korpus Analisis Data

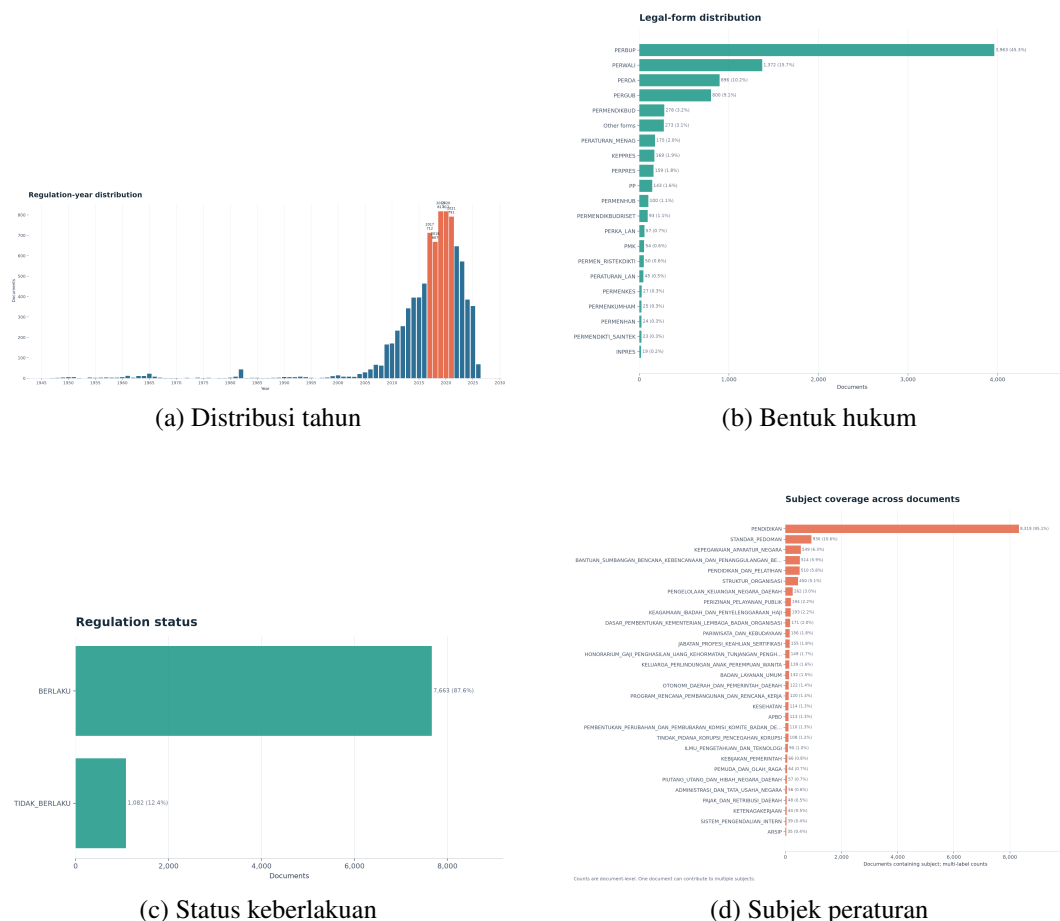
Analisis data dilakukan pada *metadata* dokumen di PostgreSQL untuk *datasource* bpk-peraturan-pendidikan. Korpus terdiri atas 8.745 dokumen dengan total ukuran penyimpanan sekitar 20,07 GB. Median ukuran berkas adalah 463,74 KB, dan tahun peraturan yang tercatat berada pada rentang 1947–2026.

Tabel IV.2: Ringkasan korpus pada analisis *metadata* BPK

Karakteristik	Nilai
Jumlah dokumen	8.745
Total <i>storage</i>	20,07 GB
Median ukuran berkas	463,74 KB
Rentang tahun peraturan	1947–2026
Dokumen dengan relasi	2.839
Jumlah relasi	4.364

IV.1.2.2 Komposisi Korpus Peraturan

Empat panel pada Gambar IV.6 memperlihatkan komposisi korpus berdasarkan tahun peraturan, bentuk hukum, status keberlakuan, dan subjek. Keempat distribusi tersebut digunakan untuk membaca periode dokumen, jenis peraturan, keadaan keberlakuan, serta fokus topik korpus yang berhasil disimpan.



Gambar IV.6: Komposisi korpus peraturan BPK berdasarkan *metadata*

Panel distribusi tahun menunjukkan bahwa dokumen dalam korpus berasal dari rentang 1947–2026. Jumlah tertinggi berada pada tahun 2019 dan 2020, masing-

masing sebanyak 817 dokumen. Dengan demikian, korpus mencakup dokumen historis, tetapi konsentrasinya lebih besar pada peraturan yang relatif baru.

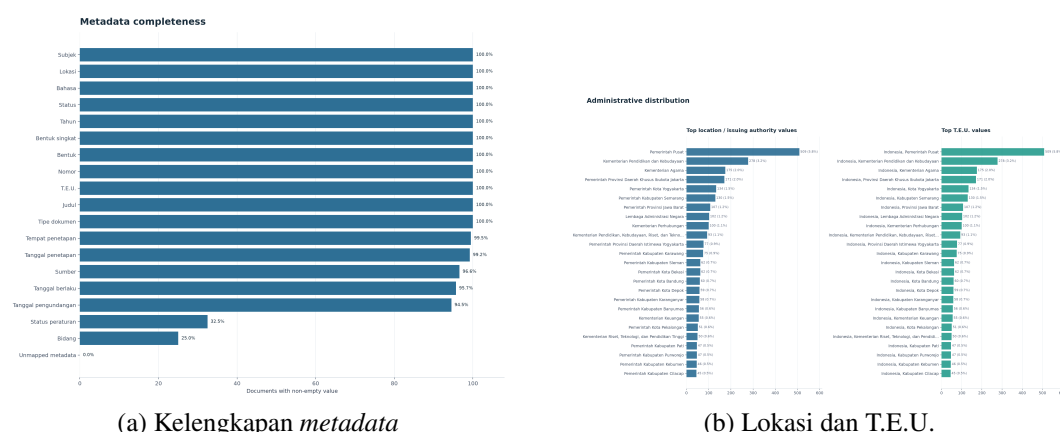
Panel bentuk hukum menunjukkan dominasi empat bentuk peraturan, yaitu PERBUP, PERWAL I, PERDA, dan PERGUB. Keempat bentuk tersebut mencakup 80,4% dari seluruh dokumen. Komposisi ini menunjukkan bahwa korpus terutama berisi peraturan yang berkaitan dengan pemerintahan daerah dan administrasi regional.

Status BERLAKU mencakup 87,6% dokumen. Status TIDAK_BERLAKU mencakup 12,4% lainnya. Dokumen yang tidak berlaku tetap menjadi bagian yang terukur dalam korpus, sehingga distribusi tersebut memperlihatkan bahwa korpus tidak hanya berisi peraturan yang masih berlaku.

Panel subjek menunjukkan bahwa PENDIDIKAN tercatat pada 95,1% dokumen. Dominasi tersebut sesuai dengan *tag* pencarian yang digunakan saat akuisisi, tetapi keberadaan subjek lain menunjukkan bahwa korpus tetap mencakup topik administrasi regional yang lebih luas.

IV.1.2.3 Kelengkapan Metadata

Gambar IV.7 memperlihatkan dua aspek metadata, yaitu kelengkapan pengisian *field* serta distribusi nilai lokasi dan T.E.U. Panel pertama menunjukkan seberapa banyak dokumen memiliki nilai pada setiap *field*, sedangkan panel kedua menunjukkan nilai administratif yang paling sering muncul.



Gambar IV.7: Kelengkapan dan variasi *metadata* korpus BPK

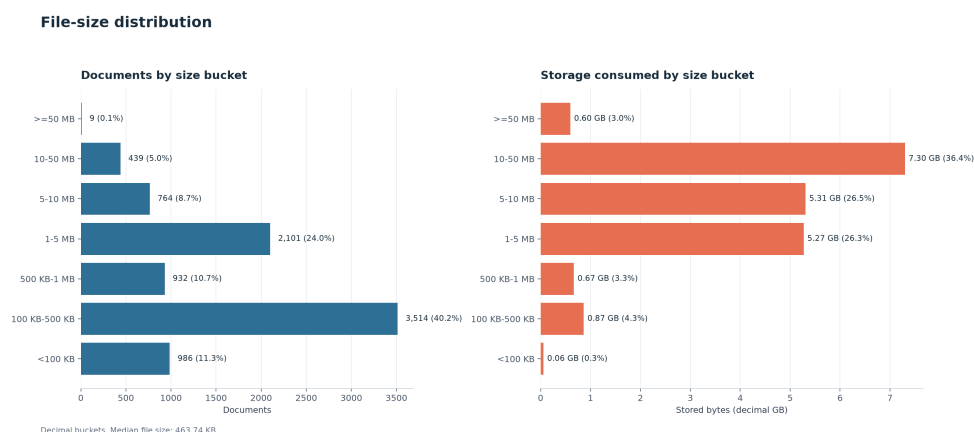
Pada panel kelengkapan, subjek tersedia pada 100% dokumen, sedangkan bidang hanya tersedia pada 25% dokumen. Perbedaan ini menunjukkan bahwa kelengkap-

an metadata tidak seragam antar-*field*. Untuk pengelompokan berdasarkan topik, sub jek menyediakan cakupan yang lebih konsisten daripada bidang.

Panel lokasi dan T.E.U. menunjukkan 552 nilai berbeda pada kedua metadata administratif tersebut. Nilai yang beragam menunjukkan bahwa korpus mencakup banyak lokasi atau instansi penerbit dan tidak terpusat pada satu nilai administratif. Pada saat yang sama, distribusi frekuensi yang tidak seragam menunjukkan bahwa sebagian nilai muncul jauh lebih sering daripada nilai lainnya.

IV.1.2.4 Distribusi Ukuran Berkas dan *Storage*

Gambar IV.8 membandingkan jumlah dokumen dan kontribusi *storage* pada setiap kelompok ukuran berkas. Panel pertama menunjukkan komposisi jumlah dokumen, sedangkan panel kedua menunjukkan jumlah ruang yang digunakan oleh kelompok ukuran yang sama.



Gambar IV.8: Distribusi ukuran berkas pada korpus BPK

Sebanyak 13,9% dokumen memiliki ukuran lebih dari 5 MB, tetapi kelompok tersebut menyumbang 65,9% dari total *storage*. Perbedaan antara proporsi dokumen dan proporsi penyimpanan menunjukkan bahwa beban *storage* tidak berbanding lurus dengan jumlah dokumen. Karena itu, distribusi ukuran berkas perlu diperhitungkan ketika memperkirakan kapasitas penyimpanan korpus.

IV.1.2.5 Relasi Antarperaturan

Gambar IV.9 memperlihatkan struktur relasi antarperaturan melalui jenis relasi, cakupan target, jumlah relasi keluar per dokumen, dan dokumen dengan jumlah relasi

Korpus memuat 4.364 relasi yang berasal dari 2.839 dokumen. Jenis relasi yang paling banyak muncul adalah pencabutan, dicabut oleh, perubahan, dan diubah oleh. Keempat jenis tersebut menunjukkan bahwa metadata korpus tidak hanya menyimpan dokumen secara terpisah, tetapi juga mencatat hubungan perubahan dan keberlakuan antarperaturan.

Distribusi jumlah relasi per dokumen menunjukkan bahwa sebagian besar dokumen memiliki sedikit atau tidak memiliki relasi, sedangkan hanya sebagian kecil dokumen memiliki banyak relasi. Panel dengan skala logaritmik menegaskan ekor panjang tersebut dengan memperlihatkan dokumen yang memiliki jumlah relasi jauh lebih tinggi daripada mayoritas dokumen.

112

daan relasi pada sumber.

IV.2 Evaluasi *Datasource* dan *Scraper* Google Drive

IV.2.1 Tujuan dan Pertanyaan Evaluasi

Evaluasi ini menilai fleksibilitas mekanisme akuisisi terhadap sumber baru menggunakan *google-drive-sanity*. Berbeda dari eksperimen BPK, evaluasi ini tidak digunakan untuk memilih konfigurasi atau membandingkan *throughput*. Fokusnya adalah memeriksa apakah sumber Google Drive dapat masuk ke alur akuisisi yang sama melalui *plugin* dan konfigurasi khusus sumber.

Pertanyaan evaluasi yang dijawab adalah sebagai berikut.

1. Apakah Google Drive dapat menggunakan kontrak *datasource* dan *scraper* yang sama dengan sumber BPK?
2. Apakah hasil enumerasi dan pengunduhan dari *plugin* Google Drive dapat diteruskan ke validasi tipe berkas, penyimpanan, deduplikasi, dan pencatatan *lifecycle* pada komponen inti?
3. Apakah penambahan sumber Google Drive dapat dilakukan tanpa mengubah orkestrasi inti?

IV.2.2 Rancangan Evaluasi

Evaluasi menggunakan satu folder Google Drive yang berisi sebelas entri, yaitu sepuluh PDF dan satu berkas ZIP. *Plugin* Google Drive mengakses folder melalui Google Drive API menggunakan *service account*, melakukan enumerasi dengan *pagination* standar, dan tidak menggunakan batas penghentian tambahan. ID berkas Google Drive digunakan sebagai *source identifier*, sedangkan nama berkas, tipe MIME, waktu perubahan, ukuran, dan tautan tampilan diteruskan sebagai metadata entri.

Datasource dikonfigurasi hanya untuk menerima berkas PDF. Kebijakan tipe berkas diterapkan oleh komponen inti, sehingga *plugin* tidak membuat aturan ekstensi kedua yang dapat berbeda dari konfigurasi *datasource*. Evaluasi menggunakan je-

da enumerasi dan pengunduhan 1–3 detik dengan *concurrency* lima. Konfigurasi tersebut dijalankan satu kali pada *datasource* baru.

Evaluasi ini tidak dirancang sebagai eksperimen pembandingan. Jumlah entri, ukuran berkas, API, mekanisme *pagination*, dan model *retry* Google Drive berbeda dari portal BPK, sehingga hasil waktunya tidak dibandingkan dengan *throughput* BPK.

IV.2.3 Hasil Pemrosesan

Plugin Google Drive memproses seluruh sebelas entri dalam 33,0 detik. Sembilan berkas PDF berhasil dibuat sebagai dokumen baru. Satu berkas PDF dilewati karena isinya sudah direpresentasikan oleh dokumen yang tersimpan, satu berkas *netgear.zip* dilewati karena tipe berkasnya tidak diterima, dan tidak terdapat *failure*. Sembilan entri PDF yang menjadi dokumen baru mengikuti jalur pending->downloading->downloaded->completed. Entri PDF duplikat mengikuti jalur pending->downloading->downloaded->skipped, sedangkan berkas ZIP mengikuti jalur pending->skipped sebelum pengunduhan.

Tabel IV.3: Hasil evaluasi integrasi *scraper* Google Drive

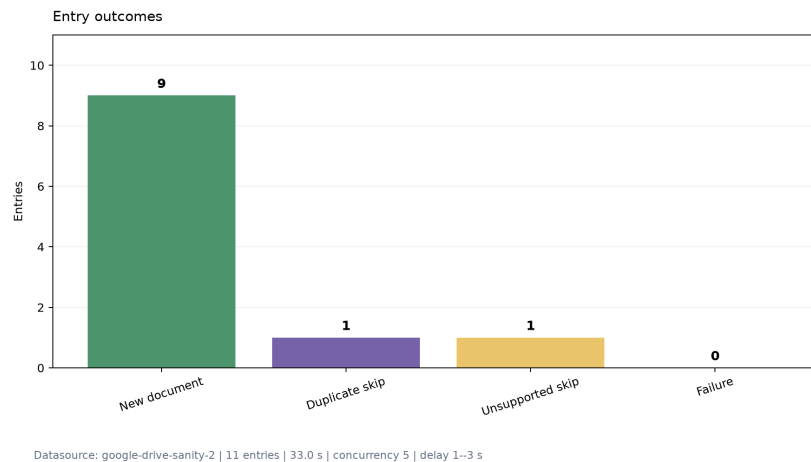
Metrik	Nilai
Total entri	11
Dokumen baru	9
<i>Skip</i> duplikat	1
<i>Skip</i> tipe berkas	1
<i>Failure</i>	0
Durasi	33,0 detik

IV.2.4 Analisis Fleksibilitas Sumber

Hasil pemrosesan menunjukkan bahwa mekanisme enumerasi Google Drive yang menggunakan API dapat dipetakan ke kontrak *scraper* yang sama dengan portal BPK. Perbedaan cara memperoleh daftar berkas dan mengunduh isi dokumen ditangani oleh *plugin* Google Drive, sedangkan komponen inti tetap menangani validasi tipe, penghitungan *hash*, deduplikasi, penyimpanan, dan pencatatan status.

Jalur sembilan PDF yang berakhir sebagai dokumen baru menunjukkan bahwa ke-luaran *plugin* dapat diteruskan sampai tahap penyimpanan tanpa perubahan pada

Google Drive latest integration run



Gambar IV.10: Hasil evaluasi integrasi *scraper* Google Drive

orkestrasi. Entri `netgear-xr500.pdf` memiliki *source identifier* yang berbeda dari `netgear-xr500(1).pdf`, tetapi nilai `content_sha256`-nya sama. Karena itu, entri tersebut dianggap sebagai duplikat setelah pengunduhan dan berakhir sebagai *skipped*.

Jalur `pending->skipped` pada `netgear.zip` menunjukkan bahwa kebijakan penerimaan berkas tetap berlaku untuk sumber kedua melalui konfigurasi *datasource*, bukan melalui logika khusus di dalam *plugin*. Dengan demikian, evaluasi ini memberikan bukti implementasi bahwa penambahan Google Drive dilakukan melalui *plugin* dan konfigurasi sumber, sementara alur inti, penyimpanan, dan deduplikasi tetap digunakan.

IV.3 Evaluasi *Legal Parser*

Evaluasi *legal parser* dibagi menjadi tiga eksperimen yang saling berurutan. Eksperimen pertama menilai *classifier* yang menentukan apakah setiap halaman diproses melalui ekstraksi teks *native* atau OCR. Eksperimen kedua menguji toleransi penggabungan fragmen teks pada korpus yang seluruh halamannya diperkirakan bertipe VECTOR. Eksperimen ketiga membandingkan model OCR dan penggunaan deteksi *layout* pada korpus yang seluruh halamannya diperkirakan bertipe SCAN.

IV.3.1 Eksperimen Klasifikasi Halaman *Vector* dan *Scan*

IV.3.1.1 Tujuan dan Pertanyaan Evaluasi

Eksperimen ini menetapkan aturan untuk mengarahkan setiap halaman PDF ke jalur ekstraksi yang sesuai. Halaman yang memiliki teks *native* yang memadai seharusnya tidak dikirim ke OCR karena OCR menambah biaya komputasi dan dapat memperkenalkan kesalahan pengenalan. Sebaliknya, halaman hasil pindai yang dilewatkan ke ekstraksi *native* dapat menghasilkan teks kosong atau teks yang tidak dapat digunakan untuk memulihkan struktur hukum.

Unit keputusan dibuat pada tingkat halaman karena satu dokumen dapat mencakup halaman *native*, halaman pindai dengan *text layer*, dan halaman pindai tanpa *text layer*, sebagaimana dianalisis pada Subsubbab III.1.3. Pertanyaan evaluasi yang dijawab adalah sebagai berikut.

1. Apakah ciri cakupan citra dan jumlah teks *native* dapat membedakan halaman VECTOR dan SCAN dengan tingkat kesalahan yang rendah?
2. Berapa nilai ambang cakupan citra dan jumlah teks minimum yang memberikan *recall* tinggi pada kelas SCAN?
3. Bagaimana penerapan ambang terpilih membentuk komposisi halaman dan dokumen pada seluruh korpus?

IV.3.1.2 Rancangan Eksperimen

Korpus eksperimen adalah *datasource* bpk-peraturan-pendidikan yang terdiri atas 8.745 dokumen aktif dengan 165.395 halaman. Sampel dipilih secara acak seragam tanpa pengembalian pada tingkat halaman menggunakan seed 20260816. Sebanyak 500 halaman diberi label secara manual tanpa memperlihatkan prediksi atau ciri *classifier* kepada pelabel.

Label VECTOR berarti teks *native* pada halaman sudah cukup untuk digunakan, sedangkan label SCAN berarti halaman membutuhkan OCR untuk memperoleh isi yang dapat diandalkan. Sebanyak 50 halaman ditampilkan kembali sehingga terdapat 550 presentasi untuk mengukur konsistensi pelabelan. Pengulangan ini mengu-

kur konsistensi terhadap halaman yang sama dan bukan kesepakatan antarpemilai.

Classifier menggunakan dua parameter. Parameter pertama adalah *scan_image_coverage_threshold*, yaitu rasio luas citra terbesar terhadap luas halaman. Parameter kedua adalah *scan_min_text_threshold*, yaitu jumlah karakter teks *native* setelah spasi di luar isi dibuang. Seluruh 2.121 kombinasi pada grid dievaluasi, dengan cakupan citra 0–1 pada interval 0,01 dan teks minimum 0–200 karakter pada interval 10 karakter.

Metrik yang dihitung adalah akurasi, *recall* dan *precision* kelas SCAN, *macro F1*, serta rasio halaman yang diarahkan ke OCR. Kandidat diprioritaskan berdasarkan *recall* SCAN, kemudian *macro F1*, *precision* SCAN, dan jumlah halaman yang diarahkan ke OCR.

IV.3.1.3 Hasil Evaluasi pada Sampel Berlabel

Kandidat yang dipilih menggunakan ambang cakupan citra 0,50 dan ambang teks minimum 100 karakter. Hasil evaluasinya dirangkum pada Tabel IV.4.

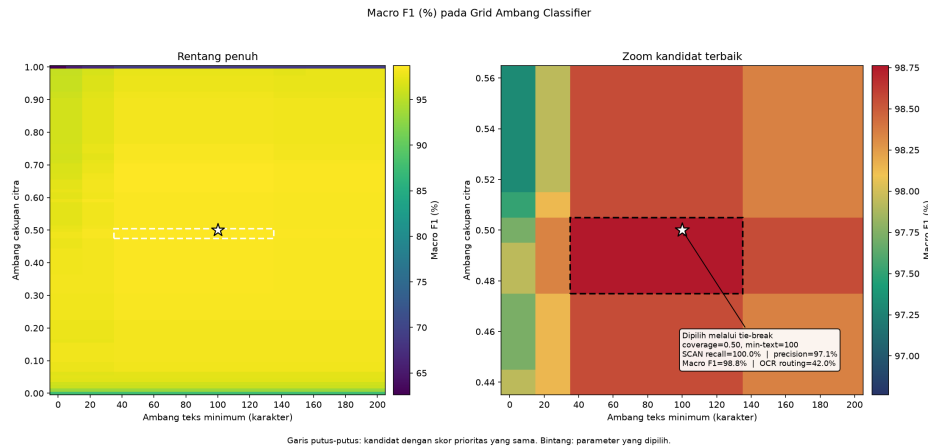
Tabel IV.4: Hasil evaluasi *classifier* halaman

Metrik	Nilai
Ambang cakupan citra	0,50
Ambang teks minimum	100 karakter
Akurasi	98,8%
<i>SCAN recall</i>	100,0%
<i>SCAN precision</i>	97,1%
<i>Macro F1</i>	98,8%
Rasio <i>routing</i> OCR pada sampel	42,0%
Konsistensi presentasi ulang	98,0%
Mismatch terhadap implementasi	0

Tidak ada halaman berlabel SCAN yang diprediksi sebagai VECTOR. Sebanyak 204 halaman berlabel SCAN diprediksi sebagai SCAN, sedangkan enam halaman berlabel VECTOR ikut diarahkan ke OCR. Hasil tersebut menunjukkan bahwa ambang yang dipilih lebih mengutamakan pencegahan halaman pindai lolos ke jalur *native* daripada meminimalkan seluruh pemanggilan OCR.

Tabel IV.5: Matriks kebingungan *classifier* pada sampel berlabel

	Prediksi SCAN	Prediksi VECTOR
Label SCAN	204	0
Label VECTOR	6	290



Gambar IV.11: Evaluasi grid ambang *classifier*; bintang menandai kandidat terpilih

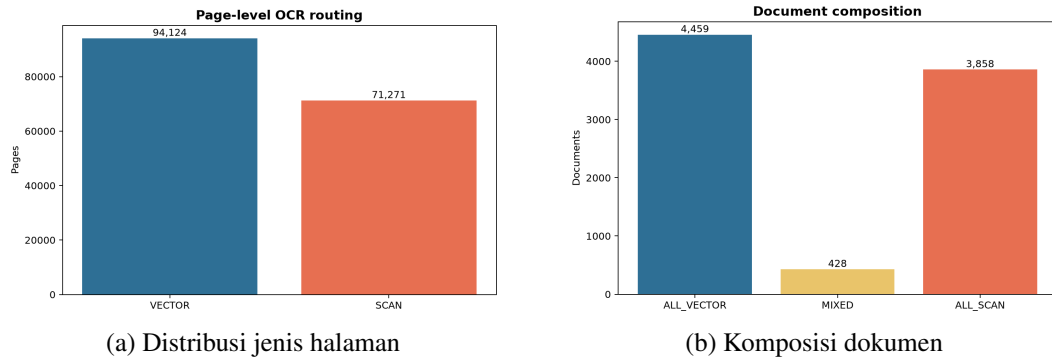
IV.3.1.4 Penerapan Ambang pada Seluruh Korpus

Ambang terpilih kemudian diterapkan pada seluruh halaman dalam *datasource*. Tahap ini merupakan analisis komposisi korpus berdasarkan prediksi *classifier*, bukan evaluasi kedua dengan label manual. Hasil *routing* pada tingkat halaman terdiri atas 71.271 halaman SCAN atau 43,09% dan 94.124 halaman VECTOR atau 56,91%.

Pada tingkat dokumen, 3.858 dokumen hanya terdiri atas halaman SCAN, 4.459 dokumen hanya terdiri atas halaman VECTOR, dan 428 dokumen memiliki campuran kedua jenis halaman. Kohort tersebut masing-masing mencakup 64.586, 79.017, dan 21.792 halaman. Keberadaan 428 dokumen campuran menjadi alasan penting untuk mempertahankan keputusan *routing* pada tingkat halaman, bukan pada tingkat dokumen.

Tabel IV.6: Komposisi korpus berdasarkan hasil *routing* halaman

Komposisi dokumen	Dokumen	Halaman	Proporsi halaman
ALL_SCAN	3.858	64.586	39,05%
ALL_VECTOR	4.459	79.017	47,77%
MIXED	428	21.792	13,18%
Total	8.745	165.395	100,00 %



Gambar IV.12: Komposisi hasil *routing* pada seluruh korpus peraturan

IV.3.1.5 Kesimpulan Eksperimen

Eksperimen menunjukkan bahwa dua ciri struktural PDF dapat digunakan untuk membuat keputusan *routing* pada tingkat halaman. Ambang cakupan citra 0,50 dan teks minimum 100 karakter dipilih karena mempertahankan *recall* kelas SCAN sebesar 100,0% pada sampel berlabel dengan *macro F1* sebesar 98,8%. Ambang tersebut digunakan sebagai kontrol tetap pada eksperimen *legal parser* berikutnya. Penerapan pada seluruh korpus memperkirakan bahwa 43,09% halaman akan menggunakan jalur OCR dan 13,18% halaman berada dalam dokumen campuran.

IV.3.2 Eksperimen *Legal Parser* pada Korpus ALL_VECTOR

IV.3.2.1 Tujuan dan Pertanyaan Evaluasi

Eksperimen ini mengevaluasi jalur ekstraksi teks *native* pada dokumen yang seluruh halamannya diprediksi sebagai VECTOR. Jalur tersebut membaca fragmen teks beserta posisi geometrinya, menggabungkan fragmen menjadi baris, lalu meneruskan baris tersebut ke analisis penanda dan pemulihan struktur hukum.

Pertanyaan evaluasi yang dijawab adalah sebagai berikut.

1. Bagaimana perubahan toleransi penggabungan vertikal dan horizontal mempengaruhi keutuhan struktur hasil *legal parser*?
2. Nilai toleransi mana yang memberikan *clean rate* terbaik pada korpus dokumen bertipe VECTOR?
3. Apakah jalur *native* dapat memproses seluruh korpus tanpa kegagalan termi-

nal dan dengan biaya yang sesuai untuk pemrosesan korpus?

IV.3.2.2 Rancangan Eksperimen

Eksperimen menggunakan sensus seluruh 4.459 dokumen ALL_VECTOR yang mencakup 79.017 halaman karena jalur *native* cukup murah secara komputasi dibandingkan jalur OCR sehingga tidak memerlukan pengambilan sampel. Dokumen MIXED tidak disertakan karena halaman pindainya akan menggunakan jalur OCR.

Sebanyak 48 profil dijalankan melalui kombinasi dua parameter. Parameter vertikal `merge_y_tolerance` diuji pada nilai 0, 2, 4, 6, 8, 10, 12, dan 16 pt. Parameter horizontal `merge_x_tolerance` diuji pada nilai 0, 0,5, 1, 2, 4, dan 6 pt. Ambang *routing* hasil eksperimen sebelumnya tetap menggunakan cakupan citra 0,50 dan teks minimum 100 karakter.

Dokumen disebut bersih apabila berhasil menghasilkan dokumen kanonik tanpa peringatan. Makna kode peringatan yang digunakan pada evaluasi telah ditetapkan pada Subsubsubbab III.2.3.6. Klasifikasi kode peringatan pada eksperimen dirangkum pada Tabel IV.7. Dokumen tanpa peringatan dikategorikan sebagai bersih. Jika terdapat kedua jenis peringatan, kategori *severe* diprioritaskan. Kegagalan dihitung terpisah karena dokumen tidak menghasilkan keluaran *parser*. Definisi *outcome* ini juga digunakan pada eksperimen jalur OCR pada subbab berikutnya.

Tabel IV.7: Klasifikasi kode peringatan pada eksperimen *legal parser*

Kategori	Kode	Interpretasi
<i>Advisory</i>	<code>sequence_gap, revise_pasal_count, BODY_STRUCTURAL_MARKER_IN_REVISION</code>	Dokumen kanonik tetap terbentuk, tetapi terdapat ketidakteraturan sumber atau konteks pembacaan yang perlu ditinjau.
<i>Severe</i>	<code>empty_leaf, revise_pasal_no_children, location_invalid, pembukaan_incomplete, empty_batang_tubuh</code>	Terdapat risiko lebih besar bahwa bagian struktur hilang, kosong, atau tidak terbentuk sesuai aturan domain.

Tabel IV.8: Rancangan eksperimen *legal parser* pada korpus ALL_VECTOR

Komponen	Nilai
Dokumen	4.459
Halaman	79.017
Jumlah profil	48
Level <code>merge_y_tolerance</code>	0, 2, 4, 6, 8, 10, 12, 16 pt
Level <code>merge_x_tolerance</code>	0, 0,5, 1, 2, 4, 6 pt
Jalur ekstraksi	Teks <i>native</i>

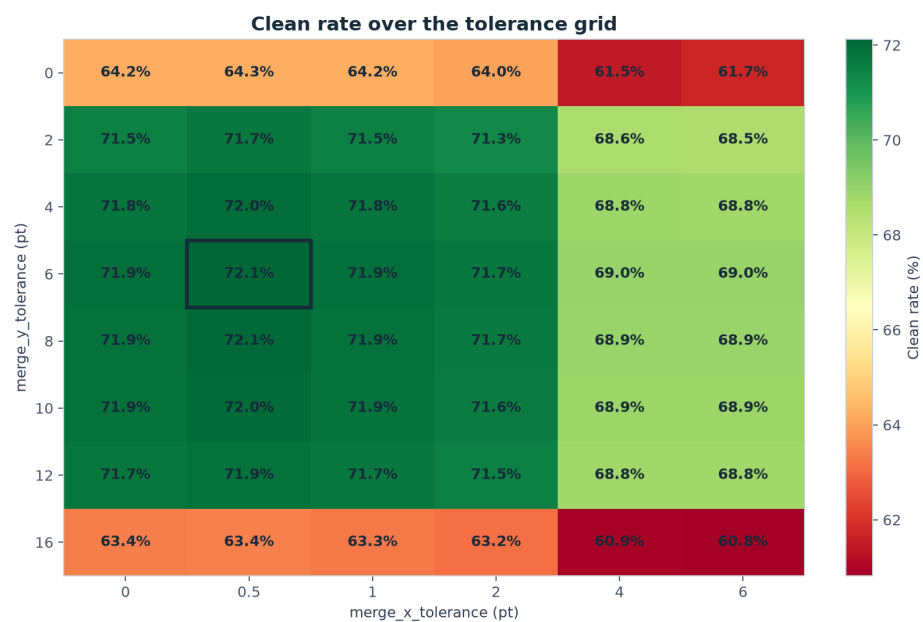
IV.3.2.3 Hasil dan Analisis Pengaruh Toleransi Penggabungan

Hasil seluruh profil memperlihatkan bahwa tidak ada dokumen yang gagal pada 48 profil. Dengan demikian, terdapat 214.032 pemrosesan dokumen tanpa kegagalan terminal. Perbedaan antarkonfigurasi terutama terlihat pada proporsi dokumen yang bersih dan jumlah peringatan struktural.

Profil y6-x0.5 dipilih sebagai profil operasional. Profil tersebut menghasilkan 3.216 dokumen bersih, 1.026 dokumen *advisory*, 217 dokumen *severe*, dan tidak memiliki kegagalan. *Clean rate*-nya adalah 72,1%. Perbandingan *clean rate* pada seluruh 48 profil ditampilkan pada Gambar IV.13.

Tabel IV.9: Hasil profil operasional *legal parser* pada korpus ALL_VECTOR

Profil	Bersih	Advisory	Severe	Gagal	Clean rate	Peringatan
y6-x0.5	3.216	1.026	217	0	72,1%	3.060



Clean means the document parsed with no warning at all. The outlined cell is the production default.

Gambar IV.13: *Clean rate* pada grid toleransi penggabungan jalur *native*

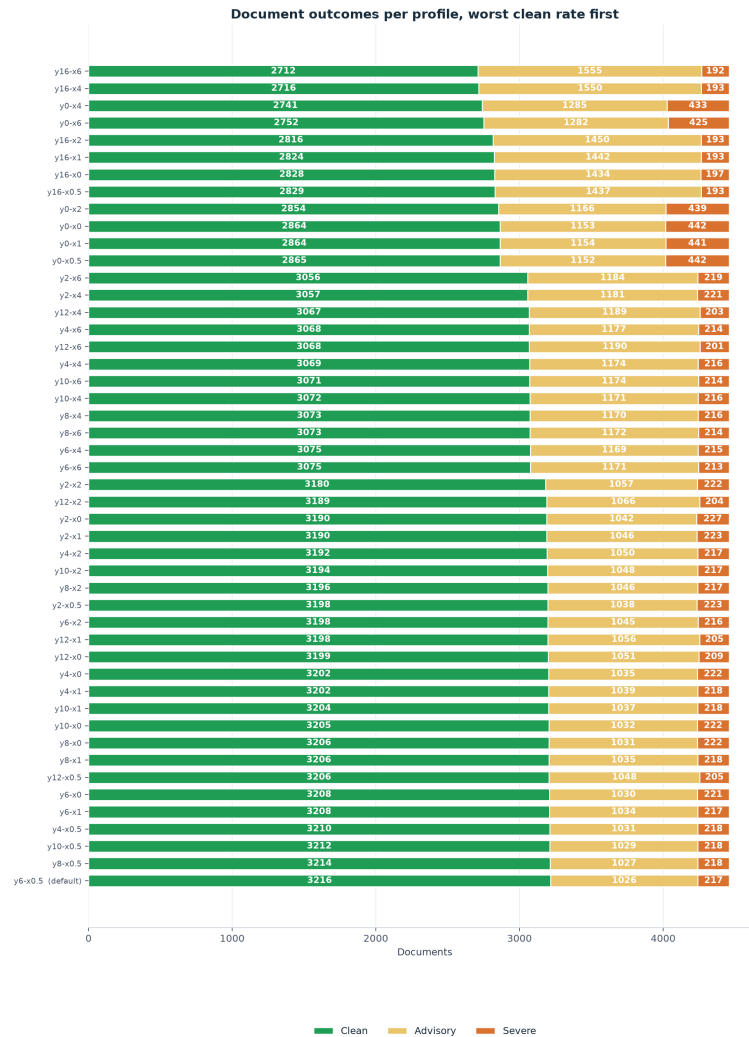
Rata-rata marginal adalah rata-rata *clean rate* pada seluruh nilai parameter lain. Dengan demikian, rata-rata untuk satu nilai *merge_y_tolerance* dihitung dari enam nilai *merge_x_tolerance*, sedangkan rata-rata untuk satu nilai *merge_x_tolerance* dihitung dari delapan nilai *merge_y_tolerance*. Angka marginal digunakan untuk membaca kecenderungan satu parameter secara terpisah dan bukan

untuk menggantikan hasil pada satu sel konfigurasi tertentu. Rata-rata marginal *merge_y_tolerance* meningkat dari 63,3% pada 0 pt menjadi 70,9% pada 6 pt. Nilainya berada pada sekitar 70,9% sampai 10 pt, lalu menurun menjadi 62,5% pada 16 pt. Pada parameter horizontal, nilai 0,5 pt menghasilkan rata-rata marginal tertinggi sebesar 69,9%, sedangkan nilai 4 dan 6 pt menghasilkan 66,9%. Artinya, toleransi vertikal antara 4–10 pt relatif stabil pada korpus ini, sementara toleransi horizontal yang terlalu besar cenderung menurunkan *clean rate*.

Tabel IV.10: *Clean rate* marginal berdasarkan toleransi penggabungan

Parameter	Nilai	<i>Clean rate</i> marginal
<i>merge_y_tolerance</i>	0 pt	63,3%
	2 pt	70,5%
	4 pt	70,8%
	6 pt	70,9%
	8 pt	70,9%
	10 pt	70,9%
	12 pt	70,7%
	16 pt	62,5%
<i>merge_x_tolerance</i>	0 pt	69,8%
	0,5 pt	69,9%
	1 pt	69,8%
	2 pt	69,6%
	4 pt	66,9%
	6 pt	66,9%

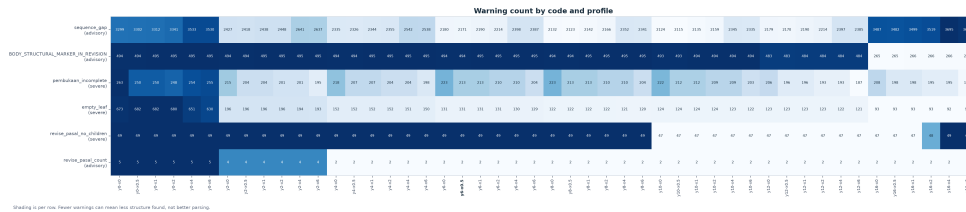
Komposisi *outcome* untuk seluruh 48 profil ditampilkan pada Gambar IV.14. Gambar tersebut memperlihatkan bahwa perubahan toleransi mengubah proporsi dokumen bersih, *advisory*, dan *severe*, meskipun seluruh profil tetap menghasilkan keluaran *parser* tanpa kegagalan.



Clean = no parser warning. Severe = at least one severe-code warning. No document failed in any profile, so the failed class is absent.

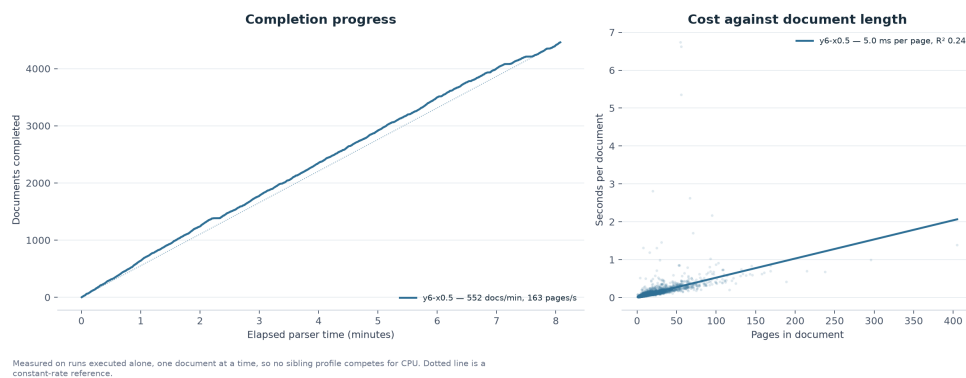
Gambar IV.14: Komposisi hasil pemrosesan pada grid profil *legal parser* jalur *native*

Pada profil y6-x0.5, peringatan yang paling banyak adalah `sequence_gap` sebanyak 2.171 kejadian. Peringatan tersebut tidak selalu menunjukkan kesalahan *parser* karena dokumen sumber sering kali memang melewati nomor tertentu. Peringatan lain yang muncul adalah `BODY_STRUCTURAL_MARKER_IN_REVISION` sebanyak 494 kejadian, `pembukaan_incomplete` sebanyak 213 kejadian, dan `empty_leaf` sebanyak 131 kejadian. Dua kode lain yang muncul adalah `revise_pasal_no_children` sebanyak 49 kejadian dan `revise_pasal_count` sebanyak 2 kejadian. Makna keenam kode tersebut mengikuti definisi pada Tabel III.7. Distribusi seluruh kode peringatan pada profil-profil yang diuji ditampilkan pada Gambar IV.15.



Gambar IV.15: Peringatan *parser* pada jalur *native*

Pengukuran *throughput* dilakukan pada profil y6-x0.5 dalam eksekusi *standalone* sehingga tidak bercampur dengan kontensi dari profil lain. Profil tersebut memproses 552 dokumen per menit atau 163 halaman per detik. Median waktu pemrosesan satu dokumen adalah 72 ms, sedangkan tambahan waktu per halaman setelah halaman pertama adalah sekitar 5,0 ms. Perkembangan pemrosesan dan hubungan waktu dengan jumlah halaman ditampilkan pada Gambar IV.16.



Gambar IV.16: *Throughput* profil y6-x0.5

IV.3.2.4 Analisis dan Kesimpulan Eksperimen

Toleransi vertikal yang terlalu ketat dapat memecah fragmen teks yang seharusnya berada pada satu baris, sedangkan toleransi yang terlalu longgar dapat menggabungkan fragmen dari baris yang berbeda. Pola tersebut terlihat pada penurunan *clean rate* di 0 pt dan 16 pt. Toleransi horizontal yang besar juga menurunkan *clean rate* karena lebih banyak fragmen yang dapat digabungkan tanpa pemisah yang semestinya. Profil $y6 - x0.5$ berada pada *plateau clean rate* terbaik dan dipilih sebagai konfigurasi *native* operasional. *Clean rate* dan peringatan pada eksperimen ini merupakan indikator operasional berbasis pemeriksaan *parser*, bukan pengukuran terhadap anotasi struktur hukum yang dibuat secara manual. Seluruh profil berha-

sil memproses korpus tanpa kegagalan, sedangkan profil operasional mencapai 552 dokumen per menit.

IV.3.3 Eksperimen *Legal Parser* pada Korpus ALL_SCAN

IV.3.3.1 Tujuan dan Pertanyaan Evaluasi

Eksperimen ini mengevaluasi jalur OCR pada dokumen hukum hasil pindai. Istilah *jalur OCR* merujuk pada tahap pengambilan teks halaman sebelum baris tersebut diproses oleh lexer, segmenter, dan pemeriksa struktur *legal parser*. Dengan demikian, eksperimen tidak hanya mengukur layanan OCR secara terpisah, tetapi mengukur dampaknya terhadap keluaran struktur hukum.

Pertanyaan evaluasi yang dijawab adalah sebagai berikut.

1. Model OCR mana yang menghasilkan proporsi dokumen bersih paling tinggi pada korpus peraturan hasil pindai?
2. Apakah deteksi *layout* meningkatkan keutuhan struktur atau justru menambah gangguan pada urutan baca?
3. Bagaimana perubahan model dan deteksi *layout* memengaruhi waktu pemrosesan, *throughput*, dan kegagalan pengenalan?

IV.3.3.2 Rancangan Eksperimen

Sampel dipilih dari 2.577 dokumen ALL_SCAN yang memiliki paling banyak 15 halaman. Sebanyak 250 dokumen dipilih secara acak tanpa pengembalian menggunakan seed 20260816. Sampel tersebut mencakup 2.249 halaman dan digunakan kembali pada keempat profil agar perbandingan dilakukan pada dokumen dan halaman yang sama.

Eksperimen menggunakan rancangan faktorial 2 x 2 yang mengombinasikan dua model OCR dan dua pilihan deteksi *layout*. Model OCR yang diuji adalah GLM-OCR dan PaddleOCR-VL 1.6. Deteksi *layout* menggunakan PP-DocLayoutV3 atau dinonaktifkan.

Parameter faktor dan kontrol eksperimen dirangkum pada Tabel IV.11.

Metrik yang dicatat adalah jumlah dan proporsi dokumen bersih, *advisory*, *severe*, dan gagal, jumlah peringatan berdasarkan kode, jenis kegagalan, waktu per halaman, *throughput* dokumen, *throughput* halaman, serta jumlah permintaan OCR per halaman.

Tabel IV.11: Parameter eksperimen *legal parser* jalur OCR

Parameter	Nilai	Peran
Model OCR	GLM-OCR; PaddleOCR-VL 1.6	Faktor eksperimen
Prompt OCR	Text Recognition;; OCR:	Prompt sesuai model OCR
Model layout	none; PP-DocLayoutV3	Faktor eksperimen
Region dikecualikan	chart, image, seal	Diterapkan saat PP-DocLayoutV3 digunakan
Resolusi rasterisasi	150 DPI	Kontrol kualitas citra halaman
scan_image_coverage_threshold	0,5	Ambang <i>routing</i> dari eksperimen classifier
scan_min_text_threshold	100	Ambang teks <i>native</i> dari eksperimen classifier
max_tokens	4.096	Batas keluaran satu permintaan OCR
temperature	0	Decoding deterministik
repetition_penalty	1,1	Mengurangi pengulangan keluaran
top_p	0,00001	Membatasi kandidat probabilitas token
top_k	1	Memilih token dengan probabilitas tertinggi
max_concurrent_requests	4	Batas permintaan konkuren
Ambang skor layout	0,3	Kontrol deteksi region

Parameter `merge_y_tolerance` dan `merge_x_tolerance` tidak menjadi faktor pada eksperimen ini karena jalur OCR membentuk baris dari keluaran model, bukan dari penggabungan fragmen teks berdasarkan geometri PDF.

Tabel IV.12: Profil eksperimen *legal parser* jalur OCR

Profil	Model OCR	Layout	Region yang dikecualikan
glm-ocr	GLM-OCR	Tidak ada	Tidak ada
glm-ocr-pp-doclayout-v3	GLM-OCR	PP-DocLayoutV3	chart, image, seal
paddle-ocr	PaddleOCR-VL 1.6	Tidak ada	Tidak ada
paddle-ocr-pp-doclayout-v3	PaddleOCR-VL 1.6	PP-DocLayoutV3	chart, image, seal

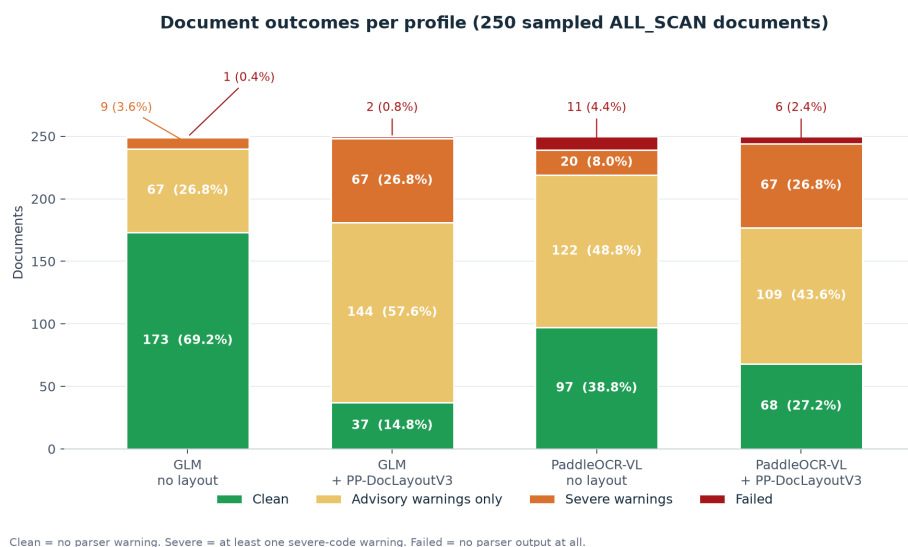
IV.3.3.3 Hasil Keutuhan Struktur

Profil `glm-ocr` tanpa deteksi *layout* menghasilkan proporsi dokumen bersih tertinggi, yaitu 173 dari 250 dokumen atau 69,2%. Profil tersebut juga menghasilkan sembilan dokumen *severe* dan satu kegagalan. Hasil keempat profil dirangkum pada Tabel IV.13.

Penambahan PP-DocLayoutV3 menurunkan *clean rate* pada kedua model OCR. Pada GLM-OCR, *clean rate* turun dari 69,2% menjadi 14,8%, sedangkan pada PaddleOCR-VL 1.6 turun dari 38,8% menjadi 27,2%. Peringatan `sequence_gap`

Tabel IV.13: Hasil keutuhan struktur pada eksperimen *legal parser* jalur OCR

Profil	Bersih	Advisory	Severe	Gagal	Clean rate	Failure rate
glm-ocr	173	67	9	1	69,2%	0,4%
glm-ocr-pp-doclayout-v3	37	144	67	2	14,8%	0,8%
paddle-ocr	97	122	20	11	38,8%	4,4%
paddle-ocr-pp-doclayout-v3	68	109	67	6	27,2%	2,4%



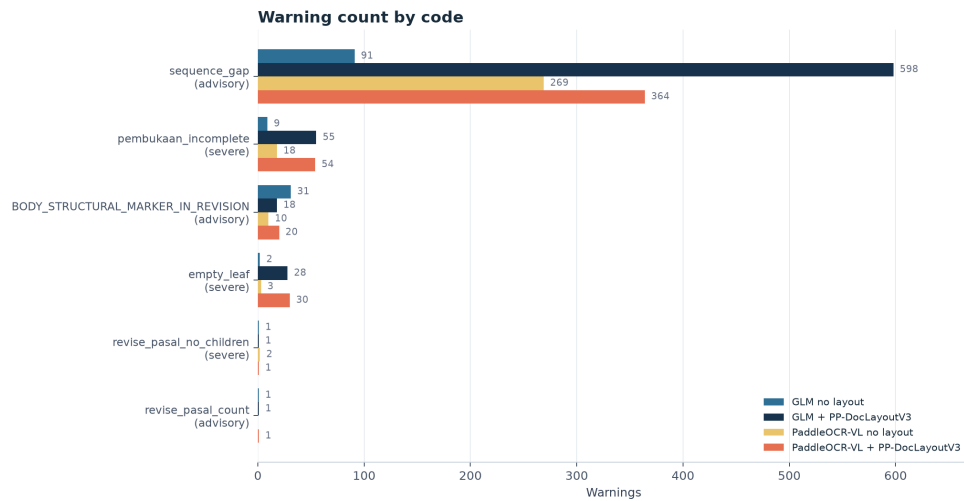
Gambar IV.17: Komposisi hasil pemrosesan pada eksperimen *legal parser* jalur OCR

juga meningkat pada GLM-OCR dari 91 menjadi 598 dan pada PaddleOCR dari 269 menjadi 364 ketika deteksi *layout* diaktifkan. Pola tersebut menunjukkan bahwa peningkatan pemrosesan berbasis region tidak otomatis menghasilkan urutan baca yang lebih sesuai untuk struktur peraturan.

IV.3.3.4 Biaya Pemrosesan dan Kegagalan OCR

PaddleOCR tanpa deteksi *layout* memiliki waktu median terendah, yaitu 3,87 detik per halaman, dengan *throughput* 1,69 dokumen per menit dan 0,240 halaman per detik. GLM-OCR tanpa deteksi *layout* membutuhkan 9,54 detik per halaman dan menghasilkan 0,68 dokumen per menit. Namun, deteksi *layout* mempercepat GLM-OCR menjadi 5,49 detik per halaman, sementara pada PaddleOCR justru memperlambatnya menjadi 6,32 detik per halaman.

Deteksi *layout* mengganti satu permintaan OCR halaman penuh menjadi sekitar 17 permintaan *region* per halaman. Karena itu, dampaknya terhadap waktu bergantung pada model OCR dan biaya penjadwalan region.

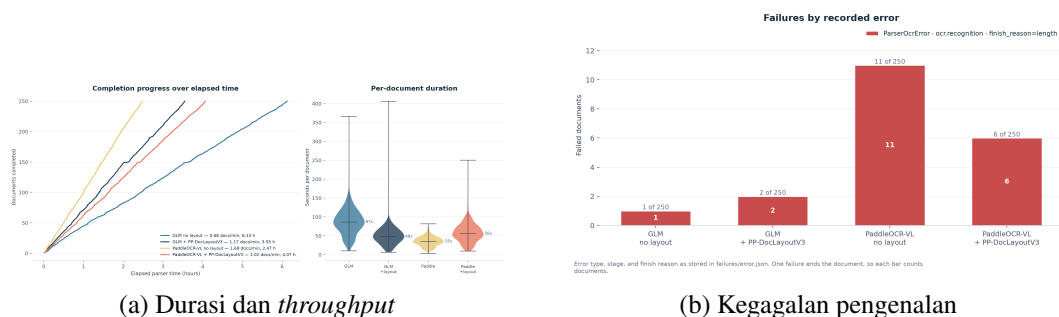


Gambar IV.18: Peringatan *parser* pada jalur OCR

Tabel IV.14: Biaya pemrosesan eksperimen *legal parser* jalur OCR

Profil	Median detik/halaman	Dokumen/menit	Halaman/detik	Permintaan OCR/halaman
glm-ocr	9,54	0,68	0,101	1,00
glm-ocr-pp-doclayout-v3	5,49	1,17	0,174	17,15
paddle-ocr	3,87	1,69	0,240	1,00
paddle-ocr-pp-doclayout-v3	6,32	1,02	0,149	17,20

Seluruh 20 kegagalan pada keempat profil terjadi pada tahap `ocr.recognition` ketika keluaran mencapai batas 4.096 token. Jumlah kegagalan berturut-turut adalah satu pada GLM-OCR, dua pada GLM-OCR dengan *layout*, sebelas pada PaddleOCR, dan enam pada PaddleOCR dengan *layout*. Kegagalan tersebut tidak dianggap dapat diselesaikan dengan pengulangan biasa karena decoding yang deterministik akan mencapai batas yang sama.



Gambar IV.19: Biaya dan kegagalan pada eksperimen *legal parser* jalur OCR

IV.3.3.5 Analisis dan Kesimpulan Eksperimen

Berdasarkan keutuhan struktur, *glm-ocr* tanpa *layout* dipilih sebagai konfigurasi operasional karena menghasilkan *clean rate* tertinggi dan jumlah kegagalan paling rendah. PP-DocLayoutV3 menurunkan *clean rate* pada kedua model, meskipun mempercepat GLM-OCR dari 9,54 menjadi 5,49 detik per halaman dan memperlambat PaddleOCR dari 3,87 menjadi 6,32 detik per halaman. PaddleOCR tanpa *layout* merupakan konfigurasi tercepat, tetapi mutu struktur yang dihasilkan lebih rendah. Dengan demikian, eksperimen memilih GLM-OCR tanpa *layout* untuk prioritas keutuhan struktur dan mencatat batas 4.096 token sebagai penyebab seluruh kegagalan OCR.

IV.4 Evaluasi *Generic Outline Parser* pada Domain *Customer Service*

Evaluasi ini menguji *generic outline parser* pada manual produk NETGEAR yang digunakan sebagai sumber pengetahuan domain *customer service*. Berbeda dari eksperimen *legal parser*, evaluasi ini merupakan *coverage check* dengan satu konfigurasi default dan tidak membandingkan beberapa model atau nilai parameter.

IV.4.1 Tujuan dan Pertanyaan Evaluasi

Evaluasi bertujuan menilai apakah *bookmark outline* pada manual produk dapat dipetakan menjadi hierarki dokumen kanonik. Evaluasi juga memeriksa keberhasilan penautan setiap *bookmark* ke baris judul dan kemampuan parser menghasilkan keluaran tanpa peringatan atau kegagalan.

Pertanyaan evaluasi yang dijawab adalah sebagai berikut.

1. Apakah seluruh manual NETGEAR pada *datasource* dapat diproses oleh *generic outline parser*?
2. Apakah setiap *bookmark* dapat ditautkan ke judul bagian yang sesuai?
3. Apakah struktur *bookmark* dapat dimaterialisasi menjadi hierarki blok kanonik?

4. Apakah bagian yang dikecualikan seperti *Contents* tidak mengganggu pembentukan struktur?

IV.4.2 Rancangan Evaluasi

Evaluasi menggunakan seluruh 9 dokumen aktif pada *datasource netgear_products*, yang mencakup 1.400 halaman dan 1.566 *bookmark*. Dokumen diurutkan berdasarkan nama berkas sehingga pengulangan evaluasi menggunakan himpunan yang sama tanpa seed dan tanpa pengambilan sampel. Seluruh dokumen diproses dengan satu konfigurasi default.

Tabel IV.15: Konfigurasi evaluasi *generic outline parser*

Parameter	Nilai	Peran
<code>header_margin_ratio</code>	0,075	Proporsi tinggi halaman yang diperlakukan sebagai <i>header</i>
<code>footer_margin_ratio</code>	0,90	Batas awal area <i>footer</i>
<code>anchor_search_window_points</code>	150 pt	Jendela pencarian judul di sekitar tujuan <i>bookmark</i>
<code>line_merge_tolerance_points</code>	4 pt	Toleransi penggabungan fragmen teks menjadi satu baris
<code>text_decoration</code>	<code>true</code>	Mempertahankan dekorasi teks sebagai <i>markdown</i>
<code>excluded_section_titles</code>	<code>contents,</code> <code>tableofcontents</code>	Bagian yang dibuang beserta turunannya dan dicatat sebagai <i>metadata</i>

Bookmark menjadi sumber hierarki karena manual produk tidak memiliki aturan penomoran domain yang dapat digunakan untuk menyimpulkan tingkat judul. Judul *bookmark* dicari pada jendela baris di sekitar tujuan *bookmark*. Jika pencocokan judul tidak ditemukan, parser menggunakan baris pertama pada atau setelah tujuan sebagai *fallback* dan mencatat peringatan sesuai kode yang berlaku. PDF tanpa *outline* atau tanpa teks yang dapat diekstraksi ditolak sebagai kegagalan sumber. Parser tidak menggunakan OCR atau deteksi *layout*, dan tabel tidak dimodelkan sebagai struktur khusus.

IV.4.3 Hasil Pemrosesan

Seluruh 9 manual berhasil diproses sebagai dokumen bersih. Tidak terdapat dokumen dengan peringatan, tidak ada dokumen gagal, dan tidak ada dokumen yang ditolak karena *outline* atau teks tidak tersedia. Ringkasan hasil pemrosesan ditampilkan pada Tabel IV.16.

Tabel IV.16: Ringkasan hasil evaluasi *generic outline parser*

Metrik	Nilai
Dokumen diproses	9
Dokumen bersih	9
Dokumen dengan peringatan	0
Dokumen gagal	0
Halaman	1.400
<i>Bookmark</i>	1.566
<i>Heading</i> terbentuk	1.557
<i>Anchor title-match</i>	1.566/1.566 (100%)
Durasi total	7,6 detik

Sebanyak 1.566 *bookmark* menghasilkan 1.557 *heading*. Selisih sembilan *heading* berasal dari bagian *Contents* yang dikecualikan pada masing-masing manual. Bagian yang dikecualikan disimpan dalam *metadata excluded_sections* dan tidak dihitung sebagai peringatan.

Perincian setiap manual ditampilkan pada Tabel IV.17.

Tabel IV.17: Hasil per dokumen pada evaluasi *generic outline parser*

Dokumen	Halaman	<i>Bookmark</i>	<i>Heading</i>	<i>Text block</i>	<i>Title-match</i>	Detik
netgear-cm2000.pdf	26	29	28	29	100%	0,15
netgear-lm1200.pdf	105	105	104	105	100%	0,50
netgear-r6350.pdf	204	226	225	226	100%	1,08
netgear-r7000.pdf	186	213	212	213	100%	0,97
netgear-rax120.pdf	175	202	201	202	100%	1,08
netgear-rax50.pdf	160	185	184	185	100%	0,87
netgear-raxe500.pdf	169	192	191	192	100%	0,93
netgear-rbk852.pdf	161	183	182	183	100%	0,85
netgear-xr500.pdf	214	231	230	231	100%	1,17
Total	1.400	1.566	1.557	1.566	100%	7,60

Jumlah *text block* melebihi jumlah *heading* sebanyak satu pada setiap dokumen. Blok tambahan tersebut merupakan materi awal dokumen sebelum *bookmark* pertama dan disimpan sebagai *front matter*.

IV.4.4 Analisis Anchoring, Kinerja, dan Kesimpulan

Seluruh 1.566 *bookmark* berhasil dicocokkan dengan judulnya. Hasil ini mendukung strategi pencarian judul pada jendela di sekitar tujuan *bookmark*, bukan sekadar mengambil baris pertama setelah tujuan. Pada konfigurasi ini, seluruh dokumen juga lolos pemeriksaan struktur tanpa *empty node* atau rentang halaman yang tidak valid.

Evaluasi selesai dalam 7,6 detik, dengan *throughput* 71 dokumen per menit dan 184 halaman per detik. Median waktu pemrosesan adalah 0,93 detik per dokumen, sedangkan biaya tambahan berdasarkan jumlah halaman adalah sekitar 5,4 ms per halaman.

Dengan demikian, *generic outline parser* berhasil memulihkan hierarki manual produk menjadi dokumen kanonik pada seluruh 9 dokumen yang diuji. Anchoring *bookmark* ke judul berhasil pada seluruh 1.566 *bookmark*. Pengecualian bagian *Contents* bekerja sesuai konfigurasi dan tidak diperlakukan sebagai cacat struktur. Hasil ini mendukung penggunaan kontrak *parser plugin* yang sama untuk domain *customer service* dengan strategi ekstraksi berbasis *outline*. Evaluasi ini tidak digeneralisasikan ke seluruh PDF ber-*bookmark* karena korpus hanya berasal dari satu produsen dan satu *toolchain*.

BAB V

KESIMPULAN DAN SARAN

Bab ini merangkum hasil pengembangan dan evaluasi komponen akuisisi serta ekstraksi dokumen pada sistem OMNI-RAG. Kesimpulan disusun untuk menjawab dua rumusan masalah pada Subbab I.2 dan dikaitkan dengan ukuran keberhasilan pada Subbab I.3.

V.1 Kesimpulan

Berdasarkan analisis masalah, rancangan solusi, dan hasil evaluasi pada Bab IV, diperoleh kesimpulan sebagai berikut.

1. **Akuisisi dan penyimpanan dokumen dari sumber eksternal.** Komponen akuisisi berhasil menyediakan alur bersama untuk portal peraturan BPK dan Google Drive melalui kontrak *scraper plugin* yang memisahkan proses enumerasi dan pengunduhan dari proses *ingestion* inti. Komponen inti menangani validasi tipe berkas, pengaturan laju permintaan, *concurrency* terbatas, kebijakan penghentian, pencatatan *lifecycle*, penyimpanan, dan deduplikasi tanpa bergantung pada cara sumber menyediakan dokumen. Pada enam konfigurasi eksperimen BPK, setiap *run* menyelesaikan enumerasi 8.822 entri dan menghasilkan 8.745 dokumen baru. *Throughput* dokumen baru berada pada rentang 618,5–4.990,5 dokumen per jam, dengan konfigurasi *bpk-05-high-stress* sebagai konfigurasi tercepat yang diamati. Konfigurasi operasional yang direkomendasikan adalah *download_concurrency=5* dengan jeda enumerasi dan pengunduhan 1–3 detik karena memberikan *trade-off* yang lebih konservatif, dengan durasi 3,58 jam dan *throughput* 2.444,2 dokumen baru per jam. Pada konfigurasi *non-baseline* terdapat 15 *failure*, sedangkan konfigurasi *baseline* memiliki 16 *failure* akibat satu perbedaan hasil percobaan ulang. Kegagalan yang berulang terutama berupa HTTP 404 dan tautan unduh yang tidak tersedia, serta tidak menghentikan pemrosesan entri lain atau keseluruhan *run*. Pemeriksaan *content hash* menemukan 19 kluster

sumber dengan isi berkas identik dan memastikan hanya satu salinan *payload* yang disimpan untuk setiap klaster. Evaluasi integrasi Google Drive terbaru memproses 11 entri dengan hasil 9 dokumen baru, 1 *skip* duplikat, 1 *skip* tipe berkas, dan 0 *failure*. Hasil tersebut menunjukkan bahwa penambahan sumber baru dapat dilakukan melalui *plugin* dan konfigurasi sumber, sedangkan orkestrasi, penyimpanan, deduplikasi, dan pencatatan status tetap digunakan bersama. Dengan demikian, rumusan masalah pertama terjawab dalam batas sumber dan kondisi eksperimen yang ditetapkan, sementara konfigurasi yang disebut optimal harus dipahami sebagai konfigurasi terbaik atau konfigurasi operasional terbaik di antara konfigurasi yang diuji, bukan sebagai optimum universal untuk semua sumber.

2. **Ekstraksi dokumen menjadi representasi hierarkis pada berbagai domain.** Komponen ekstraksi berhasil menyediakan kontrak *parser plugin* dengan keluaran berupa dokumen kanonik berbentuk hierarki, sehingga perbedaan strategi ekstraksi dapat diselesaikan di dalam *plugin* sebelum hasil diteruskan ke tahap berikutnya. Pada eksperimen classifier halaman, ambang cakupan citra 0,50 dan teks *native* minimum 100 karakter menghasilkan akurasi 98,8%, *SCAN recall* 100,0%, dan *macro F1* 98,8% pada sampel berlabel. Keputusan pada tingkat halaman memungkinkan dokumen campuran ditangani dengan jalur *native* dan OCR yang berbeda. Pada korpus ALL_VECTOR, seluruh 48 konfigurasi berhasil menghasilkan keluaran tanpa kegagalan terminal pada 214.032 pemrosesan dokumen. Profil y6-x0.5 dipilih sebagai konfigurasi operasional karena menghasilkan *clean rate* 72,1% dan *throughput* 552 dokumen per menit. Pada sampel korpus ALL_SCAN, GLM-OCR tanpa deteksi *layout* menghasilkan *clean rate* tertinggi sebesar 69,2% dengan *failure rate* 0,4%. Penambahan PP-DocLayoutV3 menurunkan *clean rate* pada kedua model yang diuji, sedangkan seluruh kegagalan OCR berasal dari keluaran yang mencapai batas 4.096 *token*. Pada domain *customer service*, *generic outline parser* memproses seluruh 9 manual sebagai dokumen bersih, mencocokkan 1.566 dari 1.566 *bookmark* dengan judul bagian, dan tidak menghasilkan peringatan maupun kegagalan. Hasil tersebut menunjukkan bahwa dua domain yang diuji dapat menggunakan kontrak *parser* dan representasi

kanonik yang sama meskipun satu domain memulihkan struktur dari penanda hukum dan domain lainnya dari *document outline*. Dengan demikian, rumusan masalah kedua terjawab untuk domain dan korpus yang diuji, tetapi keutuhan struktur yang diukur melalui *clean rate*, peringatan, dan kegagalan belum dapat dianggap sebagai jaminan keutuhan semantik untuk seluruh dokumen hukum atau seluruh PDF ber-*outline*.

V.2 Saran

Saran pengembangan berikut diturunkan dari batasan masalah dan keterbatasan evaluasi yang telah diuraikan pada bab-bab sebelumnya.

1. **Memperluas evaluasi akuisisi.** Eksperimen *scraper* dapat diulang pada waktu, jaringan, mesin, dan ukuran korpus yang berbeda serta dilengkapi sumber eksternal lain agar pengaruh *concurrency*, jeda permintaan, batas laju, dan kondisi sumber dapat dinilai dengan lebih umum. Evaluasi Google Drive juga perlu dikembangkan dari *sanity check* menjadi pengujian dengan folder yang lebih besar dan variasi kegagalan API.
2. **Mengembangkan pengelolaan perubahan dokumen.** Sistem dapat ditambahkan mekanisme untuk mendeteksi pembaruan isi, menyimpan versi dokumen, dan menandai dokumen yang telah dihapus dari sumber. Kebijakan pemilihan *winner* pada kluster duplikasi juga dapat dibuat deterministik apabila kestabilan identitas sumber yang disimpan diperlukan.
3. **Meningkatkan ketahanan jalur OCR.** Kegagalan akibat batas 4.096 *token* dapat ditangani melalui strategi pemotongan halaman, penggabungan beberapa permintaan, konfigurasi batas keluaran yang adaptif, atau mekanisme *fallback* ke model lain. Model OCR dan deteksi *layout* juga perlu diuji kembali pada korpus yang lebih beragam sebelum konfigurasi dipindahkan ke lingkungan produksi.
4. **Menambahkan acuan mutu ekstraksi.** Evaluasi selanjutnya sebaiknya menyediakan transkripsi acuan dan anotasi struktur hukum agar mutu OCR dan keutuhan hierarki dapat diukur secara langsung, bukan hanya melalui *clean*

rate, peringatan, dan kegagalan parser. Pengujian tersebut perlu mencakup dokumen MIXED, dokumen hasil pindai yang lebih panjang, serta variasi tabel dan tata letak yang belum tercakup dalam benchmark saat ini.

5. **Memperluas cakupan *generic outline parser*.** Parser perlu diuji pada manual dari lebih banyak produsen dan *toolchain*, PDF tanpa *outline*, serta dokumen yang mengandung tabel, gambar, dan catatan kaki. Untuk dokumen tanpa sumber hierarki yang memadai, dapat dipertimbangkan strategi parser alternatif berbasis *layout* atau aturan domain.
6. **Menguji kesiapan operasional dan skala produksi.** Evaluasi berikutnya perlu mengukur penggunaan CPU, memori, GPU, jaringan, dan *storage* pada deployment multi-mesin serta menambahkan metrik observabilitas untuk memantau antrean, waktu tiap tahap, kegagalan, dan perubahan kualitas dokumen dari waktu ke waktu.

DAFTAR PUSTAKA

- Adhikari, N. S., & Agarwal, S. (2024). *A Comparative Study of PDF Parsing Tools Across Diverse Document Categories*. arXiv: 2410.09871 [cs.IR]. <https://arxiv.org/abs/2410.09871>
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd edition). Pearson Addison-Wesley.
- Apple Machine Learning Research. (2026). *MLX Documentation*. Diakses pada Agustus 21, 2026, dari <https://ml-explore.github.io/mlx/>
- Artifex Software. (2026). *TextPage — PyMuPDF Documentation*. Diakses pada Agustus 21, 2026, dari <https://pymupdf.readthedocs.io/en/latest/textpage.html>
- Barrios, W. (2026). *Native LLM and MLLM Inference at Scale on Apple Silicon*. arXiv: 2601.19139 [cs.DC]. <https://arxiv.org/abs/2601.19139>
- Behnel, S., Faassen, M., & Bicking, I. (2025). *lxml: XML and HTML with Python*. Diakses pada Agustus 19, 2026, dari <https://lxml.de/>
- Birsan, D. (2005). On Plug-ins and Extensible Architectures. *ACM Queue*, 3(2), 40–46. <https://doi.org/10.1145/1053331.1053345>
- Brown, M. A., Gruen, A., Maldoff, G., Messing, S., Sanderson, Z., & Zimmer, M. (2024). *Web Scraping for Research: Legal, Ethical, Institutional, and Scientific Considerations*. arXiv: 2410.23432 [cs.CY]. <https://arxiv.org/abs/2410.23432>
- Cui, C., Gao, T., Wang, X., Zhou, C., Liu, H., Sun, T., Zhang, Y., Zhang, Z., Liu, J., Lin, M., Zhang, Y., Liang, S., Xiang, Y., & Liu, Y. (2026). *RT-DocLayout: Real-Time End-to-End Document Layout Analysis with Reading Order in the Wild*. arXiv: 2606.23344 [cs.CV]. <https://arxiv.org/abs/2606.23344>
- Duan, S., dkk. (2026). *GLM-OCR Technical Report*. arXiv: 2603.10910 [cs.CV]. <https://arxiv.org/abs/2603.10910>
- Encode OSS. (2025). *HTTPX: A Next Generation HTTP Client for Python*. Diakses pada Agustus 19, 2026, dari <https://www.python-httpx.org/>

- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv: 2312.10997 [cs .CL]. <https://arxiv.org/abs/2312.10997>
- Gao, Y., Xiong, Y., Wang, M., & Wang, H. (2024). *Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks*. arXiv: 2407 . 21059 [cs .CL]. <https://arxiv.org/abs/2407.21059>
- Garlan, D., & Shaw, M. (1994). *An Introduction to Software Architecture* (techreport **number** Technical Report CMU-CS-94-166). School of Computer Science, Carnegie Mellon University. Pittsburgh, PA.
- Google. (2025). *Google Drive API v3 Documentation*. Diakses pada Agustus 19, 2026, dari <https://developers.google.com/workspace/drive/api/reference/rest/v3>
- Google. (2026). *Usage Limits: Google Drive API*. Diakses pada Agustus 21, 2026, dari <https://developers.google.com/workspace/drive/api/guides/limits>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd edition). Pearson Addison-Wesley.
- International Organization for Standardization. (2020). *Document Management — Portable Document Format — Part 2: PDF 2.0* (techreport **number** ISO 32000-2:2020). International Organization for Standardization. Geneva.
- Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). OCR-free Document Understanding Transformer. *Computer Vision – ECCV 2022*.
- Kolobov, A., Peres, Y., Lubetzky, E., & Horvitz, E. (2019). Optimal Freshness Crawl Under Politeness Constraints. *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- Koster, M., Illyes, G., Zeller, H., & Sassman, L. (2022, September). *Robots Exclusion Protocol* (techreport **number** Request for Comments RFC 9309). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9309>

- Krotov, V., Johnson, L., & Silva, L. (2020). Tutorial: Legality and Ethics of Web Scraping. *Communications of the Association for Information Systems*, 47(1). <https://doi.org/10.17705/1CAIS.04724>
- Livathinos, N., Auer, C., Lysak, M., Nassar, A., Dolfi, M., Vagenas, P., Berrospi Ramis, C., Omenetti, M., Dinkla, K., Kim, Y., Gupta, S., Teixeira de Lima, R., Weber, V., Morin, L., Meijer, I., Kuropiatnyk, V., & Staar, P. W. J. (2025). *Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion*. arXiv: 2501.17887 [cs . CL] . <https://arxiv.org/abs/2501.17887>
- Ma, J., Du, J., Hu, P., Zhang, Z., Zhang, J., Zhu, H., & Liu, C. (2023). HRDoc: Dataset and Baseline Method toward Hierarchical Reconstruction of Document Structures. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(2), 1870–1877. <https://doi.org/10.1609/aaai.v37i2.25277>
- Meyer, D. T., & Bolosky, W. J. (2011). A Study of Practical Deduplication. *Proceedings of the 9th USENIX Conference on File and Storage Technologies (FAST '11)*.
- Misrahi, A., Chirkova, N., Louis, M., & Nikoulina, V. (2025). *Adapting Large Language Models for Multi-Domain Retrieval-Augmented-Generation*. arXiv: 2504.02411 [cs . CL] . <https://arxiv.org/abs/2504.02411>
- National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)* (techreport **number** Federal Information Processing Standards Publication FIPS PUB 180-4). National Institute of Standards dan Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>
- NETGEAR. (2019). *Nighthawk AC1900 Smart WiFi Router Model R7000: User Manual*. https://www.downloads.netgear.com/files/GDC/R7000/R7000_UM.pdf
- NETGEAR. (2023). *4-Stream AX1800 WiFi 6 Router Model RAX10: User Manual*. https://www.downloads.netgear.com/files/GDC/RAX10/RAX10_UM_EN.pdf
- NETGEAR. (2026). *NETGEAR Support: Downloads*. Diakses pada Agustus 21, 2026, dari <https://www.netgear.com/support/home/downloads>
- Niu, J., Liu, Z., Gu, Z., Wang, B., Ouyang, L., Zhao, Z., Chu, T., He, T., dkk. (2025). *MinerU2.5: A Decoupled Vision-Language Model for Efficient High-Resolution*

- Document Parsing*. arXiv: 2509.22186 [cs.CV]. <https://arxiv.org/abs/2509.22186>
- Ollama. (2026a). *Hardware Support*. Diakses pada Agustus 21, 2026, dari <https://docs.ollama.com/gpu>
- Ollama. (2026b). *OpenAI Compatibility*. Diakses pada Agustus 21, 2026, dari <https://docs.ollama.com/api/openai-compatibility>
- Olston, C., & Najork, M. (2010). Web Crawling. *Foundations and Trends in Information Retrieval*, 4(3), 175–246. <https://doi.org/10.1561/15000000017>
- oMLX Contributors. (2026). *oMLX: Inference Server for Apple Silicon*. Diakses pada Agustus 21, 2026, dari <https://github.com/jundot/omlx>
- Ouyang, L., dkk. (2025). OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Packowski, S., Halilovic, I., Schlotfeldt, J., & Smith, T. (2024). Optimizing and Evaluating Enterprise Retrieval-Augmented Generation (RAG): A Content Design Perspective. *Proceedings of the 8th International Conference on Advances in Artificial Intelligence*, 162–167. <https://doi.org/10.1145/3704137.3704181>
- PaddlePaddle. (2026). *PP-DocLayoutV3*. Diakses pada Agustus 19, 2026, dari <https://huggingface.co/PaddlePaddle/PP-DocLayoutV3>
- Palmirani, M., Sperberg, R., Vergottini, G., & Vitali, F. (2018). *Akoma Ntoso Version 1.0. Part 1: XML Vocabulary* (OASIS Standard). OASIS Open. <https://docs.oasis-open.org/legaldocml/akn-core/v1.0/os/part1-vocabulary/akn-core-v1.0-os-part1-vocabulary.html>
- Pfitzmann, B., Auer, C., Dolfi, M., Nassar, A. S., & Staar, P. W. J. (2022). DocLayoutNet: A Large Human-Annotated Dataset for Document-Layout Analysis. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/3534678.3539043>
- py-pdf. (2025). *Benchmarking PDF Libraries*. Diakses pada Agustus 19, 2026, dari <https://github.com/py-pdf/benchmarks>
- Rausch, J., Martinez, O., Bissig, F., Zhang, C., & Feuerriegel, S. (2021). DocParser: Hierarchical Document Structure Parsing from Renderings. *Proceedings of*

- the AAAI Conference on Artificial Intelligence*, 35(5), 4328–4338. <https://doi.org/10.1609/aaai.v35i5.16558>
- Undang-Undang Republik Indonesia Nomor 12 Tahun 2011 tentang Pembentukan Peraturan Perundang-undangan, Jakarta (2011). <https://peraturan.bpk.go.id/Details/39188/uu-no-12-tahun-2011>
- Undang-Undang Republik Indonesia Nomor 13 Tahun 2022 tentang Perubahan Kedua atas Undang-Undang Nomor 12 Tahun 2011 tentang Pembentukan Peraturan Perundang-undangan, Jakarta (2022). <https://peraturan.bpk.go.id/Details/212810/uu-no-13-tahun-2022>
- Undang-Undang Republik Indonesia Nomor 1 Tahun 2026 tentang Penyesuaian Pidana, Jakarta (2026). <https://peraturan.bpk.go.id/Details/337869/uu-no-1-tahun-2026>
- Richardson, L. (2025). *Beautiful Soup Documentation*. Diakses pada Agustus 19, 2026, dari <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Sallinen, A., dkk. (2025). *MMORE: Massive Multimodal Open RAG and Extraction*. arXiv: 2509.11937 [cs.CL]. <https://arxiv.org/abs/2509.11937>
- Shaukat, M. A., Adnan, M., & Kuhn, C. C. N. (2026). *A Systematic Investigation of Document Chunking Strategies and Embedding Sensitivity*. arXiv: 2603.06976 [cs.CL]. <https://arxiv.org/abs/2603.06976>
- Sun, L., Dexian, W., Huang, J., Zhang, L., Jia, C., Cheng, Z., & Zhang, X. (2026). *When Good OCR Is Not Enough: Benchmarking OCR Robustness for Retrieval-Augmented Generation*. arXiv: 2605.00911 [cs.CL]. <https://arxiv.org/abs/2605.00911>
- Tripathi, V., Odapally, T., Das, I., Allu, U., & Ahmed, B. (2025). *Vision-Guided Chunking Is All You Need: Enhancing RAG with Multimodal Document Understanding*. arXiv: 2506.16035 [cs.LG]. <https://arxiv.org/abs/2506.16035>
- Turski, M., Stanisławek, T., Kaczmarek, K., Dyda, P., & Graliński, F. (2023). CCpdf: Building a High Quality Corpus for Visually Rich Documents from Web Crawl Data. *Document Analysis and Recognition – ICDAR 2023*.
- Wang, Z., Xu, Y., Cui, L., Shang, J., & Wei, F. (2021). LayoutReader: Pre-training of Text and Layout for Reading Order Detection. *Proceedings of the 2021*

Conference on Empirical Methods in Natural Language Processing (EMNLP).

Wei, H., Liu, C., Chen, J., Wang, J., Kong, L., Xu, Y., Ge, Z., Zhao, L., Sun, J., Peng, Y., Han, C., & Zhang, X. (2024). *General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model*. arXiv: 2409.01704 [cs.CV]. <https://arxiv.org/abs/2409.01704>

Zhang, L., Hu, A., Zhang, J., Hu, S., & Jin, Q. (2023). MPMQA: Multimodal Question Answering on Product Manuals. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11), 13958–13966. <https://doi.org/10.1609/aaai.v37i11.26634>

Zhang, Z., Liu, H., Liang, S., Zhang, Y., Xiang, Y., Liu, J., Sun, T., Lin, M., Zhang, Y., Zhou, C., Gao, T., Cui, C., Liu, Y., Yu, D., & Ma, Y. (2026). *PaddleOCR-VL-1.6: Expanding the Frontier of Document Parsing with Under-Optimized Region Refinement and Progressive Post-Training*. arXiv: 2606.03264 [cs.CV]. <https://arxiv.org/abs/2606.03264>

Zhong, X., Tang, J., & Jimeno Yepes, A. (2019). PubLayNet: Largest Dataset Ever for Document Layout Analysis. *2019 International Conference on Document Analysis and Recognition (ICDAR)*, 1015–1022. <https://doi.org/10.1109/ICDAR.2019.00166>